RĪGAS METĀLAPSTRĀDES PROFESIONĀLĀ VIDUSSKOLA

Datorzinību metodiskā komisija

Kvalifikācijas eksāmena praktiskās daļas

tehniskā dokumentācija

“Sociālā tīkla tīmekļa lietotne Osmos”

Darba autors: G. Siliņš

9. grupa

_____ (paraksts)

Rīga, 2026

ANOTĀCIJA

Kvalifikācijas eksāmena praktiskās daļas darba tēma ir “Sociālā tīkla tīmekļa lietotne Osmos”. Darba mērķis ir izstrādāt sociālā tīkla tīmekļa lietotni, kas paredzēta vienas klases, kursa vai skolas audzēkņu savstarpējai saziņai.

Sistēmā lietotāji var izveidot profilu, kļūt par draugiem, publicēt ierakstus ar attēliem un video, komentēt tos, atzīmēt ar “patīk”, sarakstīties privāti, apvienoties grupās un meklēt ierakstus pēc tēmturiem. Katrs lietotājs pats izvēlas, vai viņa profila saturu redz visi tīkla lietotāji vai tikai draugi. Lietotājvārdu pārbaude, lietotāju atzīmes un tēmturu atpazīšana ir realizēta ar regulārajām izteiksmēm. Sistēmas administrators var bloķēt lietotāju kontus, piešķirt administratora tiesības, atiestatīt aizmirstas paroles un dzēst nepiemērotu saturu. Sistēma izstrādāta PHP valodā, dati tiek glabāti MariaDB datubāzē, un tā darbojas XAMPP vidē.

Dokumentācijā ir aprakstīts uzdevums, sistēmas ieejas un izejas informācija, funkcionālās un nefunkcionālās prasības, izstrādes līdzekļu izvēles pamatojums, sistēmas arhitektūra, ER modelis, lietojumgadījumu un datu plūsmu diagrammas, datubāzes tabulu struktūra, lietotāja ceļvedis un testēšanas rezultāti. Darba noslēgumā ir secinājumi par paveikto un sistēmas turpmākās attīstības iespējām.

Darbā ir 85 lappuses, 31 attēls, 36 tabulas un 1 pielikums. Pielikumā ir sistēmas pirmkods.

ANNOTATION

The topic of the practical part of the qualification exam is “Osmos, a social network web application”. The aim of the work is to develop a social network web application for communication between the students of one class, course or school.

In the system, users can create a profile, become friends, publish posts with images and videos, comment on them, like them, send private messages, join groups and search for posts by hashtags. Each user decides whether the content of their profile is visible to all users of the network or to friends only. Usernames are validated and user mentions and hashtags are recognised with regular expressions. The system administrator can block user accounts, grant administrator rights, reset forgotten passwords and delete inappropriate content. The system is written in PHP, the data is stored in a MariaDB database, and it runs in the XAMPP environment.

The documentation describes the task, the input and output information of the system, the functional and non-functional requirements, the reasons for the choice of development tools, the system architecture, the ER model, the use case and data flow diagrams, the structure of the database tables, the user guide and the test results. The work ends with conclusions on the results and on the further development of the system.

The work contains 85 pages, 31 figures, 36 tables and 1 appendix. The appendix contains the source code of the system.

SATURS

Ievads..... 6
1. Uzdevuma nostādne..... 8
2. Prasību specifikācija..... 10
2.1. Ieejas un izejas informācijas apraksts..... 10
2.1.1. Ieejas informācijas apraksts..... 10
2.1.2. Izejas informācijas apraksts..... 11
2.2. Funkcionālās prasības..... 12
2.3. Nefunkcionālās prasības..... 19
2.3.1. Izstrādes prasības..... 19
2.3.2. Saskarnes prasības..... 19
2.3.3. Programmatūras prasības..... 19
2.3.4. Aparatūras prasības..... 19
2.3.5. Drošības prasības..... 20
3. Uzdevuma risināšanas līdzekļu izvēles pamatojums..... 21
4. Programmatūras produkta modelēšana un projektēšana..... 23
4.1. Sistēmas struktūras modelis..... 23
4.1.1. Sistēmas arhitektūra..... 23
4.1.2. Sistēmas ER modelis..... 27
4.2. Funkcionālais sistēmas modelis..... 29
4.2.1. Lietojumgadījumu diagramma..... 29
4.2.2. Datu plūsmu modelis..... 30
4.2.3. Ieraksta redzamības pārbaudes algoritms..... 33
4.2.4. Regulāro izteiksmju lietojums..... 34
5. Datu struktūru apraksts..... 36
6. Lietotāja ceļvedis..... 42
6.1. Sistēmas prasības aparatūrai un programmatūrai..... 42
6.2. Sistēmas instalācija un palaišana..... 42
6.3. Programmas apraksts..... 43
7. Programmatūras testēšana..... 55
7.1. Funkcionālā testēšana..... 55
7.2. Veiktspējas testēšana..... 61
Secinājumi..... 63
Informācijas avoti..... 65
Pielikumi..... 66
1. pielikums. Programmas pirmkods..... 67

SAĪSINĀJUMU SARAKSTS

Darbā lietoto terminu latviskās atbilsmes ir saskaņotas ar Latvijas Nacionālo terminoloģijas portālu [2]. Darbā izmantoti šādi saīsinājumi.

Saīsinājums	Pilns nosaukums	Skaidrojums
AJAX	Asynchronous JavaScript and XML	Datu apmaiņa ar serveri bez lapas pārlādes
CSRF	Cross-Site Request Forgery	Starpvietņu pieprasījumu viltošana
CSS	Cascading Style Sheets	Kaskādes stilu lapas, tīmekļa lapu noformējuma valoda
ER	Entity-Relationship	Entītiju un saišu modelis
FP		Funkcionālā prasība
GB	Gigabyte	Gigabaits
GD	Graphics Draw	PHP attēlu apstrādes bibliotēka
HTML	HyperText Markup Language	Hiperteksta iezīmēšanas valoda
HTTP	HyperText Transfer Protocol	Hiperteksta pārsūtīšanas protokols
IP	Internet Protocol	Interneta protokols
JPEG	Joint Photographic Experts Group	Attēlu saspiešanas formāts
JSON	JavaScript Object Notation	Teksta formāts datu apmaiņai
MP4	MPEG-4 Part 14	Video failu formāts
OWASP	Open Worldwide Application Security Project	Organizācija, kas pēta tīmekļa lietotņu drošību
PDF	Portable Document Format	Dokumentu formāts
PHP	PHP: Hypertext Preprocessor	Servera puses programmēšanas valoda
PNG	Portable Network Graphics	Attēlu formāts
SQL	Structured Query Language	Strukturētā vaicājumu valoda
UTF-8	Unicode Transformation Format, 8 bit	Unikoda rakstzīmju kodējums
XAMPP	Cross-platform, Apache, MariaDB, PHP, Perl	Bezmaksas tīmekļa izstrādes vides komplekts
XSS	Cross-Site Scripting	Starpvietņu skriptošana

IEVADS

Mūsdienās sociālie tīkli ir kļuvuši par neatņemamu sabiedrības ikdienas sastāvdaļu. Tos izmanto saziņai, informācijas apmaiņai, satura publicēšanai un kopienu veidošanai. Viena no populārākajām sociālo tīklu platformām pasaulē ir Facebook, kurā lietotāji publicē ierakstus, komentē citu lietotāju saturu, kopīgo fotoattēlus un video, sarakstās privāti un apvienojas grupās. Šo platformu izmanto gan personīgām vajadzībām, gan uzņēmējdarbībā un izglītībā.

Pilnvērtīgā Facebook versija ir funkcionāli sarežģīta un prasa daudz tehnisko resursu. Tajā ir liels skaits interaktīvu elementu, reklāmu un papildu funkciju, kas var palēnināt darbu ierīcēs ar zemāku veiktspēju vai lēnu interneta savienojumu. Šīs problēmas risināšanai ir izveidotas vienkāršotas versijas, piemēram, mobilā lietotne Facebook Lite, kas paredzēta mazāk jaudīgām ierīcēm un reģioniem ar ierobežotu interneta pieejamību. Tomēr šādu sistēmu iekšējā uzbūve nav publiski pieejama, un tās nav iespējams pielāgot konkrētas lietotāju grupas vajadzībām.

Mācību vidē vienas klases vai kursa audzēkņu saziņa bieži notiek vairākās lietotnēs vienlaikus. Lielās publiskās platformas apvieno mācību saziņu ar reklāmām, svešu cilvēku saturu un daudzām funkcijām, kas klases ikdienā nav vajadzīgas. Turklāt lietotāju dati tiek glabāti citu uzņēmumu serveros, un klase pati nevar noteikt, kādi dati tiek uzkrāti un kas tos redz. Tāpēc ir lietderīgi izveidot slēgtu sociālo tīklu, kurā reģistrējas tikai vienas klases vai skolas audzēkņi un kurā katrs lietotājs pats izlemj, vai viņa ierakstus redz visi tīkla dalībnieki vai tikai draugi.

Šāda sistēma ir piemērota arī mācību mērķiem, jo tās izstrāde aptver lielāko daļu tīmekļa programmēšanas pamatjautājumu: lietotāju reģistrāciju un autentifikāciju, sesiju pārvaldību, relāciju datubāzes projektēšanu, dinamiska satura attēlošanu un datu apmaiņu ar serveri bez lapas pārlādes. Sociālajā tīklā lielāko daļu satura ievada paši lietotāji, tāpēc izstrādē īpaša uzmanība jāpievērš drošībai. Sistēmas izstrādē tika ņemti vērā OWASP organizācijas apkopotie biežākie tīmekļa lietotņu drošības riski [9], piemēram, SQL injekcijas un starpvietņu skriptošana (XSS), kā arī OWASP ieteikumi aizsardzībai pret starpvietņu pieprasījumu viltošanu (CSRF) [8].

Sistēma tika izstrādāta lokālajā servera vidē XAMPP, kas apvieno Apache tīmekļa serveri, PHP programmēšanas valodu un MariaDB datubāzu pārvaldības sistēmu [3]. Servera daļa ir rakstīta PHP valodā, lietotāja saskarne veidota ar HTML un CSS, bet atsevišķas darbības, piemēram, “patīk” atzīmes pievienošana un lietotājvārdu ieteikumi, realizētas ar JavaScript. Dati tiek glabāti MariaDB datubāzē, kas ir saderīga ar MySQL. Klienta un servera arhitektūra

ļauj sistēmu palaist jebkurā datorā, kurā darbojas XAMPP, un lietot to ar jebkuru mūsdienīgu pārlūkprogrammu.

Kvalifikācijas darba mērķis ir izstrādāt sociālā tīkla tīmekļa lietotni "Osmos", kas paredzēta vienas klases vai skolas audzēkņu savstarpējai saziņai. Sistēmā lietotāji var izveidot profilu, kļūt par draugiem, publicēt ierakstus ar attēliem un video, komentēt tos, atzīmēt ar "patīk", sarakstīties privāti, apvienoties grupās un meklēt ierakstus pēc tēmturiem. Sistēmas administrators var pārvaldīt lietotāju kontus un dzēst nepiemērotu saturu.

Mērķa sasniegšanai tika izvirzīti šādi uzdevumi:

- izpētīt līdzīgu sociālo tīklu funkcionalitāti un noteikt sistēmas prasības;
- izvēlēties izstrādes rīkus un tehnoloģijas;
- izstrādāt sistēmas arhitektūru un datubāzes struktūru;
- realizēt sistēmas funkcijas PHP valodā un izveidot lietotāja saskarni;
- pārbaudīt sistēmas darbību, veicot testēšanu;
- sagatavot sistēmas tehnisko dokumentāciju un lietotāja ceļvedi.

Dokumentācija sastāv no septiņām nodaļām. Pirmajā nodaļā ir formulēts uzdevums, bet otrajā aprakstīta sistēmas ieejas un izejas informācija, kā arī funkcionālās un nefunkcionālās prasības. Trešajā nodaļā ir pamatota izstrādes līdzekļu izvēle, ceturtajā aprakstīti sistēmas struktūras un funkcionālie modeļi, bet piektajā datubāzes tabulu struktūra. Sestā nodaļa ir lietotāja ceļvedis, un septītajā nodaļā apkopoti testēšanas rezultāti. Darba noslēgumā ir secinājumi, izmantoto informācijas avotu saraksts un pielikumi ar programmas pirmkodu.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izstrādāt sociālā tīkla tīmekļa lietotni “Osmos”, kas darbojas kā vienkāršota Facebook versija vienas klases, kursa vai skolas vajadzībām. Lietotāji sistēmā pieslēdzas ar lietotājvārdu un paroli, un viss tās saturs ir pieejams tikai pieslēgušamies lietotājiem. Lietotājiem jābūt iespējai izveidot profilu, nosūtīt draudzības pieprasījumus, publicēt ierakstus ziņu lentā, komentēt tos un atzīmēt ar “patīk”. Katrs lietotājs var noteikt, vai viņa profila saturu redz visi tīkla lietotāji vai tikai draugi. Lietotājvārdu pārbaude un tēmturu meklēšana jārealizē ar regulārajām izteiksmēm.

Papildus uzdevumā noteiktajām funkcijām sistēmā tika realizētas grupas, privātās ziņas, paziņojumi par lietotāju darbībām un sistēmas administratora panelis. Grupas ļauj apvienot lietotājus pēc kopīgām interesēm vai mācību priekšmetiem, bet administratora panelis ļauj uzturēt kārtību tīklā bez tiešas iejaukšanās datubāzē.

Sistēmā ir paredzētas trīs lietotāju lomas ar šādām iespējām:

- neautentificētam apmeklētājam būs iespēja:
    - reģistrēties sistēmā, izvēloties sev lietotājvārdu;
    - pieslēgties sistēmai ar lietotājvārdu un paroli;
- reģistrētam lietotājam būs iespēja:
    - rediģēt sava profila informāciju, mainīt profila un vāka attēlu;
    - izvēlēties, vai profila saturu redz visi lietotāji vai tikai draugi;
    - nosūtīt, apstiprināt un noraidīt draudzības pieprasījumus, kā arī dzēst lietotājus no draugu saraksta;
    - publicēt, rediģēt un dzēst ierakstus ar tekstu, attēlu vai video;
    - apskatīt ziņu lentu ar saviem un draugu ierakstiem;
    - komentēt ierakstus, atbildēt uz komentāriem un atzīmēt tos ar “patīk”;
    - atzīmēt citus lietotājus ierakstos ar simbolu @ un pievienot ierakstiem tēmturus ar simbolu #;
    - meklēt lietotājus, grupas un ierakstus pēc tēmtura;
    - sūtīt privātas ziņas citiem lietotājiem;
    - saņemt paziņojumus par darbībām, kas saistītas ar viņa saturu;
    - izveidot grupas, pievienoties tām, uzaicināt draugus un pārvaldīt grupas dalībniekus;
- sistēmas administratoram papildus reģistrēta lietotāja iespējām būs iespēja:
    - apskatīt sistēmas statistiku un lietotāju sarakstu;
    - bloķēt un atbloķēt lietotāju kontus;

- piešķirt un noņemt administratora tiesības citiem lietotājiem;
- atiestatīt lietotāja aizmirstu paroli;
- dzēst jebkuru ierakstu, komentāru vai grupu.

Pirmo sistēmas administratoru norāda datubāzē, bet turpmāk administratora tiesības var piešķirt jau esošs administrators. Sistēmas lietojumgadījumu diagramma, kurā attēlotas lietotāju lomas un to iespējas, ir aplūkojama 4.2. apakšnodaļā.

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Sistēma apstrādā datus, kurus lietotāji ievada tīmekļa formās vai augšupielādē kā failus. Visi ievadītie dati pirms saglabāšanas tiek pārbaudīti servera pusē, bet augšupielādēto failu tips tiek noteikts pēc to satura, nevis pēc faila nosaukuma vai paplašinājuma.

1. Informācija par lietotāju sastāv no šādiem datiem.
    - **Vārds** (First name): lietotāja vārds, teksts līdz 50 rakstzīmēm bez atstarpēm, kas nedrīkst būt skaitlis, piemēram, “Jānis”.
    - **Uzvārds** (Last name): lietotāja uzvārds ar tādiem pašiem ierobežojumiem kā vārdam, piemēram, “Bērziņš”.
    - **Lietotājvārds** (Username): unikāls lietotāja identifikators, ar kuru lietotājs pieslēdzas sistēmai un ar kuru citi lietotāji viņu atzīmē ierakstos. Tas sastāv no 3 līdz 20 rakstzīmēm, drīkst saturēt mazos latīņu alfabēta burtus, ciparus un pasvītrojuma zīmi, un tam jāsākas ar burtu, piemēram, “janis_b”.
    - **Dzimums** (Gender): izvēle no saraksta “Male” vai “Female”. Pēc dzimuma tiek izvēlēts noklusējuma profila attēls.
    - **Parole** (Password): teksts, kas sastāv no vismaz 6 rakstzīmēm. Reģistrējoties un mainot paroli, tā jāievada divas reizes.
    - **Apraksts par sevi** (About me): teksts līdz 1024 rakstzīmēm.
    - **Privātuma iestatījums**: izvēle “Public” (saturu redz visi lietotāji) vai “Friends only” (saturu redz tikai draugi).
    - **Profila attēls** un **vāka attēls**: JPEG formāta attēla fails. Profila attēlam tiek izveidots apgriezts sīktēls 600 × 600 pikseļu izmērā, bet vāka attēlam 1366 × 488 pikseļu izmērā.
2. Informācija par ierakstu un komentāru sastāv no šādiem datiem.
    - **Teksts**: brīvs teksts, kurā var atzīmēt lietotājus formā @lietotājvārds un pievienot tēmturus formā #tēmturis, piemēram, “Gatavojamies eksāmenam #php @janis_b”.
    - **Pievienotais fails**: JPEG attēls vai MP4 video. Attēli, kuru platums vai augstums pārsniedz 1500 pikseļus, tiek proporcionāli samazināti.
    - Ierakstam vai komentāram jāsatur teksts, fails vai abi.

3. Informācija par privāto ziņu sastāv no ziņas teksta un neobligāta attēla JPEG vai PNG formātā. Saņēmēju lietotājs izvēlas, atverot otra lietotāja profilu un nospiežot pogu “Message”.
4. Informācija par grupu sastāv no šādiem datiem.
    - **Nosaukums** (Group name): teksts līdz 50 rakstzīmēm, kas nedrīkst būt skaitlis, piemēram, “Programmēšana”.
    - **Apraksts** (Description): teksts līdz 1024 rakstzīmēm.
    - **Grupas tips**: izvēle “Public” (publiska grupa) vai “Private” (privāta grupa).
    - **Vāka attēls**: JPEG formāta attēla fails.
5. Meklēšanas vaicājums ir teksts, ko lietotājs ievada meklēšanas laukā lapas augšdaļā. Ja vaicājums sākas ar simbolu #, sistēma meklē ierakstus ar šo tēmturi, bet, ja ar simbolu @, meklē lietotājus pēc lietotājvārda.
6. Sistēmas administrators ievada lietotāju meklēšanas tekstu un izvēlas darbību ar lietotāja kontu vai grupu: bloķēt, atbloķēt, piešķirt vai noņemt administratora tiesības, atiestatīt paroli vai dzēst grupu.

2.1.2. Izejas informācijas apraksts

Sistēma lietotājam attēlo šādu izejas informāciju.

1. **Ziņu lenta**: lietotāja un viņa draugu ieraksti apgrieztā hronoloģiskā secībā pa 10 ierakstiem lappusē. Pie katra ieraksta redzams autora vārds un profila attēls, teksts ar saitēm uz atzīmētajiem lietotājiem un tēmturiem, pievienotais attēls vai video, publicēšanas laiks, “patīk” atzīmju un komentāru skaits. Publicēšanas laiks tiek attēlots relatīvi, piemēram, “5 minutes ago”, bet ierakstiem, kas ir vismaz trīs dienas veci, kā datums.
2. **Populārākie tēmturi**: astoņi biežāk lietotie tēmturi no pēdējiem ierakstiem, kurus lietotājs drīkst redzēt, ar lietošanas reižu skaitu.
3. **Profila lapa**: vāka un profila attēls, vārds, uzvārds, lietotājvārds, draugu skaits, lietotāja ieraksti, fotoattēli, draugu saraksts, grupas un apraksts par lietotāju.
4. **Ieraksta lapa**: viens ieraksts ar visiem tā komentāriem un komentāra pievienošanas formu.
5. **Paziņojumu saraksts**: pēdējie 30 paziņojumi ar darbības veicēja vārdu, darbības aprakstu un laiku. Jaunie paziņojumi ir izcelti ar krāsu, bet to skaits redzams lapas galvenē.
6. **Privāto ziņu sarakste**: sarunu saraksts, kurā katrai sarunai redzama pēdējā ziņa, un atvērtās sarunas pēdējās 20 ziņas. Nelasīto ziņu skaits redzams lapas galvenē.

7. **Meklēšanas rezultāti**: līdz 30 lietotājiem un līdz 12 grupām, bet, meklējot tēmturi, līdz 30 ierakstiem ar šo tēmturi.
8. **Grupas lapa**: vāka attēls, nosaukums, tips, dalībnieku skaits, grupas ieraksti, dalībnieku saraksts, fotoattēli un grupas apraksts.
9. **Administratora panelis**: lietotāju, ierakstu, komentāru, grupu un bloķēto kontu skaits, lietotāju saraksts ar to statusu un grupu saraksts.
10. **Paziņojumi par darbību rezultātu**: zaļā krāsā par veiksmīgu darbību, piemēram, “Your settings were saved.”, un sarkanā krāsā par kļūdu, piemēram, “Wrong username or password!”.

2.2. Funkcionālās prasības

Sistēmas funkcionālās prasības ir aprakstītas pa funkcijām. Katrai funkcijai tabulā ir norādīts identifikators, īss apraksts, ievaddati, apstrādes soļi, izvaddati un kļūdu paziņojumi. Lietotāja saskarne ir angļu valodā, tāpēc pogu nosaukumi un paziņojumu teksti ir norādīti tādā formā, kādā tos redz lietotājs.

2.1. tabula

Lietotāja reģistrācijas prasību apraksts

Identifikators	FP-01
Apraksts	Funkcija ļauj jaunam lietotājam izveidot kontu sistēmā.
Ievaddati	Vārds, uzvārds, lietotājvārds, dzimums, parole un paroles atkārtojums.
Apstrāde	1. Sistēma pārbauda, vai ir aizpildīti visi lauki. 2. Vārds un uzvārds nedrīkst būt skaitlis, saturēt atstarpes un būt garāks par 50 rakstzīmēm. 3. Lietotājvārds tiek pārveidots mazajos burtos un pārbaudīts ar regulāro izteiksmi `^[a-z][a-z0-9_]{2,19}$`. Tas nedrīkst sakrist ar cita lietotāja vai grupas lietotājvārdu. 4. Parolei jābūt vismaz 6 rakstzīmes garai un jāsakrīt ar atkārtojumu. 5. Ja kļūdu nav, sistēma izveido kontu ar unikālu skaitlisku identifikatoru, bet paroli saglabā kā jaucējvērtību, kas iegūta ar funkciju `password_hash`.
Izvaddati	Lietotājs tiek novirzīts uz pieslēgšanās lapu ar paziņojumu “Your account was created. You can log in now.”
Kļūdu paziņojumi	“First name is empty!” (atbilstoši katram tukšajam laukam); “First name can't be a number!”, “First name can't have spaces!”; “The username must be 3 to 20 characters long, start with a letter and contain only lowercase letters (a-z), numbers and _”; “That username is already taken!”; “The password must be at least 6 characters long!”, “The passwords do not match!”.

2.2. tabula

Pieslēgšanās un atslēgšanās prasību apraksts

Identifikators	FP-02
Apraksts	Funkcija ļauj reģistrētam lietotājam pieslēgties sistēmai un beigt darbu ar to.
Ievaddati	Lietotājvārds un parole.
Apstrāde	1. Sistēma pārbauda, vai abi lauki ir aizpildīti. 2. Sistēma atrod lietotāja kontu pēc lietotājvārda (grupas nevar pieslēgties) un pārbauda paroli ar funkciju `password_verify`. 3. Ja konts ir bloķēts, pieslēgšanās tiek atteikta. 4. Pēc veiksmīgas pieslēgšanās sistēma izveido jaunu sesijas identifikatoru un saglabā sesijā lietotāja identifikatoru. 5. Atslēdzoties (Logout) lietotāja dati tiek dzēsti no sesijas. 6. Visas pārējās lapas atver tikai pieslēgušies lietotāji, bet bloķēta lietotāja sesija tiek pārtraukta nākamajā lapas ielādē.
Izvaddati	Pēc pieslēgšanās tiek atvērta lietotāja profila lapa, pēc atslēgšanās pieslēgšanās lapa.
Kļūdu paziņojumi	“Please enter your username and password!”; “Wrong username or password!”; “This account has been blocked by an administrator.”.

2.3. tabula

Profila iestatījumu prasību apraksts

Identifikators	FP-03
Apraksts	Funkcija ļauj lietotājam mainīt savu profila informāciju, privātuma iestatījumu un paroli.
Ievaddati	Vārds, uzvārds, lietotājvārds, dzimums, apraksts par sevi, privātuma iestatījums, jaunā parole un tās atkārtojums (paroles lauki nav obligāti).
Apstrāde	1. Iestatījumu lapu var atvērt tikai savā profilā. 2. Vārds, uzvārds un lietotājvārds tiek pārbaudīti tāpat kā reģistrējoties, bet lietotājvārds nedrīkst sakrist ar citu lietotāju lietotājvārdiem. 3. Apraksts par sevi nedrīkst pārsniegt 1024 rakstzīmes. 4. Ja ievadīta jauna parole, tai jābūt vismaz 6 rakstzīmes garai un jāsakrīt ar atkārtojumu. Ja paroles lauki ir tukši, parole netiek mainīta.
Izvaddati	Iestatījumi tiek saglabāti, un tiek parādīts paziņojums “Your settings were saved.”
Kļūdu paziņojumi	“First name can't be empty!”, “First name can't have spaces!”; “That username is already taken!”; “The About me text is too long (max 1024 characters)!”; “The passwords do not match!”, “The new password must be at least 6 characters long!”.

2.4. tabula

Profila un vāka attēla maiņas prasību apraksts

Identifikators	FP-04

Apraksts	Funkcija ļauj lietotājam nomainīt profila attēlu vai vāka attēlu.
Ievaddati	JPEG attēla fails vai jau publicēts paša lietotāja attēls.
Apstrāde	1. Sistēma nosaka faila tipu pēc tā satura un pieņem tikai JPEG attēlus. 2. Attēls tiek saglabāts lietotāja mapē, un tam tiek izveidots samazināts attēls (600 × 600 pikseļi profila attēlam, 1366 × 488 pikseļi vāka attēlam). 3. Jauna attēla augšupielādes gadījumā ziņu lentā tiek publicēts ieraksts “updated his profile image” vai “updated her cover image”. 4. Ja lietotājs izvēlas savu jau publicētu attēlu (Set as Profile Picture), jauns ieraksts netiek veidots un attēls netiek kopēts.
Izvaddati	Profila lapā tiek parādīts jaunais attēls un paziņojums “Your profile picture was changed.”
Kļūdu paziņojumi	“Only JPEG files are allowed!”; “Please select an image file first!”; “That file is too large.”.

2.5. tabula

Ierakstu publicēšanas, rediģēšanas un dzēšanas prasību apraksts

Identifikators	FP-05
Apraksts	Funkcija ļauj lietotājam publicēt ierakstus savā profilā vai grupā, kā arī labot un dzēst tos.
Ievaddati	Ieraksta teksts un neobligāts fails (JPEG attēls vai MP4 video).
Apstrāde	1. Ierakstam jāsatur teksts vai fails. Faila tips tiek noteikts pēc satura. 2. Sistēma tekstā atrod lietotāju atzīmes ar regulāro izteiksmi `MENTION_PATTERN` un nosūta paziņojumu katram atzīmētajam lietotājam. 3. Attēlojot ierakstu, atzīmes un tēmturi tiek pārveidoti par saitēm. 4. Rediģēt ierakstu var tikai tā autors. Ja tiek pievienots jauns fails, iepriekšējais fails tiek dzēsts. 5. Dzēst ierakstu var tā autors, komentētā ieraksta autors, grupas moderators vai administrators un sistēmas administrators. Kopā ar ierakstu tiek dzēsti tā komentāri, “patīk” atzīmes, paziņojumi un faili.
Izvaddati	Ieraksts parādās ziņu lentā, autora profilā vai grupā. Pēc dzēšanas tiek parādīts paziņojums “The post was deleted.”
Kļūdu paziņojumi	“Please type something to post!”; “Only JPEG and MP4 files are allowed!”; “Access denied! You cannot edit this post.”, “Access denied! You cannot delete this post!”.

2.6. tabula

Ziņu lentas attēlošanas prasību apraksts

Identifikators	FP-06
Apraksts	Funkcija attēlo lietotājam viņa paša un viņa draugu ierakstus.
Ievaddati	Lappuses numurs (no lietotāja puses netiek prasīti citi dati).
Apstrāde	1. Sistēma atrod lietotāja draugu identifikatorus. 2. No datubāzes tiek atlasīti lietotāja un viņa draugu ieraksti, kas nav

	komentāri un nav publicēti grupās, sakārtoti no jaunākā uz vecāko. 3. Lappusē tiek attēloti 10 ieraksti. Pogas “Next page” un “Previous page” ir redzamas tikai tad, ja attiecīgā lappuse eksistē.
Izvaddati	Ierakstu saraksts un populārāko tēmturu bloks.
Kļūdu paziņojumi	Ja ierakstu nav, tiek parādīts teksts “No posts found! Write your first post or add some friends.”

2.7. tabula

Komentēšanas prasību apraksts

Identifikators	FP-07
Apraksts	Funkcija ļauj lietotājam komentēt ierakstus un atbildēt uz komentāriem.
Ievaddati	Komentāra teksts, neobligāts fails un ieraksts, kuram tiek pievienots komentārs.
Apstrāde	1. Komentārs tiek saglabāts kā ieraksts, kuram norādīts vecākieraksts, tāpēc uz komentāru var atbildēt tāpat kā uz ierakstu. 2. Vecākieraksta komentāru skaits tiek palielināts par vienu, un tā autoram tiek nosūtīts paziņojums. 3. Lietotājs sāk sekot ierakstam, ko komentējis, un turpmāk saņem paziņojumus par jauniem komentāriem. 4. Grupas ierakstus var komentēt tikai grupas dalībnieki.
Izvaddati	Komentāri tiek attēloti zem ieraksta hronoloģiskā secībā.
Kļūdu paziņojumi	“You can't comment on this post.”; “The post you are replying to no longer exists!”.

2.8. tabula

“Patīk” atzīmju prasību apraksts

Identifikators	FP-08
Apraksts	Funkcija ļauj lietotājam atzīmēt ierakstu vai komentāru ar “patīk”.
Ievaddati	Ieraksts, uz kura lietotājs nospiež saiti “Like”.
Apstrāde	1. Pieprasījums tiek nosūtīts serverim ar JavaScript (AJAX), lapa netiek pārlādēta. 2. Ja lietotājs ierakstu vēl nav atzīmējis, tiek pievienota atzīme, bet atkārtots klikšķis atzīmi noņem. 3. Ieraksta autors saņem paziņojumu par atzīmi tikai vienu reizi. 4. Atzīmēt var tikai tos ierakstus, kurus lietotājs drīkst redzēt.
Izvaddati	Atzīmju skaits, saites teksts “Like” vai “Unlike” un teksts, piemēram, “You and 1 other liked this post”. Atverot šo tekstu, redzams atzīmējušo lietotāju saraksts.
Kļūdu paziņojumi	“You can't like this post.”

2.9. tabula

Draudzības pieprasījumu prasību apraksts

Identifikators	FP-09
Apraksts	Funkcija ļauj lietotājiem kļūt par draugiem un beigt draudzību.

Ievaddati	Otrs lietotājs un darbība: “Add friend”, “Accept friend request”, “Decline”, “Cancel request” vai “Unfriend”.
Apstrāde	1. Pieprasījumu var nosūtīt tikai citam aktīvam lietotājam, bet ne sev, ne grupai. 2. Starp diviem lietotājiem glabājas viens ieraksts ar statusu “pending” (gaida atbildi) vai “accepted” (draugi). 3. Ja otrs lietotājs jau ir nosūtījis pieprasījumu, pieprasījuma sūtīšana to apstiprina. 4. Saņēmējs pieprasījumu var apstiprināt vai noraidīt, bet sūtītājs to var atsaukt. Noraidot, atsaucot vai beidzot draudzību, ieraksts tiek dzēsts. 5. Par jaunu pieprasījumu un apstiprinātu pieprasījumu tiek nosūtīts paziņojums.
Izvaddati	Profila lapā redzams draudzības statuss un atbilstošās pogas, bet cilnē “Friend requests” saņemtie un nosūtītie pieprasījumi. Paziņojumi: “Friend request sent.”, “You are now friends!”, “Friend request declined.”
Kļūdu paziņojumi	Ja darbība nav iespējama (piemēram, pieprasījums jau nosūtīts), sistēma to neizpilda un atgriež lietotāju profila lapā.

2.10. tabula

Privātuma iestatījumu prasību apraksts

Identifikators	FP-10
Apraksts	Funkcija ļauj lietotājam noteikt, kas redz viņa profila saturu.
Ievaddati	Iestatījums “Public” vai “Friends only” (iestatījumu lapā).
Apstrāde	1. Ja izvēlēts “Friends only”, lietotāja ierakstus, fotoattēlus, draugu un grupu sarakstu redz tikai pats lietotājs, viņa draugi un sistēmas administratori. 2. Vārds, profila attēls un cilne “About” ir redzami visiem, lai lietotāju varētu atrast un nosūtīt viņam draudzības pieprasījumu. 3. Ierobežojums tiek pārbaudīts servera pusē ieraksta lapā, attēla skatā, “patīk” atzīmju sarakstā, tēmturu meklēšanā un populārāko tēmturu blokā.
Izvaddati	Lietotājs, kuram nav piekļuves, redz tekstu “This profile is visible only to friends” un pogu “Add friend”.
Kļūdu paziņojumi	“Please choose who can see your posts!” (ja nosūtīta nederīga vērtība).

2.11. tabula

Meklēšanas un tēmturu prasību apraksts

Identifikators	FP-11
Apraksts	Funkcija ļauj atrast lietotājus, grupas un ierakstus pēc tēmtura.
Ievaddati	Meklēšanas teksts vai tēmtura saite ierakstā.
Apstrāde	1. Ja teksts atbilst regulārajai izteiksmei `^#([a-zA-Z_][a-zA-Z0-9_]*)$`, tiek atvērta tēmtura lapa. 2. Ja teksts sākas ar @, simbols tiek noņemts un meklēts lietotājvārds. 3. Citādi lietotāji tiek meklēti pēc vārda, uzvārda, pilnā vārda un lietotājvārda (bloķētie lietotāji netiek rādīti), bet grupas pēc nosaukuma. 4. Tēmtura lapā ieraksti tiek atlasīti ar MariaDB operatoru REGEXP [6], kas atrod tēmturi kā atsevišķu vārdu, un tiek parādīti tikai tie ieraksti, kurus

	lietotājs drīkst redzēt. 5. Tēmturi ierakstos tiek atpazīti ar regulāro izteiksmi `HASHTAG_PATTERN`, bet populārākie tēmturi tiek saskaitīti pēdējos 300 ierakstos.
Izvaddati	Lietotāju un grupu saraksts vai ierakstu saraksts ar virsrakstu “Posts tagged with: #php”.
Kļūdu paziņojumi	“No people were found”, “No posts found for this tag.”

2.12. tabula

Privāto ziņu prasību apraksts

Identifikators	FP-12
Apraksts	Funkcija ļauj diviem lietotājiem sarakstīties privāti.
Ievaddati	Saņēmējs, ziņas teksts un neobligāts attēls (JPEG vai PNG).
Apstrāde	1. Ziņu nevar nosūtīt sev vai grupai. Ziņai jāsatur teksts vai attēls. 2. Visas divu lietotāju ziņas tiek apvienotas vienā sarunā. 3. Atverot sarunu, saņemtās ziņas tiek atzīmētas kā izlasītas, un nelasīto ziņu skaits galvenē samazinās. 4. Lietotājs var dzēst atsevišķu ziņu vai visu sarunu. Dzēšana attiecas tikai uz viņa paša skatu, otrs lietotājs ziņas joprojām redz.
Izvaddati	Sarunu saraksts un saruna, kurā lietotāja ziņas ir labajā pusē, bet otra lietotāja ziņas kreisajā pusē.
Kļūdu paziņojumi	“Please type something to send.”; “Only JPEG and PNG files are allowed!”; “You can't send a message to this user.”.

2.13. tabula

Paziņojumu prasību apraksts

Identifikators	FP-13
Apraksts	Funkcija informē lietotāju par citu lietotāju darbībām, kas saistītas ar viņu.
Ievaddati	Citu lietotāju darbības sistēmā.
Apstrāde	1. Paziņojums tiek izveidots, ja kāds atzīmē lietotāja ierakstu ar “patīk”, komentē to, atzīmē lietotāju ierakstā, nosūta vai apstiprina draudzības pieprasījumu, uzaicina lietotāju grupā vai maina viņa lomu grupā. 2. Lietotājs saņem paziņojumus arī par jauniem komentāriem ierakstos, kurus viņš komentējis. 3. Paziņojums par “patīk” atzīmi, atzīmi ierakstā, draudzības pieprasījumu un uzaicinājumu tiek izveidots tikai vienu reizi. 4. Atverot paziņojumu lapu, visi parādītie paziņojumi tiek atzīmēti kā skatīti.
Izvaddati	Paziņojumu skaits galvenē un paziņojumu saraksts, piemēram, “Anna Bērziņa sent you a friend request.” ar saiti uz attiecīgo saturu.
Kļūdu paziņojumi	“No notifications to display.” (ja paziņojumu nav).

2.14. tabula

Grupu prasību apraksts

Identifikators	FP-14
Apraksts	Funkcija ļauj lietotājiem veidot grupas un apvienoties tajās.
Ievaddati	Grupas nosaukums, apraksts, tips, vāka attēls un dalībnieku pārvaldības darbības.
Apstrāde	1. Grupas veidotājs kļūst par tās īpašnieku. Īpašnieku nevar noņemt no grupas un nevar mainīt viņa lomu. 2. Publiskai grupai var pievienoties uzreiz, un tās saturu redz visi lietotāji. Privātai grupai tiek nosūtīts pievienošanās pieprasījums, bet tās ierakstus, dalībniekus un fotoattēlus redz tikai dalībnieki. 3. Grupas dalībnieki var publicēt ierakstus, komentēt un uzaicināt grupā savus draugus. Uzaicināts lietotājs var uzaicinājumu pieņemt vai noraidīt. 4. Moderators apstiprina vai noraida pievienošanās pieprasījumus un var dzēst grupas ierakstus. 5. Grupas administrators papildus maina dalībnieku lomas, noņem dalībniekus, maina grupas iestatījumus un vāka attēlu. 6. Dalībnieks, kas nav īpašnieks, var pamest grupu (Leave Group).
Izvaddati	Grupas lapa ar cilnēm “Discussion”, “About”, “Members”, “Photos”, “Requests” un “Settings” (pēdējās divas tikai moderatoriem un administratoriem).
Kļūdu paziņojumi	“Group name is empty!”, “Group name cannot be a number.”; “Only members can post in this group.”; “The group owner can't be changed or removed.”.

2.15. tabula

Sistēmas administrēšanas prasību apraksts

Identifikators	FP-15
Apraksts	Funkcija ļauj sistēmas administratoram pārvaldīt lietotāju kontus un grupas.
Ievaddati	Lietotāju meklēšanas teksts un izvēlētā darbība.
Apstrāde	1. Administratora paneli var atvērt tikai lietotāji, kuriem datubāzē ir administratora tiesības. 2. Panelī tiek attēlota statistika, lietotāju saraksts ar meklēšanu un grupu saraksts. 3. Bloķētais lietotājs nevar pieslēgties sistēmai, un viņa sesija tiek pārtraukta. Administratoru nevar bloķēt, pirms viņam nav noņemtas administratora tiesības, un savu kontu panelī mainīt nevar. 4. Atiestatot paroli, sistēma izveido nejaušu 10 rakstzīmju pagaidu paroli un parāda to administratoram vienu reizi. 5. Dzēšot grupu, tiek dzēsti arī visi tās ieraksti, dalībnieki, pieprasījumi un uzaicinājumi.
Izvaddati	Paziņojumi par darbības rezultātu, piemēram, “Anna Bērziņa was blocked and can no longer log in.”
Kļūdu paziņojumi	“You don't have access to that page.”; “Remove the admin rights first, then block the user.”; “You can't change your own account here.”.

2.3. Nefunkcionālās prasības

2.3.1. Izstrādes prasības

- Sistēma jāizstrādā PHP 8 valodā, izmantojot atsevišķas klases datubāzes savienojumam, lietotājiem, ierakstiem, grupām, privātajām ziņām un attēlu apstrādei.
- Visi pieprasījumi jāapstrādā vienā ieejas punktā `index.php`, kas pēc adreses nosaka atveramo lapu un atver tikai atļautās lapas.
- Regulārās izteiksmes lietotājvārdu pārbaudei un atzīmju un tēmturu atpazīšanai jādefinē vienā vietā (`classes/functions.php`).
- Programmas kodā jābūt komentāriem, kas paskaidro funkciju nozīmi.

2.3.2. Saskarnes prasības

- Visām lapām jābūt vienotam noformējumam ar dzeltenu galveni un baltiem satura blokiem.
- Lietotāja saskarne ir angļu valodā.
- Saskarne paredzēta datora ekrānam, kura platums ir vismaz 1024 pikseļi.
- Par katras darbības rezultātu lietotājam jāparāda paziņojums.

2.3.3. Programmatūras prasības

- Serverim: Apache 2.4 tīmekļa serveris ar moduli mod_rewrite [4], PHP 8.0 vai jaunāka versija ar paplašinājumiem mysqli, GD, fileinfo un mbstring, kā arī MariaDB 10.4 vai jaunāka versija vai MySQL 8. Visas šīs programmas ietver XAMPP 8.2 [3].
- Klientam: jaunākā Google Chrome, Mozilla Firefox, Microsoft Edge vai Safari versija ar ieslēgtu JavaScript.
- Operētājsistēma: jebkura, kurā darbojas XAMPP (Windows, macOS vai Linux).

2.3.4. Aparatūras prasības

- Serverim: dators ar vismaz 2 GB operatīvās atmiņas un vismaz 1 GB brīvas vietas diskā sistēmai un augšupielādētajiem failiem.
- Klientam: jebkurš dators ar pārlūkprogrammu un ekrānu, kura izšķirtspēja ir vismaz 1024 × 768 pikseļi.
- Ziņu lentai jāielādējas ne ilgāk kā 2 sekundēs, ja datubāzē ir līdz 1000 ierakstiem.

2.3.5. Drošības prasības

- Paroles jāglabā tikai kā jaucējvērtības, kas iegūtas ar PHP funkciju `password_hash` [13] [10].
- Visām vērtībām, kas tiek iekļautas SQL vaicājumos, jābūt apstrādātām ar funkciju `mysqli_real_escape_string`, bet identifikatoriem jābūt pārbaudītiem kā skaitļiem, lai novērstu SQL injekcijas.
- Lietotāju ievadītais teksts pirms attēlošanas jāapstrādā ar funkciju `htmlspecialchars`, lai novērstu starpvietņu skriptošanu.
- Katrai formai un katrai saitei, kas maina datus, jāsatur sesijai piesaistīts nejaušs žetons, kas aizsargā pret starpvietņu pieprasījumu viltošanu [8].
- Sesijas sīkdatnei jābūt pieejamai tikai serverim (HttpOnly) un nesūtāmai no citām vietnēm (SameSite=Lax). Pēc pieslēgšanās jāizveido jauns sesijas identifikators.
- Augšupielādēto failu tips jānosaka pēc to satura, un mapē `uploads` jābūt aizliegtai PHP skriptu izpildei.
- Piekļuves tiesības (privātums, grupu lomas, administratora tiesības) jāpārbauda servera pusē katrā pieprasījumā.
- Sistēmā netiek glabāti personas dati, kas tās darbībai nav nepieciešami, piemēram, e-pasta adrese. Tas atbilst Vispārīgās datu aizsardzības regulas datu minimizēšanas principam [1].

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Izstrādes līdzekļi tika izvēlēti, ņemot vērā sistēmas uzdevumu un darba apstākļus. Sociālā tīkla dati pēc būtības ir relāciju dati: lietotāji publicē ierakstus, ieraksti saņem komentārus un "patīk" atzīmes, lietotāji savā starpā veido draudzības saites un apvienojas grupās. Tāpēc datu glabāšanai tika izvēlēta relāciju datubāze. Sistēmai jādarbojas lokālā datorā, neiegādājoties programmatūras licences, un tai jāpalaižas gan Windows, gan macOS vidē, tāpēc visas izmantotās programmas ir bezmaksas un pieejamas abām operētājsistēmām. Svarīgs kritērijs bija arī plaša un brīvi pieejama dokumentācija.

Servera daļas izstrādē tika izmantotas šādas tehnoloģijas.

- **XAMPP** (versija 8.2): bezmaksas izstrādes vides komplekts, kas vienā instalācijā apvieno Apache tīmekļa serveri, MariaDB datubāzu pārvaldības sistēmu, PHP interpretatoru un phpMyAdmin [3]. Izvēlēts, jo tas tiek uzstādīts ar vienu instalācijas programmu un darbojas Windows, macOS un Linux operētājsistēmās.
- **Apache HTTP Server** (versija 2.4) ar moduli **mod_rewrite**: tīmekļa serveris, kas apstrādā pārlūkprogrammas pieprasījumus. Modulis mod_rewrite pēc faila `.htaccess` noteikumiem pārveido saprotamas adreses, piemēram, `profile/1234/friends`, par pieprasījumu vienotajam ieejas punktam `index.php` [4]. Tādējādi adresēs nav redzami PHP failu nosaukumi, un veidņu failus nevar atvērt tieši.
- **PHP** (versija 8): servera puses programmēšanas valoda, kas paredzēta dinamisku tīmekļa lapu veidošanai [11]. Izvēlēta, jo to atbalsta praktiski jebkurš tīmekļa vietņu mitināšanas pakalpojums un tajā ir iebūvēti visi sistēmai nepieciešamie līdzekļi: paroļu jaukšana ar funkcijām `password_hash` un `password_verify` [13], regulārās izteiksmes ar funkcijām `preg_match` un `preg_replace_callback` [14], sesiju pārvaldība un failu augšupielāde. Sistēmā izmantoti šādi PHP paplašinājumi:
    - mysqli: savienojums ar MariaDB datubāzi un SQL vaicājumu izpilde;
    - GD: attēlu apgriešana, samazināšana un sīktēlu izveide [12];
    - fileinfo: augšupielādētā faila tipa noteikšana pēc tā satura;
    - mbstring un iconv: darbs ar UTF-8 tekstu, piemēram, teksta garuma pārbaude rakstzīmēs un grupas lietotājvārda izveide no nosaukuma bez garumzīmēm (no "Skolas sports" iegūst, piemēram, "skolassports482").

- **MariaDB** (versija 10.4 vai jaunāka): relāciju datubāzu pārvaldības sistēma, kas ir saderīga ar MySQL [5]. Izvēlēta, jo tā ir iekļauta XAMPP, atbalsta pilnu Unicode kodējumu utf8mb4 (latviešu burti un emocijzīmes) un operatoru REGEXP, ar kuru tiek meklēti ieraksti pēc tēmtura [6].
- **phpMyAdmin**: tīmekļa saskarne datubāzes pārvaldībai. Ar to tika izveidotas un mainītas tabulas, pārbaudīti SQL vaicājumi un veikts datu imports un eksports.

Klienta daļas izstrādē tika izmantotas šādas tehnoloģijas.

- **HTML5 un CSS3**: lapu struktūra un noformējums. Kopīgie stili ir apvienoti lapas galvenes failā `header.php`, tāpēc visām lapām ir vienots izskats. CSS ietvari, piemēram, Bootstrap, netika izmantoti, lai lapas būtu vieglas un noformējums pilnībā kontrolējams.
- **JavaScript** ar objektu **XMLHttpRequest** [7]: datu apmaiņa ar serveri bez lapas pārlādes (AJAX). Šādi darbojas "patīk" atzīmju pievienošana un lietotājvārdu ieteikumi, kas parādās, kad lietotājs ieraksta tekstā simbolu @. Ārējas bibliotēkas, piemēram, jQuery, netika izmantotas, jo šīm divām funkcijām pietiek ar pārlūkprogrammas iebūvētajiem līdzekļiem.

Sistēmas izstrādes procesā tika izmantoti šādi rīki.

- **Visual Studio Code**: koda redaktors ar sintakses izcelšanu un paplašinājumiem PHP, HTML, CSS un JavaScript valodai.
- **Pārlūkprogrammas izstrādātāja rīki**: HTML struktūras un CSS stilu pārbaude, kā arī AJAX pieprasījumu un JavaScript kļūdu analīze.
- **LibreOffice Writer**: tehniskās dokumentācijas noformēšana un saglabāšana PDF formātā.

Sistēma tika izstrādāta bez PHP ietvariem, piemēram, Laravel vai Symfony. Ietvars būtu paātrinājis atsevišķu funkciju izveidi, taču tas paslēptu tieši tos mehānismus, kurus šajā darbā bija svarīgi izprast un realizēt pašam: pieprasījumu maršrutēšanu, sesiju un piekļuves tiesību pārbaudi, SQL vaicājumu veidošanu un aizsardzību pret biežākajiem uzbrukumiem. Turklāt sistēma bez ietvara un tā atkarībām darbojas jebkurā standarta XAMPP instalācijā bez papildu programmām.

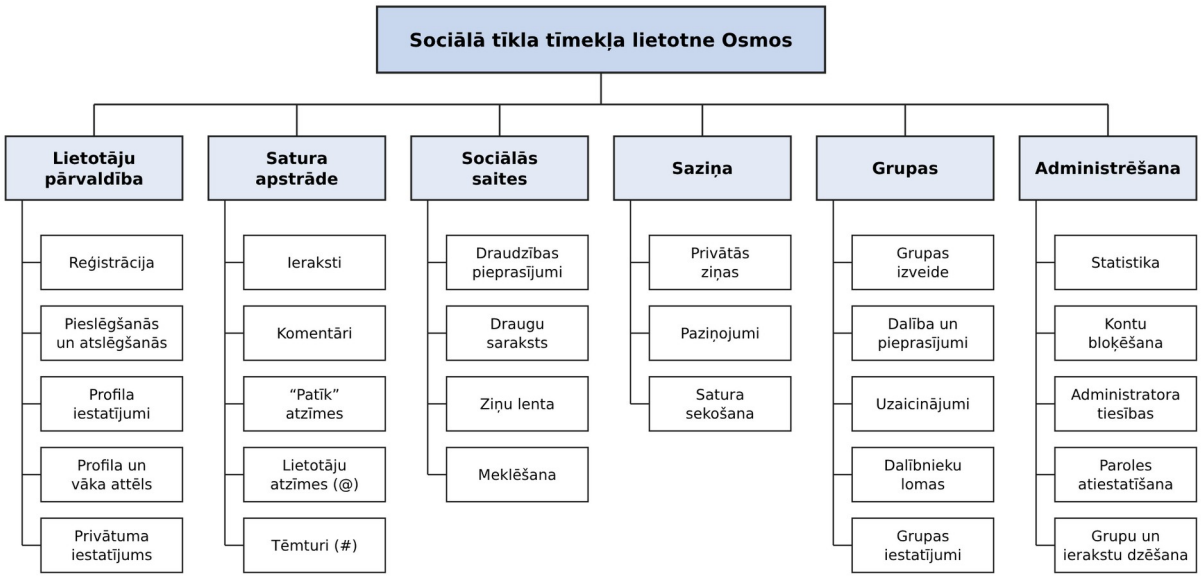
Kā alternatīva tika apsvērta dokumentu datubāze MongoDB. Tā netika izvēlēta, jo sistēmas galvenās darbības balstās uz saitēm starp datiem: ziņu lentā jāatlasa draugu ieraksti, privātuma pārbaudē jānosaka draudzības statuss, bet grupās dalībnieka loma. Relāciju datubāzē šādi vaicājumi ir vienkārši un efektīvi.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēmas funkcijas ir sadalītas sešās apakšsistēmās (skat. 4.1. att.): lietotāju pārvaldības, satura apstrādes, sociālo saišu, saziņas, grupu un administrēšanas apakšsistēmā. Katra apakšsistēma sastāv no moduļiem, kas atbilst otrajā nodaļā aprakstītajām funkcionālajām prasībām.



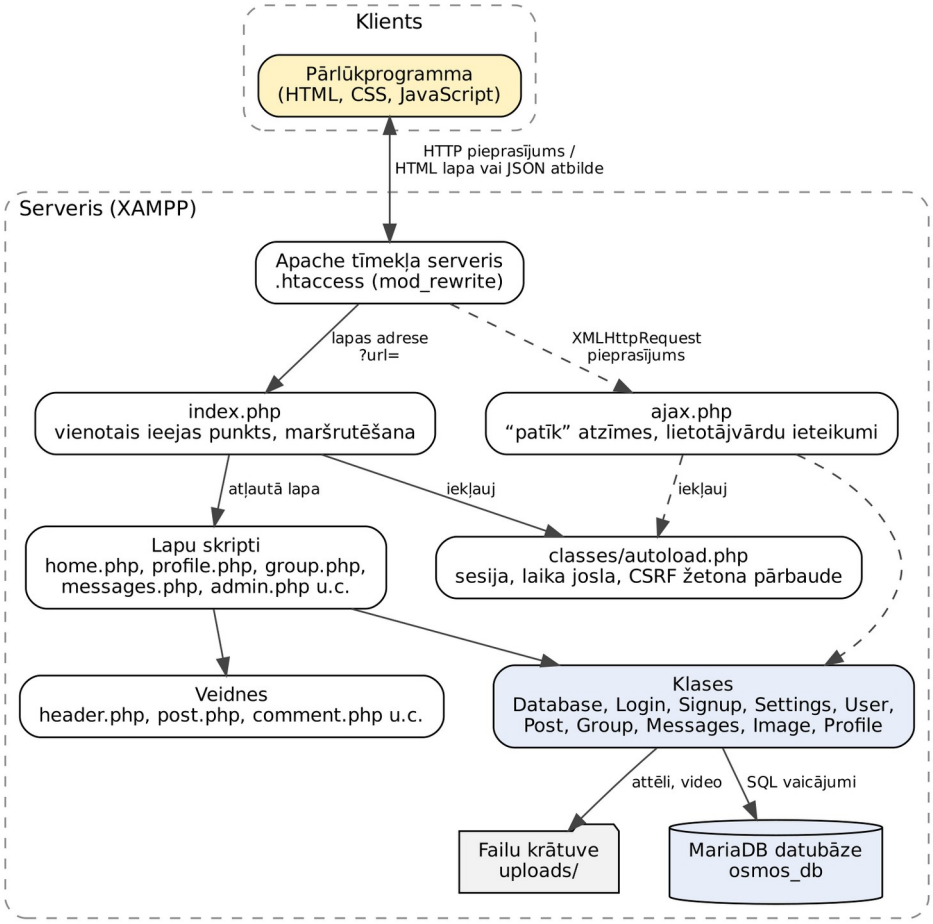
4.1. att. Funkcionālās dekompozīcijas hierarhiskā diagramma

- Lietotāju pārvaldības apakšsistēma nodrošina darbu ar lietotāja kontu.
    - Reģistrācija: jauna konta izveide un lietotājvārda pārbaude ar regulāro izteiksmi.
    - Pieslēgšanās un atslēgšanās: lietotāja identitātes pārbaude un sesijas pārvaldība.
    - Profila iestatījumi: vārda, lietotājvārda, apraksta un paroles maiņa.
    - Profila un vāka attēls: attēla augšupielāde, apgriešana un sīktēla izveide.
    - Privātuma iestatījums: izvēle, vai profila saturu redz visi lietotāji vai tikai draugi.
- Satura apstrādes apakšsistēma pārvalda lietotāju publicēto saturu.

    - Ieraksti: ierakstu ar tekstu, attēlu vai video publicēšana, rediģēšana un dzēšana.
    - Komentāri: komentāru un atbilžu uz komentāriem pievienošana.
    - “Patīk” atzīmes: atzīmes pievienošana un noņemšana bez lapas pārlādes.
    - Lietotāju atzīmes: lietotājvārdu atpazīšana tekstā un paziņojumu nosūtīšana atzīmētajiem lietotājiem.
    - Tēmturi: tēmturu atpazīšana tekstā, saišu izveide un populārāko tēmturu saskaitīšana.
- Sociālo saišu apakšsistēma veido saites starp lietotājiem.
    - Draudzības pieprasījumi: pieprasījumu nosūtīšana, apstiprināšana, noraidīšana un atsaukšana.
    - Draugu saraksts: draugu attēlošana profilā un draudzības pārtraukšana.
    - Ziņu lenta: lietotāja un viņa draugu ierakstu atlase un sadalīšana lappusēs.
    - Meklēšana: lietotāju, grupu un ierakstu meklēšana pēc vārda, lietotājvārda vai tēmtura.
- Saziņas apakšsistēma informē lietotājus un ļauj tiem sazināties.
    - Privātās ziņas: ziņu nosūtīšana, sarunu attēlošana un dzēšana.
    - Paziņojumi: paziņojumu izveide par citu lietotāju darbībām un neskatīto paziņojumu skaitīšana.
    - Satura sekošana: lietotājs, kas komentējis ierakstu, saņem paziņojumus par jauniem komentāriem.
- Grupu apakšsistēma ļauj lietotājiem apvienoties pēc interesēm.
    - Grupas izveide: nosaukuma, apraksta un grupas tipa ievade.
    - Dalība un pieprasījumi: pievienošanās publiskai grupai un pieprasījumi privātai grupai.
    - Uzaicinājumi: draugu uzaicināšana grupā.
    - Dalībnieku lomas: lomu “Member”, “Moderator” un “Admin” piešķiršana un dalībnieku noņemšana.
    - Grupas iestatījumi: nosaukuma, apraksta, tipa un vāka attēla maiņa.
- Administrēšanas apakšsistēma ir pieejama tikai sistēmas administratoriem.
    - Statistika: lietotāju, ierakstu, komentāru, grupu un bloķēto kontu skaits.
    - Kontu bloķēšana: lietotāja piekļuves liegšana un atjaunošana.
    - Administratora tiesības: tiesību piešķiršana un noņemšana.
    - Paroles atiestatīšana: pagaidu paroles izveide lietotājam, kurš aizmirsis paroli.

- Grupu un ierakstu dzēšana: nepiemērota satura dzēšana.

Sistēmai ir klienta un servera arhitektūra (skat. 4.2. att.). Klients ir lietotāja pārlūkprogramma, kas attēlo HTML lapas un izpilda JavaScript kodu. Serveris ir Apache tīmekļa serveris ar PHP interpretatoru, kas apstrādā pieprasījumus, un MariaDB datubāze, kurā glabājas visi sistēmas dati. Lietotāju augšupielādētie attēli un video glabājas failu sistēmā mapē `uploads`, katram lietotājam vai grupai atsevišķā apakšmapē, bet datubāzē tiek saglabāts tikai ceļš uz failu.



4.2. att. Sistēmas arhitektūra

Pieprasījuma apstrāde serverī notiek šādos soļos.

1. Pārlūkprogramma nosūta pieprasījumu, piemēram, uz adresi `profile/1234/friends`.
2. Apache pārbauda, vai šāds fails vai mape eksistē. Ja neeksistē, modulis mod_rewrite pēc faila `.htaccess` noteikuma nodod pieprasījumu failam `index.php`, adreses daļu ievietojot parametrā `url`. Attēli, video un fails `ajax.php` tiek atvērti tieši.
3. Fails `index.php` iekļauj failu `classes/autoload.php`, kas iestata laika joslu Europe/Riga, sāk sesiju, iekļauj visas klases, sadala adresi masīvā `$URL` un katram POST pieprasījumam pārbauda CSRF žetonu.

4. Fails `index.php` pārbauda, vai adreses pirmā daļa ir atļauto lapu sarakstā. Ja ir, tiek izpildīts attiecīgais lapas skripts, piemēram, `profile.php`, bet citādi tiek parādīta lapa “Page not found” ar HTTP statusu 404.
5. Lapas skripts pārbauda, vai lietotājs ir pieslēdzies, apstrādā iesniegto formu un izsauc klašu metodes. Klases izpilda SQL vaicājumus caur klasi `Database`, kas visam pieprasījumam izmanto vienu kopīgu datubāzes savienojumu.
6. Lapas skripts ģenerē HTML lapu, izmantojot veidnes, piemēram, `header.php` (lapas galvene ar meklēšanas lauku un paziņojumu skaitu) un `post.php` (viena ieraksta attēlojums), un nosūta to pārlūkprogrammai.

Pēc veiksmīgas formas apstrādes lapas skripts pārvirza pārlūkprogrammu uz citu adresi (HTTP statuss 302), bet darbības rezultāta paziņojumu saglabā sesijā un parāda nākamajā lapā. Tādējādi, atjaunojot lapu, forma netiek iesniegta atkārtoti. Sistēmas galvenās klases un to uzdevumi ir apkopoti 4.1. tab.

4.1. tabula

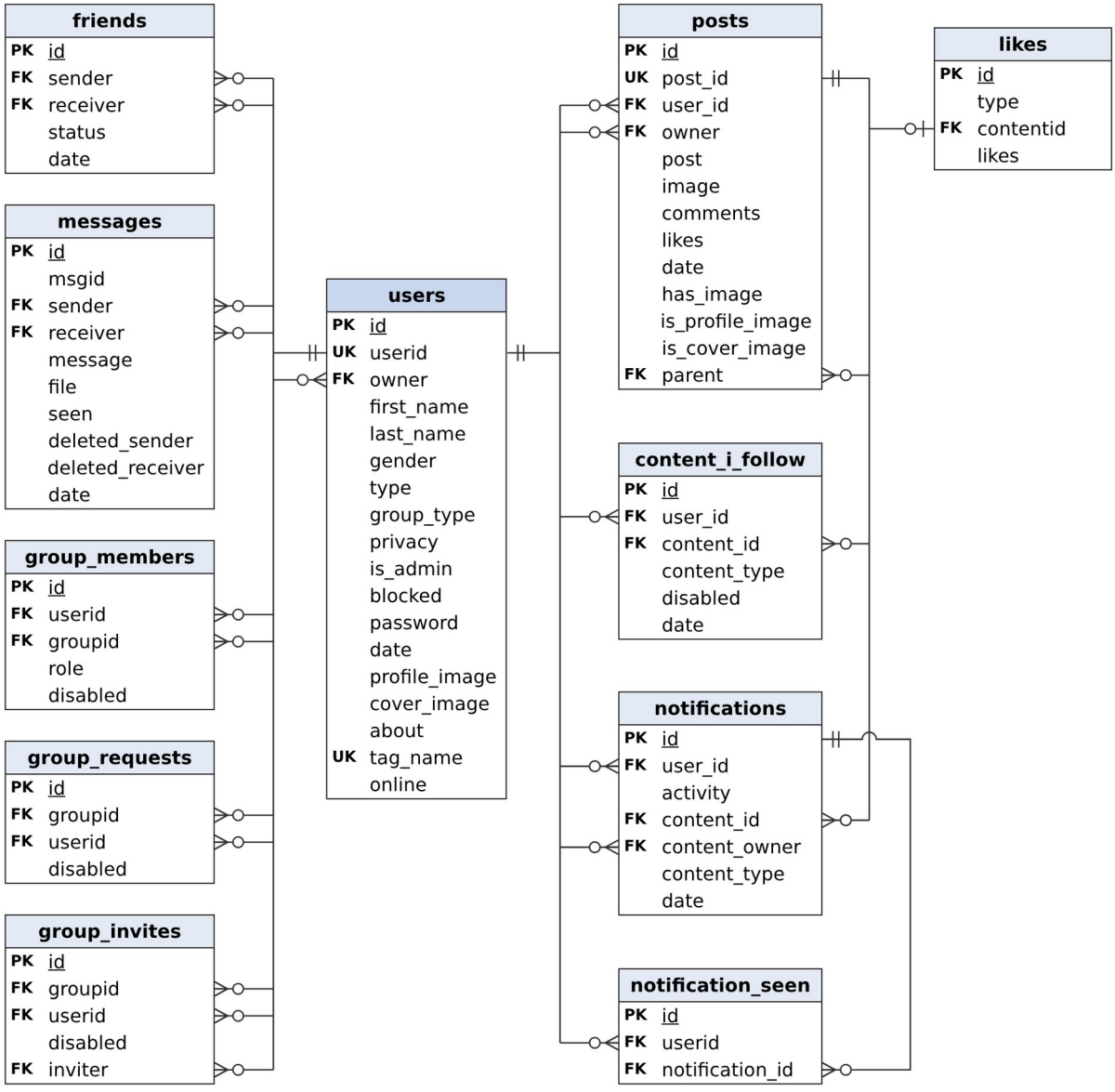
Sistēmas klases un to uzdevumi

Klase	Fails	Uzdevums
Database	classes/connect.php	Savienojums ar datubāzi, vaicājumu izpilde (metodes `read` un `save`) un vērtību aizsargāšana
Login	classes/login.php	Lietotājvārda un paroles pārbaude, sesijas izveide, pieslēgšanās pārbaude katrā lapā
Signup	classes/signup.php	Reģistrācijas datu pārbaude un jauna lietotāja izveide
Settings	classes/settings.php	Profila iestatījumu un privātuma iestatījuma saglabāšana
User	classes/user.php	Lietotāja dati, draudzības statuss un pieprasījumi, profila redzamības pārbaude
Post	classes/post.php	Ierakstu un komentāru izveide, labošana un dzēšana, “patīk” atzīmes, ieraksta redzamības pārbaude
Group	classes/group.php	Grupu izveide, dalībnieki, pieprasījumi, uzaicinājumi, lomas un grupas dzēšana
Messages	classes/messages.php	Privāto ziņu nosūtīšana, sarunu nolasīšana un dzēšana
Image	classes/image.php	Attēlu apgriešana, samazināšana un sīktēlu izveide ar GD bibliotēku
Time	classes/time.php	Laika attēlošana relatīvā formā, piemēram, “5 minutes ago”

Bez klasēm failā `classes/functions.php` ir apkopotas palīgfunkcijas, kuras izmanto vairākas lapas: CSRF žetonu izveide un pārbaude, paziņojumu izveide, failu augšupielādes pārbaude, administratora tiesību un grupas lomu pārbaude, kā arī regulārās izteiksmes lietotājvārdiem, lietotāju atzīmēm un tēmturiem.

4.1.2. Sistēmas ER modelis

Sistēmas datubāze sastāv no 11 tabulām, kuru savstarpējās saites ir attēlotas ER diagrammā (skat. 4.3. att.). Diagrammā izmantots “vārnas pēdas” apzīmējums: divas svītras apzīmē tieši vienu ierakstu, aplis ar svītru apzīmē nulli vai vienu, bet aplis ar trim līnijām apzīmē nulli vai vairākus ierakstus. Saites starp tabulām ir loģiskas, jo datubāzē tās nav definētas kā ārējo atslēgu ierobežojumi. Datu integritāti nodrošina programmas kods, piemēram, dzēšot ierakstu, tiek dzēsti arī visi tā komentāri, “patīk” atzīmes, paziņojumi un faili.



4.3. att. Sistēmas ER diagramma

Lietotāji un grupas glabājas vienā tabulā `users`, un tos atšķir lauks `type` (vērtība "profile" vai "group"). Šāds risinājums ļauj grupām izmantot tās pašas funkcijas kā lietotāju profiliem: vāka attēlu, unikālu identifikatoru saitēs un ierakstu attēlošanu. Tāpēc tabulu `group_members`, `group_requests` un `group_invites` lauki `userid` un `groupid` abi norāda uz tabulas `users` lauku `userid`. Tabulas savstarpēji saista šādas attiecības:

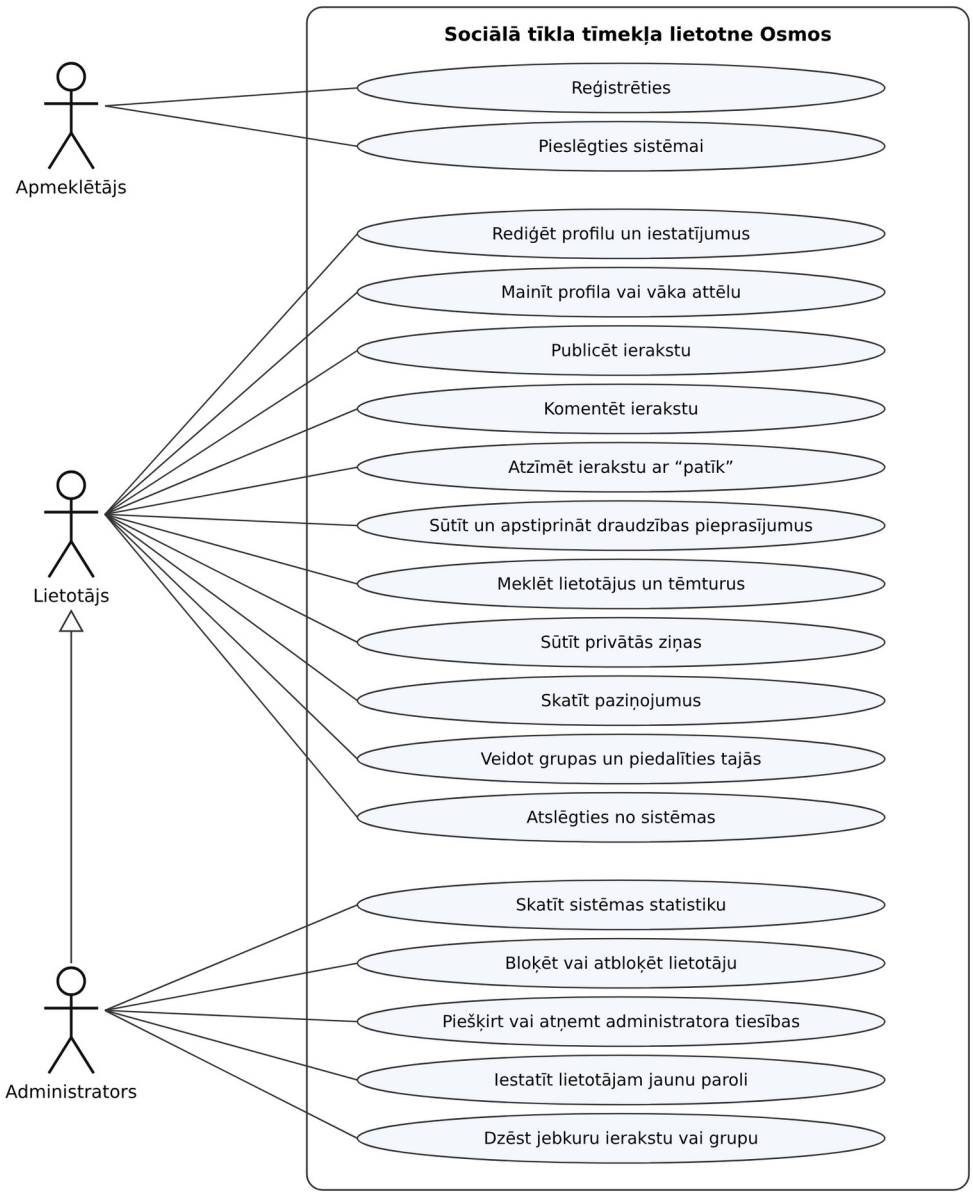
- starp tabulām "users" un "posts" ir attiecība viens pret daudziem, jo viens lietotājs var publicēt daudzus ierakstus, bet katram ierakstam ir tieši viens autors (lauks `user_id`);
- starp grupām, kas glabājas tabulā "users", un tabulu "posts" ir attiecība viens pret daudziem, jo grupā var būt daudzi ieraksti, bet ieraksts pieder ne vairāk kā vienai grupai (lauks `owner`, kura vērtība 0 nozīmē, ka ieraksts nav publicēts grupā);
- tabulai "posts" ir attiecība pašai ar sevi, jo komentārs tiek glabāts kā ieraksts, kura lauks `parent` norāda komentētā ieraksta identifikatoru, un vienam ierakstam var būt daudzi komentāri;
- starp tabulām "posts" un "likes" ir attiecība viens pret nulli vai vienu, jo visas viena ieraksta "patīk" atzīmes tiek glabātas vienā rindā JSON masīva veidā;
- starp tabulām "users" un "friends" ir divas attiecības viens pret daudziem (lauki `sender` un `receiver`), un tabula "friends" kopumā realizē attiecību daudzi pret daudziem starp lietotājiem. Viens lietotāju pāris tabulā ir ierakstīts tikai vienu reizi;
- starp tabulām "users" un "messages" ir divas attiecības viens pret daudziem, jo lietotājs var nosūtīt un saņemt daudzas ziņas, bet katrai ziņai ir viens sūtītājs un viens saņēmējs;
- starp tabulām "users" un "notifications" ir divas attiecības viens pret daudziem: lauks `user_id` norāda darbības veicēju, bet lauks `content_owner` paziņojuma saņēmēju;
- starp tabulām "notifications" un "notification_seen" ir attiecība viens pret daudziem, jo tabulā "notification_seen" tiek atzīmēts, kurš lietotājs paziņojumu jau ir skatījis;
- tabula "content_i_follow" realizē attiecību daudzi pret daudziem starp lietotājiem un ierakstiem, kuriem viņi seko;
- tabulas "group_members", "group_requests" un "group_invites" realizē attiecības daudzi pret daudziem starp grupām un lietotājiem, bet tabulas "group_invites" lauks `inviter` norāda lietotāju, kurš uzaicinājumu nosūtījis;
- tabulai "users" ir attiecība pašai ar sevi, jo grupas lauks `owner` norāda lietotāju, kurš grupu izveidojis, un viens lietotājs var būt vairāku grupu īpašnieks.

Tabulu “notifications” un “content_i_follow” lauks `content_id` var norādīt gan uz ierakstu, gan uz lietotāja profilu vai grupu. Saistītā satura veidu nosaka lauks `content_type`, tāpēc diagrammā attēlota tikai biežākā saite ar tabulu “posts”.

4.2. Funkcionālais sistēmas modelis

4.2.1. Lietojumgadījumu diagramma

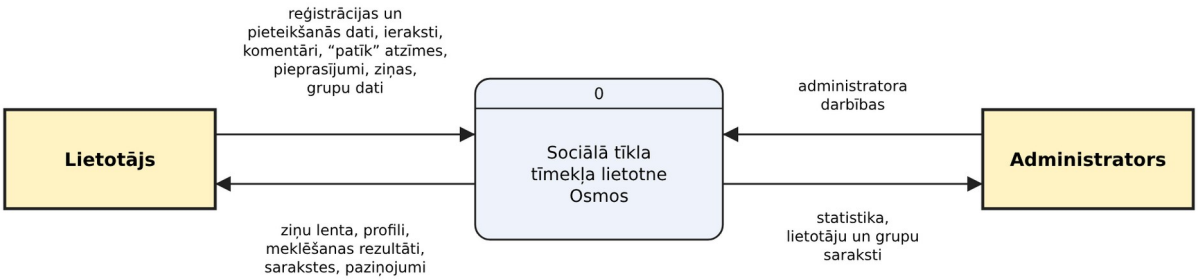
Lietojumgadījumu diagrammā (skat. 4.4. att.) attēlotas sistēmas lietotāju lomas un darbības, kuras katra loma var veikt. Apmeklētājs, kurš nav pieslēdzies, var tikai reģistrēties un pieslēgties, jo viss pārējais sistēmas saturs ir pieejams tikai reģistrētiem lietotājiem. Administrators ir reģistrēta lietotāja paveids, tāpēc diagrammā starp abām lomām attēlota vispārināšanas saite. Administrators var veikt visas reģistrēta lietotāja darbības un papildus tās, kas attēlotas diagrammas apakšdaļā.



4.4. att. Sistēmas lietojumgadījumu diagramma

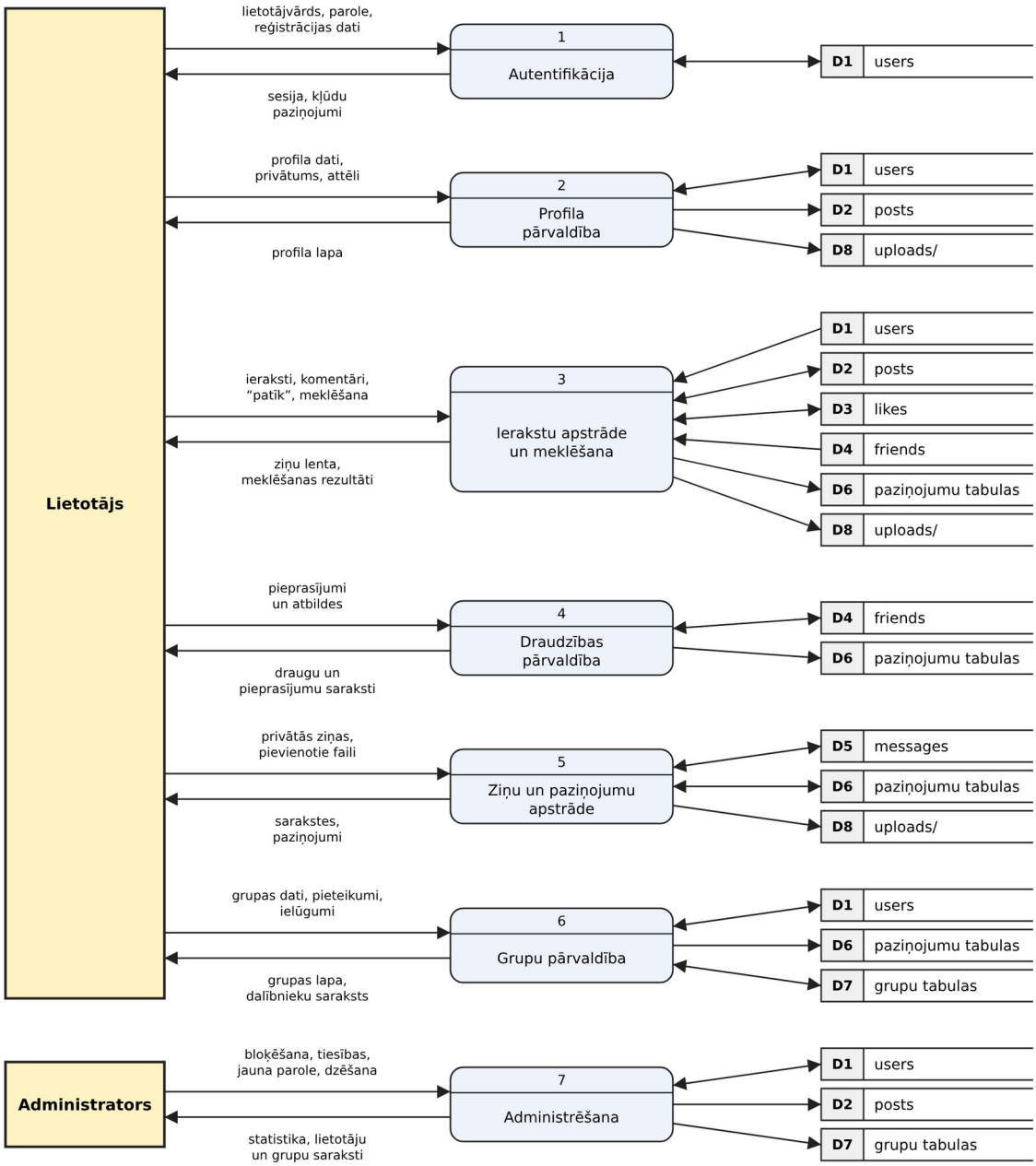
4.2.2. Datu plūsmu modelis

Datu plūsmu diagrammas veidotas Geina un Sārsona notācijā: ārējās vienības attēlotas kā taisnstūri ar biezu kontūru, procesi kā taisnstūri ar noapaļotiem stūriem un numuru augšdaļā, bet datu krātuves kā labajā pusē atvērti taisnstūri ar apzīmējumu D1 līdz D8. Konteksta diagrammā (skat. 4.5. att.) visa sistēma attēlota kā viens process, kas datus apmaina ar divām ārējām vienībām: lietotāju un administratoru.



4.5. att. Sistēmas konteksta diagramma

Pirmā līmeņa datu plūsmu diagrammā (skat. 4.6. att.) sistēma sadalīta septiņos procesos. Datu krātuve D6 apvieno tabulas “notifications”, “notification_seen” un “content_i_follow”, bet D7 tabulas “group_members”, “group_requests” un “group_invites”. Datu krātuve D8 ir mape `uploads`. Pārskatāmības labad viena un tā pati datu krātuve attēlota pie katra procesa, kas to izmanto. Bultas virziens rāda, vai process datus nolasa, ieraksta vai dara abus.



4.6. att. Sistēmas pirmā līmeņa datu plūsmu diagramma

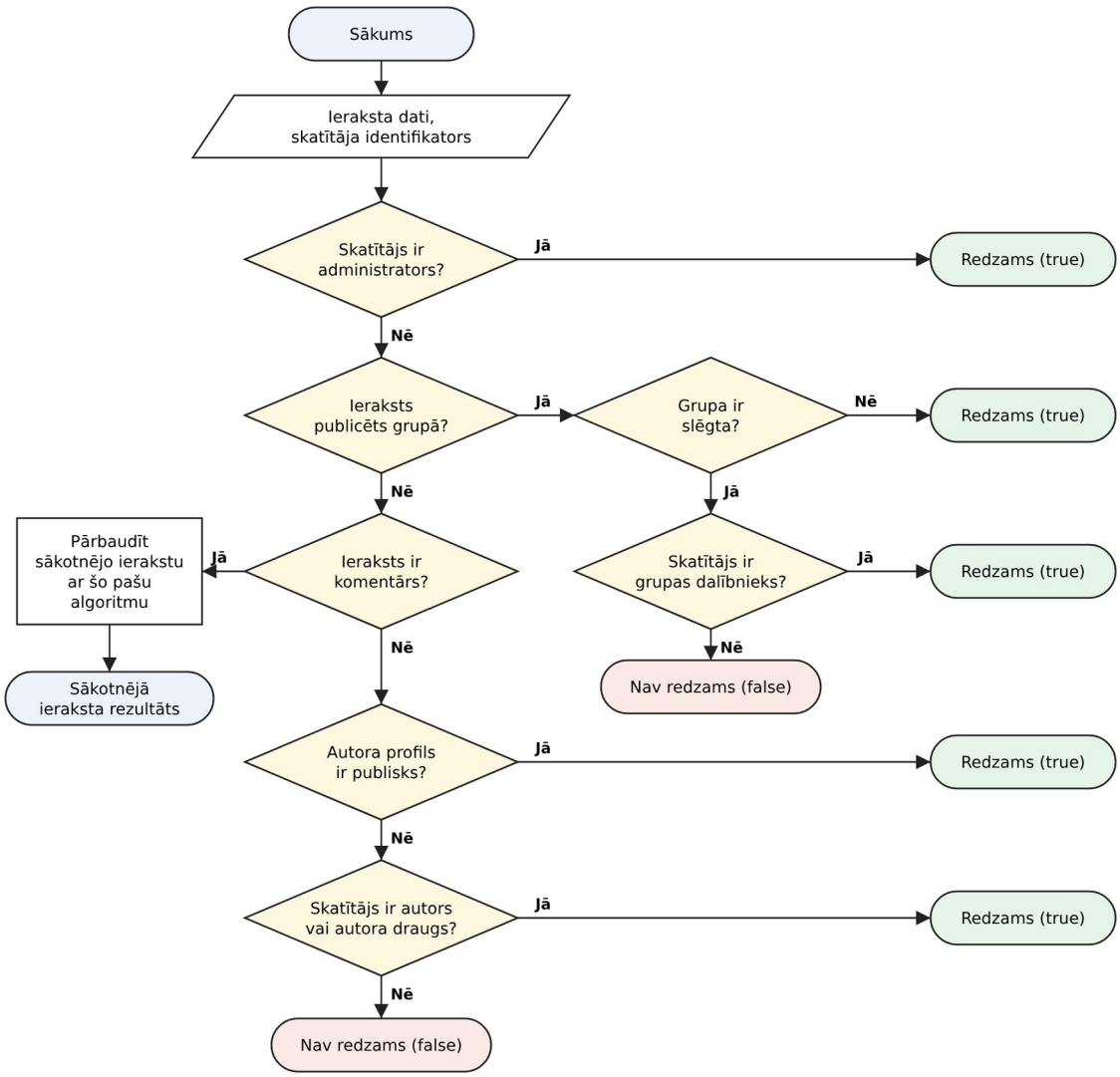
1. **Autentifikācija.** Apmeklētājs ievada reģistrācijas datus vai lietotājvārdu un paroli. Process pārbauda datus, reģistrējoties ieraksta jaunu lietotāju datu krātuvē D1, bet pieslēdzoties nolasa paroles jaucējvērtību un salīdzina to ar ievadīto paroli. Veiksmīgas pieslēgšanās rezultātā tiek izveidota sesija, bet kļūdas gadījumā lietotājs saņem kļūdas paziņojumu.
2. **Profila pārvaldība.** Lietotājs maina profila datus, privātuma iestatījumu vai attēlus. Profila dati tiek saglabāti datu krātuvē D1, attēlu faili mapē `uploads` (D8), bet par jaunu profila vai vāka attēlu tiek publicēts ieraksts (D2).
3. **Ierakstu apstrāde un meklēšana.** Lietotājs publicē ierakstus un komentārus, atzīmē tos ar "patīk" un meklē pēc tēmtura. Process ieraksta datus datu krātuvēs D2

un D3, pievienotos failus D8, bet paziņojumus D6. Veidojot ziņu lentu, process nolasa draugu sarakstu (D4) un autoru privātuma iestatījumus (D1).

4. **Draudzības pārvaldība.** Lietotājs nosūta, apstiprina vai noraida draudzības pieprasījumus. Pieprasījumi un draudzības saites tiek glabātas datu krātuvē D4, bet saņēmējs par pieprasījumu saņem paziņojumu (D6).
5. **Ziņu un paziņojumu apstrāde.** Lietotājs sūta privātās ziņas un apskata paziņojumus. Ziņas tiek glabātas datu krātuvē D5, ziņām pievienotie attēli D8, bet paziņojumu skatīšanas atzīmes D6.
6. **Grupu pārvaldība.** Lietotājs izveido grupu, pievienojas tai, uzaicina draugus un pārvalda dalībniekus. Grupas pamatdati tiek glabāti datu krātuvē D1, dalībnieki, pieprasījumi un uzaicinājumi D7, bet paziņojumi par uzaicinājumiem un lomu maiņu D6.
7. **Administrēšana.** Administrators bloķē lietotājus, piešķir tiesības, atiestata paroles un dzēš saturu. Process maina lietotāju datus D1 un dzēš ierakstus (D2) un grupas (D7), bet administratoram attēlo statistiku un lietotāju sarakstu.

4.2.3. Ieraksta redzamības pārbaudes algoritms

Viens no svarīgākajiem sistēmas algoritmiem ir ieraksta redzamības pārbaude, kas realizēta klases `Post` metodē `can_view_post`. To izsauc ieraksta lapā, attēla skatā, "patīk" atzīmju pievienošanā, tēmturu meklēšanā un populārāko tēmturu saskaitīšanā, tāpēc privātuma noteikumi visās sistēmas daļās darbojas vienādi. Algoritma blokshēma attēlota 4.7. att.



4.7. att. Ieraksta redzamības pārbaudes algoritma blokshēma

Vispirms tiek pārbaudīts, vai skatītājs ir sistēmas administrators, jo administratoram jāredz viss saturs, lai viņš varētu dzēst nepiemērotus ierakstus. Ja ieraksts publicēts grupā, tā redzamību nosaka grupas tips: publiskas grupas ierakstus redz visi lietotāji, bet privātas grupas ierakstus tikai dalībnieki. Komentāram nav sava privātuma iestatījuma, tāpēc algoritms rekursīvi izsauc pats sevi ar komentētā ieraksta datiem. Visbeidzot tiek pārbaudīts autora privātuma iestatījums: ja profils ir publisks, ierakstu redz visi, bet, ja tas pieejams tikai draugiem, ierakstu redz pats autors un viņa draugi. Lai vienas lapas ielādē nebūtu jāatkārto vieni un tie paši vaicājumi, profila pārbaudes rezultāts tiek saglabāts statiskā masīvā katram autora un skatītāja pārim.

4.2.4. Regulāro izteiksmju lietojums

Uzdevumā noteiktās regulārās izteiksmes ir definētas kā konstantes failā `classes/functions.php`, lai vienu un to pašu noteikumu izmantotu visās sistēmas daļās. Regulārās izteiksmes servera pusē apstrādā PHP funkcijas `preg_match`, `preg_match_all`

un `preg_replace_callback` [14], bet tēmturu meklēšanā datubāzē tiek izmantots MariaDB operators REGEXP [6]. Sistēmā izmantotās regulārās izteiksmes ir apkopotas 4.2. tab.

4.2. tabula

Sistēmā izmantotās regulārās izteiksmes

Nosaukums un vieta	Regulārā izteiksme	Nozīme un piemēri
USERNAME_PATTERN (reģistrācija, iestatījumi)	`/^[a-z][a-z0-9_]{2,19}$/`	Lietotājvārds no 3 līdz 20 rakstzīmēm: mazie latīņu burti, cipari un pasvītrojums, pirmajai rakstzīmei jābūt burtam. Derīgi: “anna_b”, “janis2008”; nederīgi: “1karlis”, “jā”, “ab”.
MENTION_PATTERN (ieraksti, komentāri)	`/(^|[^\w&@])@([a-zA-Z0-9_]+)/`	Lietotāja atzīme @lietotājvārds. Pirms simbola @ nedrīkst būt burts vai cipars, tāpēc e-pasta adrese “janis@gmail.com” netiek uztverta kā atzīme.
HASHTAG_PATTERN (ieraksti, populārākie tēmturi)	`/(^|[^\w&#])#([a-zA-Z_][a-zA-Z0-9_]*)/`	Tēmturis #vārds, kas sākas ar burtu vai pasvītrojumu, piemēram, “#php”, “#mysql”. Neatpazīst “C#” un HTML vienības, piemēram, “'”.
Meklēšanas lauks (search.php)	`/^#([a-zA-Z_][a-zA-Z0-9_]*)$/`	Ja viss meklēšanas teksts ir tēmturis, piemēram, “#php”, lietotājs tiek pārvirzīts uz tēmtura lapu.
Tēmtura lapa (tags.php)	`/^[a-zA-Z0-9_]+$/`	Pārbauda tēmturi adresē, pirms tas tiek iekļauts SQL vaicājumā.
Tēmturu meklēšana datubāzē (tags.php)	`(^|[^a-zA-Z0-9_])[#@]php([^a-zA-Z0-9_]|$)`	MariaDB operators REGEXP atrod tēmturi kā atsevišķu vārdu, tāpēc, meklējot “#ph”, netiek atrasti ieraksti ar “#php”.
Lietotājvārdu ieteikumi (ajax.php)	`/^[a-zA-Z0-9_]*$/`	Pārbauda tekstu, ko lietotājs ievadījis pēc simbola @, pirms tiek meklēti atbilstošie lietotāji.

Atzīmju un tēmturu izteiksmēs pirms simbola ir grupa `(^|[^\w&@])` vai `(^|[^\w&#])`, kas prasa, lai simbols būtu teksta sākumā vai aiz rakstzīmes, kas nav burts, cipars, pasvītrojums vai simbols “&”. Tāpēc programmēšanas valodas nosaukums “C#” netiek uztverts kā tēmturis, un e-pasta adrese “janis@gmail.com” netiek uztverta kā lietotāja atzīme. Izteiksmes tiek lietotas tekstam, kas jau apstrādāts ar funkciju `htmlspecialchars`, tāpēc tajā var būt HTML vienības, piemēram, apostrofs tiek pārveidots par “'”. Šāda vienība netiek uztverta kā tēmturis, jo tēmturim jāsākas ar burtu vai pasvītrojumu, bet izslēgtais simbols “&” papildus nodrošina, ka simbols # HTML vienības iekšpusē netiek uzskatīts par tēmtura sākumu. Funkcija `preg_replace_callback` katru atrasto atzīmi aizstāj ar saiti uz lietotāja profilu, bet katru tēmturi ar saiti uz tēmtura lapu. Ja atzīmētais lietotājvārds neeksistē, teksts paliek nemainīts.

5. DATU STRUKTŪRU APRAKSTS

Datubāzes fiziskā struktūra tika iegūta no ER modeļa (skat. 4.1.2. apakšnodaļu), katrai entītijai izveidojot atsevišķu tabulu. Datubāze “osmos_db” sastāv no 11 tabulām. Visas tabulas izmanto InnoDB glabāšanas dzinēju un kodējumu utf8mb4 ar salīdzināšanas kārtulu utf8mb4_general_ci, tāpēc tajās var glabāt latviešu burtus un emocijzīmes. Katrai tabulai ir automātiski palielināms primārās atslēgas lauks `id`. Lietotājiem, grupām un ierakstiem papildus ir nejauši ģenerēts 12 līdz 17 ciparu identifikators (`userid` un `post_id`), kas tiek izmantots saitēs, lai pēc adreses nevarētu uzminēt citu lietotāju vai ierakstu skaitu un secību.

Datubāzē ir šādas tabulas.

- Tabula “users” glabā informāciju par lietotājiem un grupām.
- Tabula “posts” glabā ierakstus un komentārus.
- Tabula “likes” glabā “patīk” atzīmes.
- Tabula “friends” glabā draudzības pieprasījumus un draudzības saites.
- Tabula “messages” glabā privātās ziņas.
- Tabula “notifications” glabā paziņojumus par lietotāju darbībām.
- Tabula “notification_seen” glabā informāciju par to, kurus paziņojumus lietotājs jau ir skatījis.
- Tabula “content_i_follow” glabā informāciju par ierakstiem, kuriem lietotājs seko.
- Tabula “group_members” glabā grupu dalībniekus un to lomas.
- Tabula “group_requests” glabā pieprasījumus pievienoties privātām grupām.
- Tabula “group_invites” glabā uzaicinājumus pievienoties grupām.

Tabula “users” glabā reģistrēto lietotāju un grupu datus. Grupas ierakstam lauks `type` satur vērtību “group”, lauks `first_name` satur grupas nosaukumu, bet lauks `owner` grupas īpašnieka identifikatoru. Lauks `tag_name` ir unikāls, tāpēc divi lietotāji vai lietotājs un grupa nevar izmantot vienu un to pašu lietotājvārdu.

5.1. tabula

Tabulas “users” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Ieraksta identifikators, primārā atslēga
2.	userid	BIGINT	19	Lietotāja vai grupas unikālais identifikators, ko izmanto saitēs
3.	owner	BIGINT	19	Grupas īpašnieka identifikators (lietotājiem 0)
4.	first_name	VARCHAR	50	Lietotāja vārds vai grupas nosaukums
5.	last_name	VARCHAR	50	Lietotāja uzvārds (grupām tukšs)

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
6.	gender	VARCHAR	6	Dzimums: “Male” vai “Female”
7.	type	VARCHAR	10	Ieraksta veids: “profile” (lietotājs) vai “group” (grupa)
8.	group_type	VARCHAR	8	Grupas tips: “public” vai “private”
9.	privacy	VARCHAR	10	Profila privātums: “public” vai “friends”
10.	is_admin	TINYINT	1	1, ja lietotājs ir sistēmas administrators
11.	blocked	TINYINT	1	1, ja administrators ir bloķējis kontu
12.	password	VARCHAR	100	Paroles jaucējvērtība, kas iegūta ar funkciju `password_hash`
13.	date	TIMESTAMP	–	Reģistrācijas vai grupas izveides datums un laiks
14.	profile_image	VARCHAR	1000	Ceļš uz profila attēlu mapē `uploads`
15.	cover_image	VARCHAR	1000	Ceļš uz vāka attēlu mapē `uploads`
16.	about	VARCHAR	1024	Apraksts par lietotāju vai grupu
17.	tag_name	VARCHAR	100	Unikāls lietotājvārds, ar kuru lietotājs pieslēdzas un tiek atzīmēts ierakstos
18.	online	INT	11	Pēdējās aktivitātes laiks sekundēs kopš 1970. gada 1. janvāra

Tabula “posts” glabā lietotāju ierakstus un komentārus. Komentāram lauks `parent` satur komentētā ieraksta identifikatoru, bet ierakstam šī lauka vērtība ir 0. Komentāru un “patīk” atzīmju skaits tiek glabāts arī ieraksta rindā, lai ziņu lentā tas nebūtu jāskaita ar atsevišķiem vaicājumiem.

5.2. tabula

Tabulas “posts” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	19	Ieraksta identifikators, primārā atslēga
2.	post_id	BIGINT	19	Unikālais ieraksta identifikators, ko izmanto saitēs
3.	user_id	BIGINT	19	Ieraksta autora identifikators
4.	owner	BIGINT	19	Grupas identifikators, ja ieraksts publicēts grupā, citādi 0
5.	post	TEXT	–	Ieraksta teksts
6.	image	VARCHAR	500	Ceļš uz pievienoto attēlu vai video
7.	comments	INT	11	Komentāru skaits
8.	likes	INT	11	“Patīk” atzīmju skaits
9.	date	TIMESTAMP	–	Publicēšanas datums un laiks
10.	has_image	TINYINT	1	1, ja ierakstam pievienots fails
11.	is_profile_image	TINYINT	1	1, ja ieraksts publicēts, mainot profila attēlu
12.	is_cover_image	TINYINT	1	1, ja ieraksts publicēts, mainot vāka attēlu

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
13.	parent	BIGINT	20	Komentētā ieraksta identifikators (ierakstiem 0)

Tabula “likes” glabā visas viena ieraksta vai komentāra “patīk” atzīmes vienā rindā. Lauks `likes` satur JSON masīvu, kura katrs elements satur atzīmējušā lietotāja identifikatoru un atzīmes laiku, piemēram, `[{"user_id":"473897691038","date":"2026-10-12 18:40:00"}]`. Lauku pāris `type` un `contentid` ir unikāls.

5.3. tabula

Tabulas “likes” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	type	VARCHAR	10	Atzīmētā satura veids: “post” vai “comment”
3.	contentid	BIGINT	20	Atzīmētā ieraksta vai komentāra identifikators
4.	likes	TEXT	–	JSON masīvs ar atzīmējušajiem lietotājiem un atzīmes laiku

Tabula “friends” glabā draudzības pieprasījumus. Kamēr saņēmējs nav atbildējis, pieprasījuma statuss ir “pending”, bet pēc apstiprināšanas tas mainās uz “accepted”. Noraidīts vai atsaukts pieprasījums, kā arī pārtraukta draudzība tiek dzēsta.

5.4. tabula

Tabulas “friends” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	sender	BIGINT	20	Pieprasījuma sūtītāja identifikators
3.	receiver	BIGINT	20	Pieprasījuma saņēmēja identifikators
4.	status	VARCHAR	10	Statuss: “pending” (gaida atbildi) vai “accepted” (draugi)
5.	date	DATETIME	–	Pieprasījuma nosūtīšanas vai apstiprināšanas datums un laiks

Tabula “messages” glabā privātās ziņas. Visām viena lietotāju pāra ziņām ir kopīgs sarunas identifikators `msgid`. Dzēšot ziņu, tā netiek izņemta no tabulas, bet tiek atzīmēta kā dzēsta tikai dzēsējam, tāpēc otrs sarunas dalībnieks to joprojām redz.

5.5. tabula

Tabulas “messages” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	msgid	VARCHAR	60	Sarunas identifikators
3.	sender	BIGINT	20	Sūtītāja identifikators

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
4.	receiver	BIGINT	20	Saņēmēja identifikators
5.	message	TEXT	–	Ziņas teksts
6.	file	VARCHAR	500	Ceļš uz pievienoto attēlu
7.	seen	TINYINT	1	1, ja saņēmējs ziņu ir izlasījis
8.	deleted_sender	TINYINT	1	1, ja sūtītājs ziņu ir dzēsis
9.	deleted_receiver	TINYINT	1	1, ja saņēmējs ziņu ir dzēsis
10.	date	DATETIME	–	Nosūtīšanas datums un laiks

Tabula “notifications” glabā paziņojumus par lietotāju darbībām. Lauks `activity` nosaka darbības veidu, bet lauks `content_type` saistītā satura veidu, pēc kura sistēma izveido saiti uz ierakstu, profilu vai grupu.

5.6. tabula

Tabulas “notifications” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	user_id	BIGINT	20	Darbības veicēja identifikators
3.	activity	VARCHAR	30	Darbība: “like”, “comment”, “tag”, “friend_request”, “friend_accept”, “invite” vai “role”
4.	content_id	BIGINT	20	Saistītā ieraksta, profila vai grupas identifikators
5.	content_owner	BIGINT	20	Paziņojuma saņēmēja (satura īpašnieka) identifikators
6.	content_type	VARCHAR	30	Satura veids: “post”, “comment”, “profile” vai “group”
7.	date	DATETIME	–	Paziņojuma izveides datums un laiks

Tabula “notification_seen” glabā atzīmes par skatītajiem paziņojumiem. Paziņojums tiek uzskatīts par jaunu, kamēr šajā tabulā nav rindas ar attiecīgo lietotāju un paziņojumu. Lauku pāris `userid` un `notification_id` ir unikāls.

5.7. tabula

Tabulas “notification_seen” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	userid	BIGINT	20	Lietotāja identifikators
3.	notification_id	BIGINT	20	Skatītā paziņojuma identifikators

Tabula “content_i_follow” glabā informāciju par ierakstiem, kuriem lietotājs seko. Lietotājs sāk sekot savam ierakstam, to publicējot, un citu lietotāju ierakstam, to komentējot. Pēc šīs tabulas datiem lietotājam tiek rādīti paziņojumi par jauniem komentāriem.

5.8. tabula

Tabulas “content_i_follow” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	user_id	BIGINT	20	Lietotāja identifikators
3.	content_id	BIGINT	20	Ieraksta, kuram lietotājs seko, identifikators
4.	content_type	VARCHAR	30	Satura veids: “post”, “comment” vai “profile”
5.	disabled	TINYINT	1	Paredzēts sekošanas izslēgšanai (pašreizējā versijā vienmēr 0)
6.	date	DATETIME	–	Sekošanas sākuma datums un laiks

Tabula “group_members” glabā grupu dalībniekus. Grupas īpašnieks tiek noteikts pēc tabulas “users” lauka `owner`, tāpēc viņam vienmēr ir visas grupas tiesības. Kad dalībnieks pamet grupu vai tiek no tās noņemts, rinda netiek dzēsta, bet lauka `disabled` vērtība tiek mainīta uz 1.

5.9. tabula

Tabulas “group_members” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	userid	BIGINT	20	Dalībnieka identifikators
3.	groupid	BIGINT	20	Grupas identifikators
4.	role	VARCHAR	10	Loma grupā: “member”, “moderator” vai “admin”
5.	disabled	TINYINT	1	1, ja lietotājs vairs nav grupas dalībnieks

Tabula “group_requests” glabā lietotāju pieprasījumus pievienoties privātām grupām. Pēc tam, kad moderators vai administrators pieprasījumu apstiprina vai noraida, lauka `disabled` vērtība tiek mainīta uz 1.

5.10. tabula

Tabulas “group_requests” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	groupid	BIGINT	20	Grupas identifikators
3.	userid	BIGINT	20	Pieprasījuma sūtītāja identifikators
4.	disabled	TINYINT	1	1, ja pieprasījums ir apstrādāts

Tabula “group_invites” glabā uzaicinājumus pievienoties grupām. Grupas dalībnieks var uzaicināt savus draugus, un uzaicinātais lietotājs uzaicinājumu var pieņemt vai noraidīt.

5.11. tabula

Tabulas “group_invites” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	BIGINT	20	Ieraksta identifikators, primārā atslēga
2.	groupid	BIGINT	20	Grupas identifikators
3.	userid	BIGINT	20	Uzaicinātā lietotāja identifikators
4.	disabled	TINYINT	1	1, ja uzaicinājums ir pieņemts vai noraidīts
5.	inviter	BIGINT	20	Uzaicinātāja identifikators

Lai paātrinātu biežāk izmantotos vaicājumus, tabulām ir izveidoti indeksi. Piemēram, tabulā “posts” indeksēti lauki `user_id`, `owner` un `date`, ko izmanto ziņu lentas un profila ierakstu atlasē, tabulā “friends” lauki `receiver` un `status`, ko izmanto draudzības pieprasījumu atlasē, bet tabulā “notifications” lauku pāris `content_owner` un `date`, ko izmanto paziņojumu saraksta un neskatīto paziņojumu skaita noteikšanā.

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Sistēma ir tīmekļa lietotne, tāpēc lietotājam pietiek ar pārlūkprogrammu, bet serverī jādarbojas tīmekļa serverim, PHP interpretatoram un datubāzei. Sistēmas lietošanai ir šādas prasības.

- Servera dators:
  - operētājsistēma Microsoft Windows 10 vai jaunāka, Apple macOS 11 vai jaunāka vai Linux;
  - XAMPP 8.2 vai jaunāka versija, kas ietver Apache 2.4, PHP 8 un MariaDB 10.4 [3];
  - vismaz 2 GB operatīvās atmiņas un vismaz 1 GB brīvas vietas diskā.
- Klienta dators:
  - jaunākā Google Chrome, Mozilla Firefox, Microsoft Edge vai Apple Safari versija ar ieslēgtu JavaScript;
  - ekrāns, kura izšķirtspēja ir vismaz 1024 × 768 pikseļi.

Ja sistēmu lieto vienā datorā, serveris un klients ir viens un tas pats dators. Ja sistēmu lieto vairāki lietotāji, viņi to atver no citiem datoriem tajā pašā tīklā, pārlūkprogrammā ievadot servera datora IP adresi, ja to neierobežo servera datora ugunsmūris.

6.2. Sistēmas instalācija un palaišana

Sistēmas faili ir apvienoti mapē `osmos`. Tajā atrodas PHP faili, mape `classes` ar klasēm, mape `images` ar noklusējuma attēliem, mape `uploads` augšupielādētajiem failiem un mape `database` ar datubāzes izveides skriptiem. Lai uzstādītu sistēmu, jāizpilda šādi soļi.

1. Lejupielādējiet XAMPP no vietnes https://www.apachefriends.org un instalējiet to ar noklusējuma iestatījumiem.
2. Atveriet XAMPP vadības paneli (Windows vidē “XAMPP Control Panel”, macOS vidē “manager-osx”) un palaidiet serverus Apache un MySQL. Abu serveru statusam jākļūst zaļam.
3. Nokopējiet mapi `osmos` uz XAMPP mapi `htdocs`. Windows vidē tā parasti ir `C:\xampp\htdocs`, bet macOS vidē `/Applications/XAMPP/xamppfiles/htdocs`. Kopējot jāpārliecinās, ka līdzi

ir nokopēts arī slēptais fails `.htaccess`, jo bez tā neatveras neviena lapa, izņemot sākumlapu.

4. Pārlūkprogrammā atveriet phpMyAdmin adresē http://localhost/phpmyadmin, izvēlieties "New" un izveidojiet datubāzi `osmos_db` ar salīdzināšanas kārtulu `utf8mb4_general_ci`.
5. Atveriet izveidoto datubāzi, izvēlieties cilni "Import" un importējiet vienu no mapes `database` failiem:
    - `osmos_db.sql`, lai iegūtu tukšu sistēmu bez lietotājiem;
    - `osmos_demo.sql`, lai iegūtu sistēmu ar demonstrācijas datiem (lietotāji, ieraksti, grupas un ziņas). Visu demonstrācijas lietotāju parole ir `osmos123`, bet administratora lietotājvārds ir `gustavs`.
6. Ja serveris darbojas macOS vai Linux vidē, piešķiriet tīmekļa serverim tiesības rakstīt mapē `uploads`, terminālī izpildot šādas komandas (Windows vidē šis solis nav nepieciešams):
    - `cd /Applications/XAMPP/xamppfiles/htdocs/osmos`
    - `chmod -R 777 uploads`
7. Pārlūkprogrammā atveriet adresi http://localhost/osmos. Tiek atvērta pieslēgšanās lapa.

Ja tika importēts fails `osmos_db.sql`, pirmo administratoru norāda datubāzē. Vispirms sistēmā jāreģistrējas, pēc tam phpMyAdmin cilnē "SQL" jāizpilda vaicājums `UPDATE users SET is_admin = 1 WHERE tag_name = 'lietotājvārds';`, kurā vārda "lietotājvārds" vietā ierakstīts reģistrētā lietotāja lietotājvārds. Turpmāk administratora tiesības citiem lietotājiem var piešķirt administratora panelī.

Datubāzes savienojuma iestatījumi atrodas failā `classes/connect.php`. Pēc noklusējuma sistēma pieslēdzas datubāzei `osmos_db` ar XAMPP noklusējuma lietotāju `root` bez paroles. Ja datubāzes lietotājs vai parole ir citi, tie jānorāda šajā failā. Ja, atverot lapu, parādās paziņojums "Object not found!", visticamāk, mapē nav faila `.htaccess` vai Apache konfigurācijā ir izslēgts modulis mod_rewrite. Ja, augšupielādējot attēlu, parādās kļūda par to, ka serveris nevar saglabāt failus, jāpārbauda mapes `uploads` rakstīšanas tiesības.

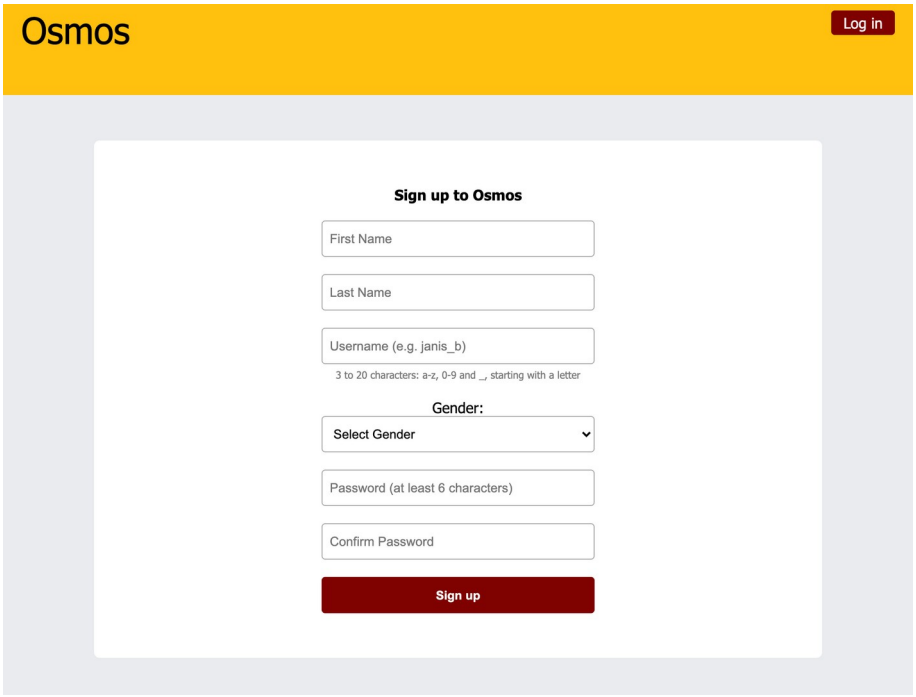
6.3. Programmas apraksts

Visas lapas, izņemot reģistrācijas un pieslēgšanās lapu, ir pieejamas tikai pieslēgušamies lietotājiem. Lietotāja saskarne ir angļu valodā, tāpēc tālāk pogu, ciļņu un

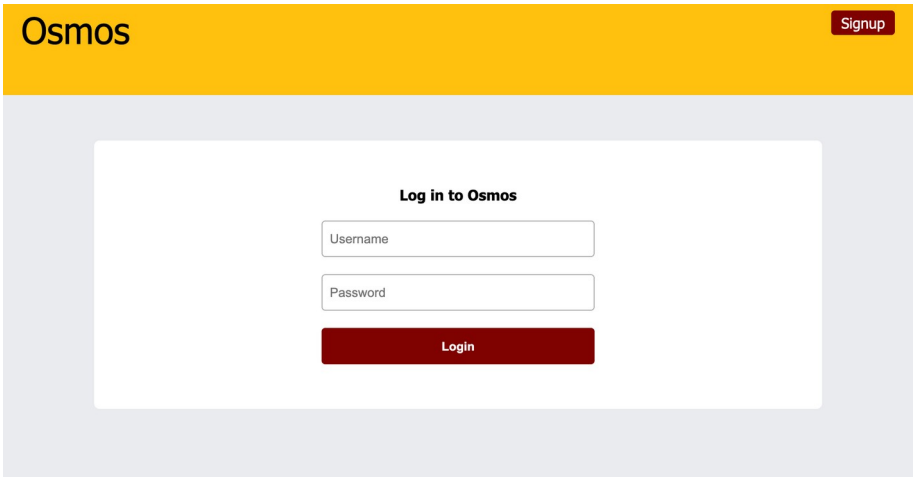
paziņojumu nosaukumi norādīti tādā formā, kādā tie redzami ekrānā. Ekrānattēlos redzami demonstrācijas dati.

Reģistrācija un pieslēgšanās.

Jauns lietotājs atver reģistrācijas lapu, nospiežot pogu “Signup” pieslēgšanās lapas augšējā labajā stūrī. Reģistrācijas formā jāievada vārds, uzvārds, lietotājvārds, dzimums un divas reizes parole (skat. 6.1. att.). Zem lietotājvārda lauka ir atgādinājums par lietotājvārda noteikumiem. Pēc veiksmīgas reģistrācijas lietotājs tiek novirzīts uz pieslēgšanās lapu (skat. 6.2. att.), kurā jāievada lietotājvārds un parole un jānospiež poga “Login”.



6.1. att. Reģistrācijas forma



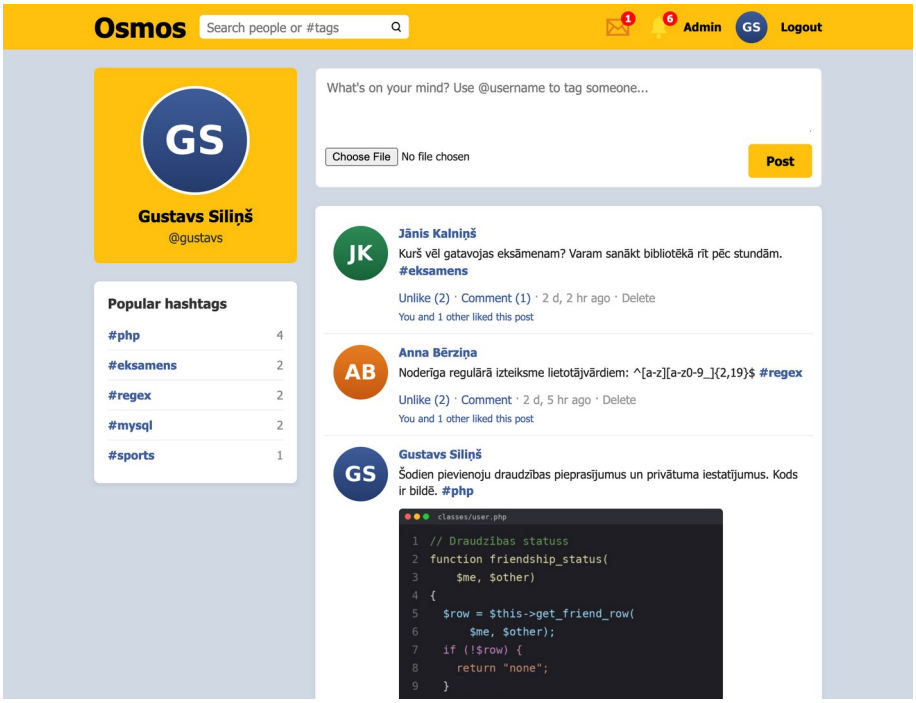
6.2. att. Pieslēgšanās forma

Ziņu lenta.

Lapā “Timeline”, kas tiek atvērta, nospiežot uz logotipa “Osmos”, redzami lietotāja un viņa draugu ieraksti, sākot ar jaunāko (skat. 6.3. att.). Lapas augšdaļā ir ieraksta publicēšanas

forma: tekstu ievada laukā “What's on your mind?”, ar pogu “Choose File” var pievienot JPEG attēlu vai MP4 video, bet ar pogu “Post” ieraksts tiek publicēts. Ierakstot simbolu @ un lietotājvārda sākumu, zem teksta lauka parādās atbilstošo lietotāju saraksts, no kura lietotājvārdu var izvēlēties ar klikšķi. Kreisajā pusē redzama lietotāja profila kartīte un bloks “Popular hashtags” ar biežāk lietotajiem tēmturiem.

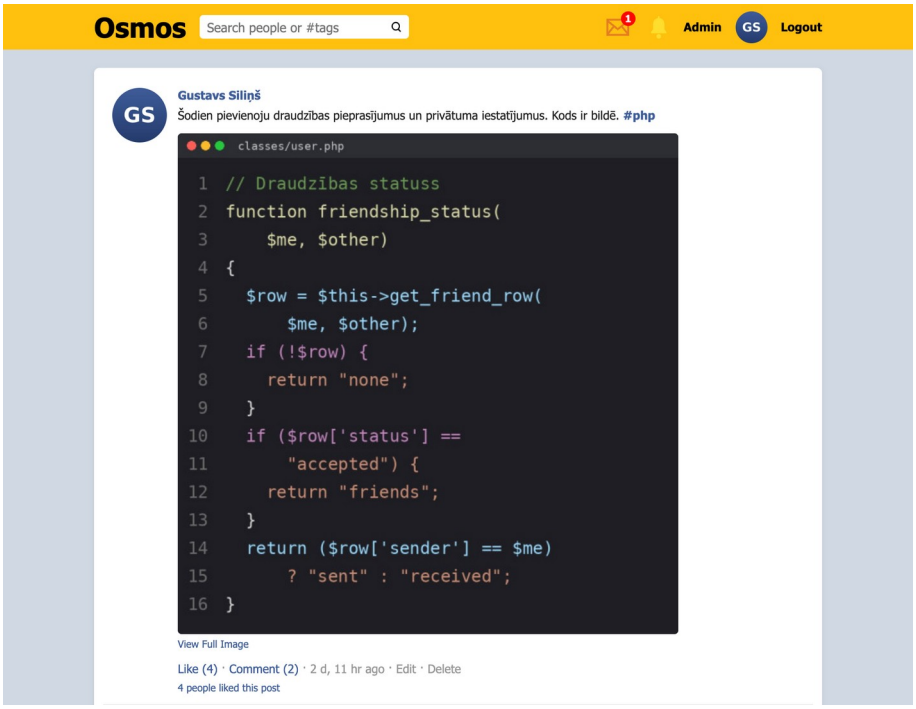
Zem katra ieraksta ir saites “Like” (atzīmēt ar “patīk”), “Comment” (atvērt ieraksta lapu un komentēt), ieraksta publicēšanas laiks, bet saviem ierakstiem arī saites “Edit” un “Delete”. Lapas galvenē ir meklēšanas lauks, aploksnes ikona ar nelasīto ziņu skaitu, zvana ikona ar jauno paziņojumu skaitu, saite uz savu profilu un poga “Logout”. Administratoriem galvenē redzama arī saite “Admin”.



6.3. att. Ziņu lenta

Ieraksts un komentāri.

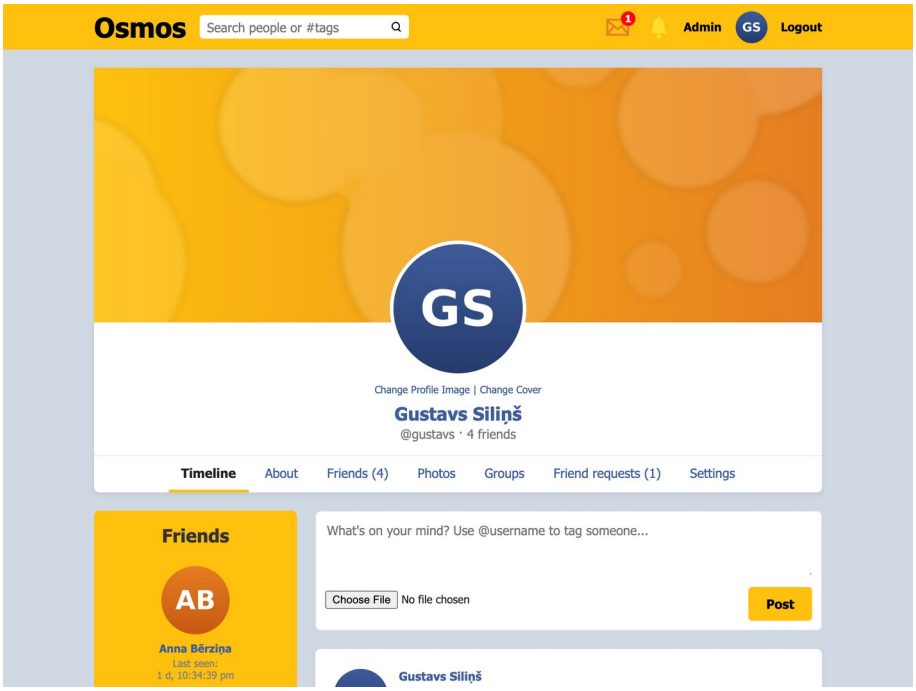
Nospiežot uz saites “Comment”, tiek atvērta ieraksta lapa (skat. 6.4. att.). Tajā redzams ieraksts ar visiem komentāriem un komentāra pievienošanas forma. Komentāram var pievienot attēlu, un komentārus var atzīmēt ar “patīk” un atbildēt uz tiem ar saiti “Reply”. Nospiežot uz ieraksta attēla vai saites “View Full Image”, attēls tiek atvērts pilnā izmērā. Savu publicēto attēlu var izvēlēties par profila vai vāka attēlu ar pogām “Set as Profile Picture” un “Set as Cover Image”.



6.4. att. Ieraksta lapa ar komentāriem

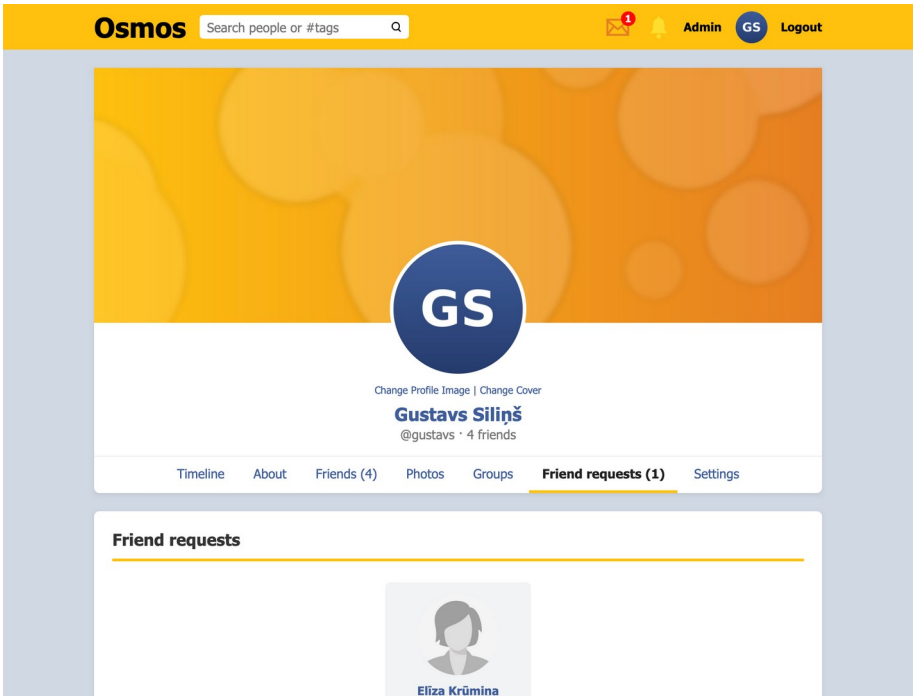
Profils un draugi.

Profila lapā redzams vāka attēls, profila attēls, vārds, lietotājvārds un draugu skaits (skat. 6.5. att.). Savā profilā ar saitēm “Change Profile Image” un “Change Cover” var nomainīt attēlus. Profila saturs ir sadalīts cilnēs: “Timeline” (lietotāja ieraksti), “About” (apraksts par lietotāju), “Friends” (draugi), “Photos” (publicētie attēli) un “Groups” (grupas). Savā profilā lietotājs redz arī cilnes “Friend requests” un “Settings”. Kreisajā pusē redzams draugu saraksts ar informāciju par to, vai draugs pašlaik ir tiešsaistē.



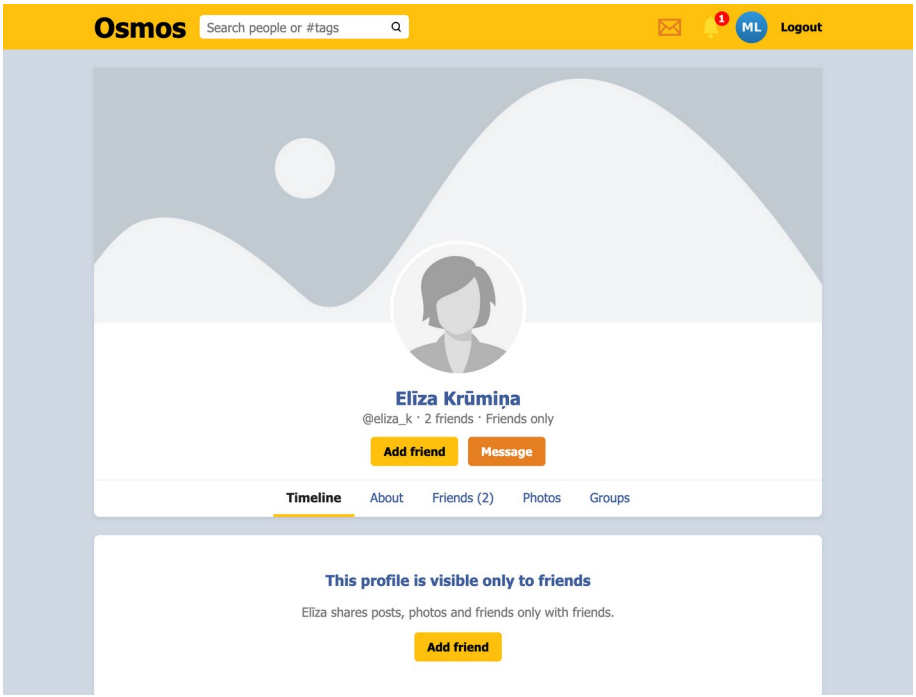
6.5. att. Lietotāja profila lapa

Cita lietotāja profilā ir pogas “Add friend” un “Message”. Pēc pieprasījuma nosūtīšanas poga “Add friend” tiek aizstāta ar pogu “Cancel request”, bet draugam ar pogu “Unfriend”. Saņemtie un nosūtītie pieprasījumi ir redzami cilnē “Friend requests” (skat. 6.6. att.), kurā saņemto pieprasījumu var apstiprināt ar pogu “Accept” vai noraidīt ar pogu “Decline”.



6.6. att. Saņemtie draudzības pieprasījumi

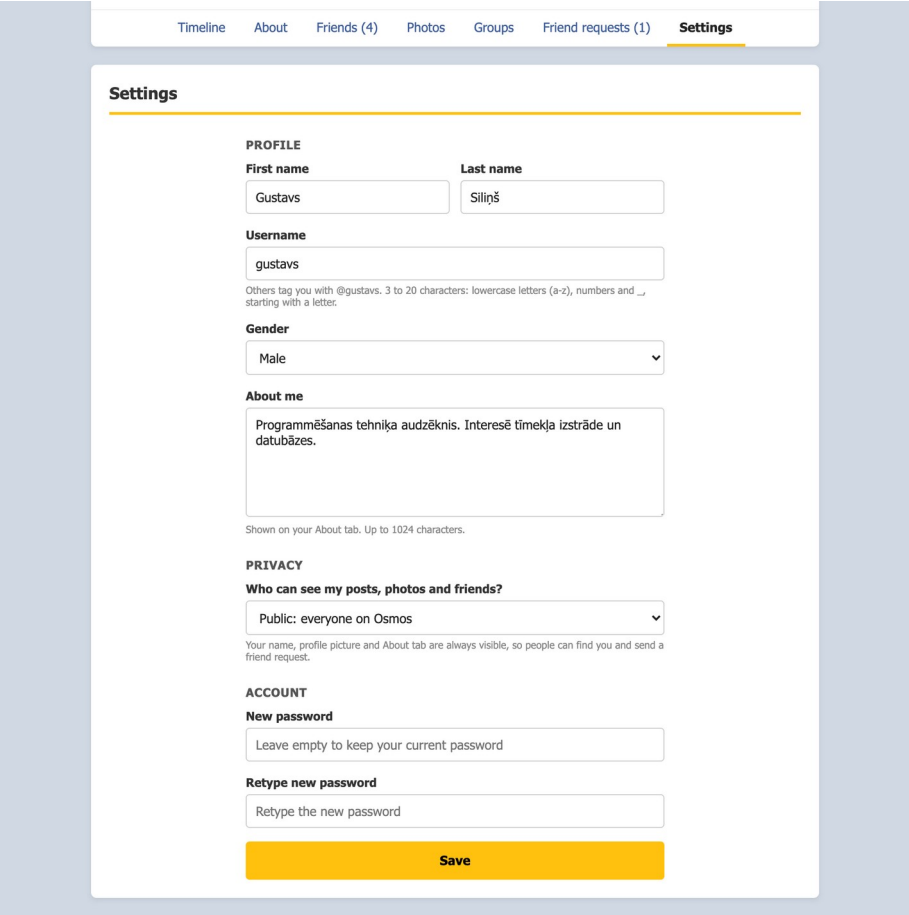
Ja lietotājs savos iestatījumos izvēlējies, ka profila saturu redz tikai draugi, citi lietotāji viņa profilā redz tikai vārdu, profila attēlu, cilni “About” un paziņojumu “This profile is visible only to friends” (skat. 6.7. att.).



6.7. att. Tikai draugiem redzams profils

Iestatījumi.

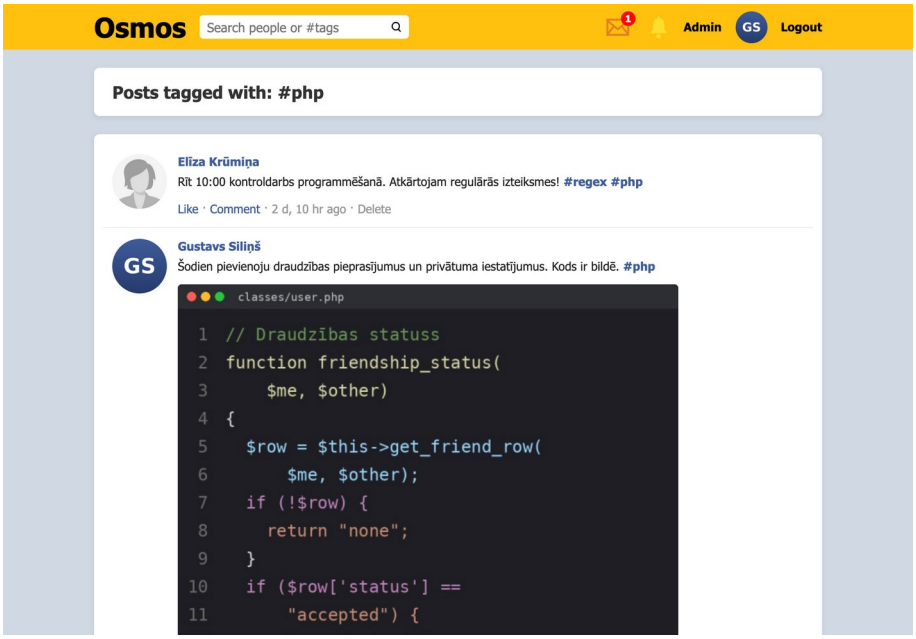
Cilnē “Settings” lietotājs var mainīt vārdu, uzvārdu, lietotājvārdu, dzimumu un aprakstu par sevi (skat. 6.8. att.). Sadaļā “Privacy” tiek izvēlēts, kas redz lietotāja ierakstus, fotoattēlus un draugus: “Public: everyone on Osmos” vai “Friends only”. Formas apakšdaļā var nomainīt paroli, divas reizes ievadot jauno paroli. Ja paroles lauki ir tukši, parole netiek mainīta. Izmaiņas tiek saglabātas ar pogu “Save”.



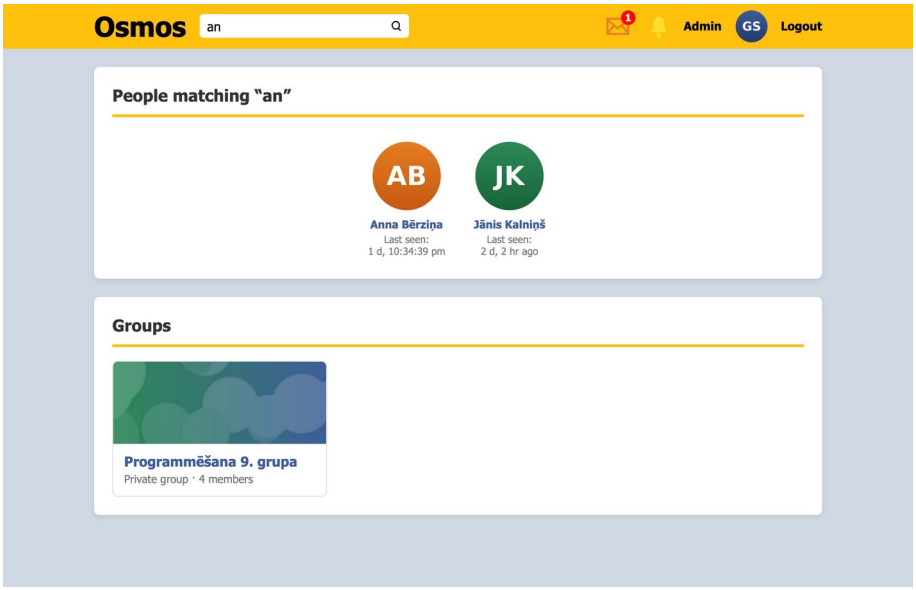
6.8. att. Profila iestatījumi

Tēmturi un meklēšana.

Nospiežot uz tēmtura ierakstā vai blokā “Popular hashtags”, tiek atvērta tēmtura lapa ar visiem ierakstiem, kuros šis tēmturis ir lietots un kurus lietotājs drīkst redzēt (skat. 6.9. att.). Tēmtura lapu var atvērt arī, meklēšanas laukā ierakstot tēmturi, piemēram, “#php”. Ja meklēšanas laukā ievada citu tekstu, tiek atrasti lietotāji un grupas, kuru vārdā, uzvārdā, lietotājvārdā vai nosaukumā ir šis teksts (skat. 6.10. att.).



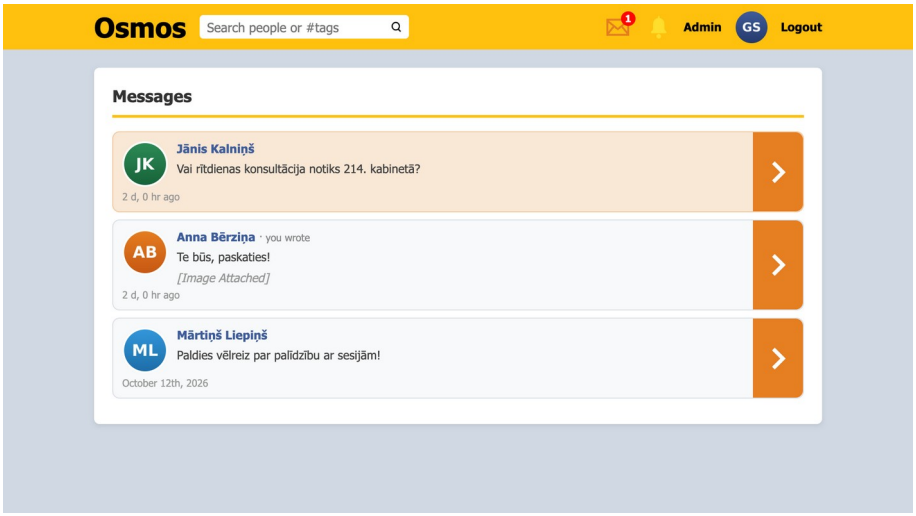
6.9. att. Tēmtura lapa



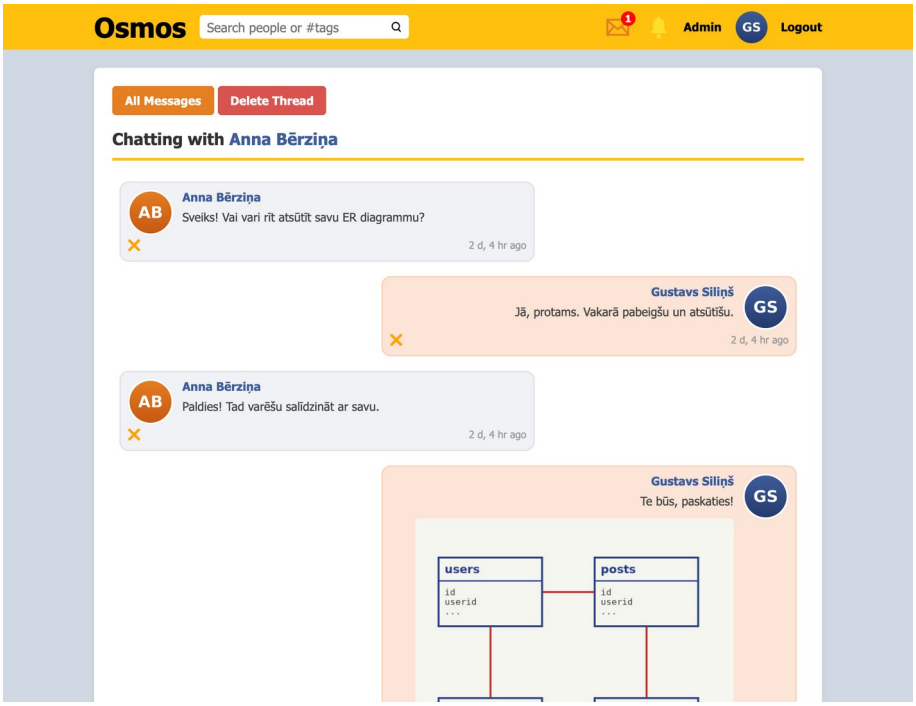
6.10. att. Meklēšanas rezultāti

Privātās ziņas.

Jaunu sarunu sāk, cita lietotāja profilā nospiežot pogu “Message”. Nospiežot aploksnes ikonu lapas galvenē, tiek atvērts sarunu saraksts, kurā katrai sarunai redzama pēdējā ziņa (skat. 6.11. att.). Atvērtajā sarunā lietotāja ziņas redzamas labajā pusē, bet sarunas biedra ziņas kreisajā pusē (skat. 6.12. att.). Ziņu ievada laukā “Write your message here”, tai var pievienot attēlu un nosūtīt ar pogu “Send”. Atsevišķu ziņu var dzēst ar krustiņu pie tās, bet visu sarunu ar pogu “Delete Thread”.



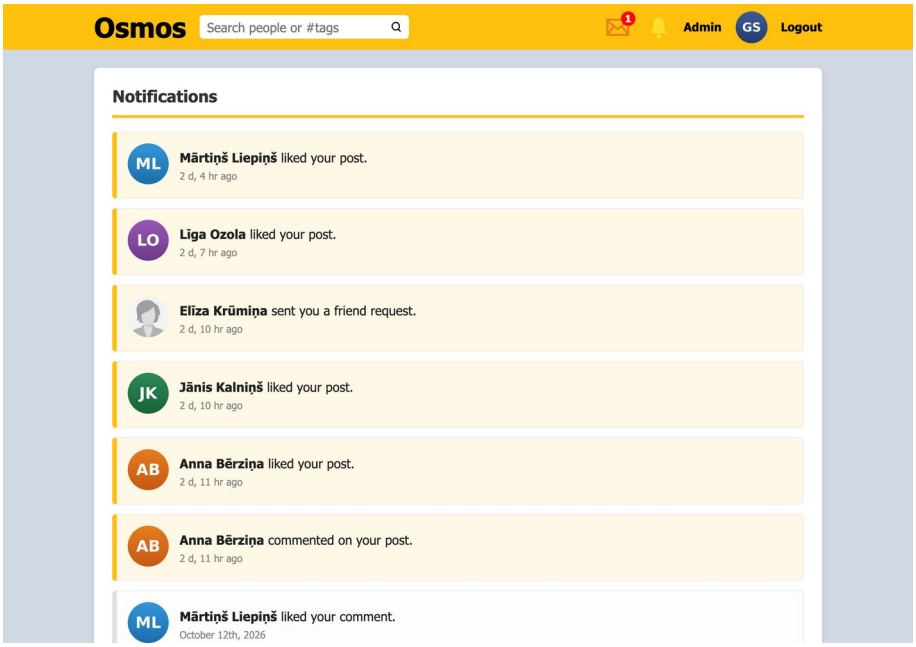
6.11. att. Sarunu saraksts



6.12. att. Saruna ar citu lietotāju

Paziņojumi.

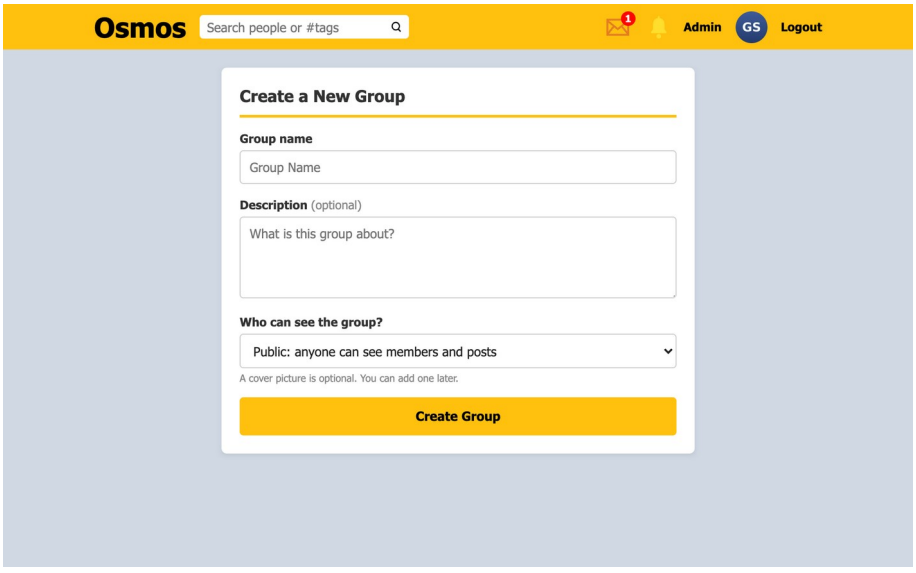
Nospiežot zvana ikonu lapas galvenē, tiek atvērts paziņojumu saraksts (skat. 6.13. att.). Jaunie paziņojumi ir izcelti ar dzeltenu fonu. Nospiežot uz paziņojuma, tiek atvērts saistītais ieraksts, profils vai grupa.



6.13. att. Paziņojumu saraksts

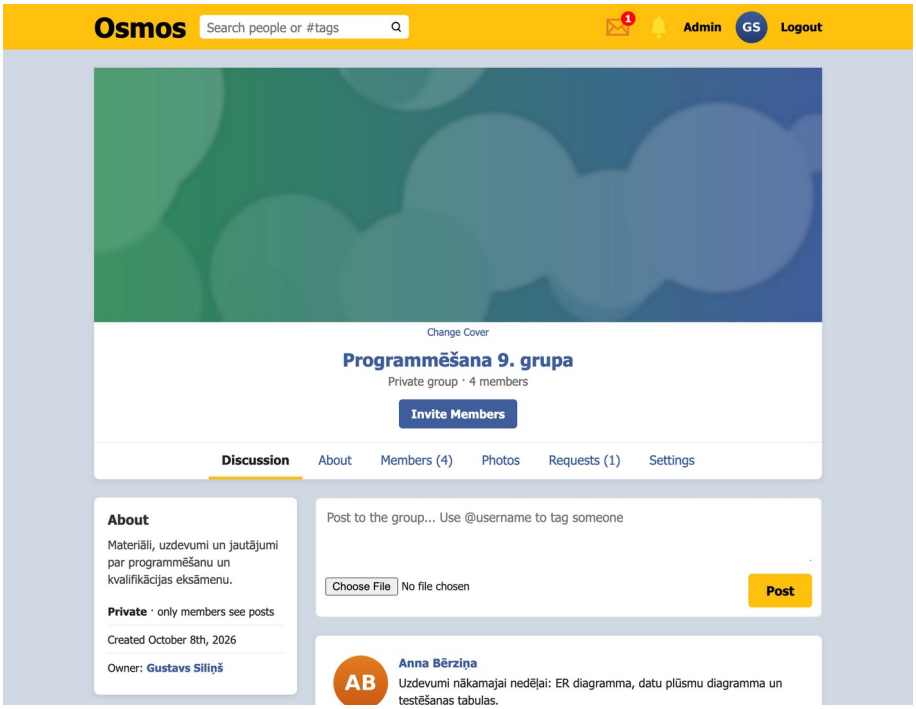
Grupas.

Jaunu grupu izveido profila cilnē “Groups”, nospiežot pogu “+ Create Group”. Grupas izveides formā jāievada nosaukums, neobligāts apraksts un jāizvēlas, vai grupa ir publiska vai privāta (skat. 6.14. att.). Grupas veidotājs kļūst par tās īpašnieku.



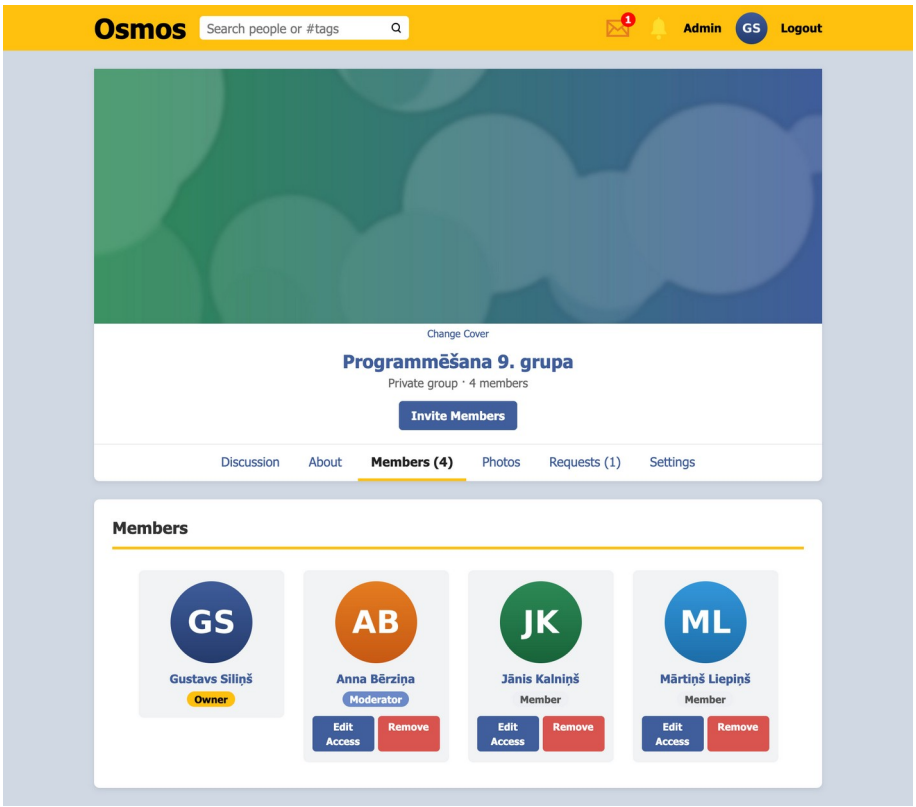
6.14. att. Grupas izveides forma

Grupas lapā (skat. 6.15. att.) ir cilnes “Discussion” (grupas ieraksti), “About” (apraksts), “Members” (dalībnieki) un “Photos” (attēli), bet moderatoriem un administratoriem arī “Requests” (pievienošanās pieprasījumi) un “Settings” (iestatījumi). Lietotājs, kurš nav dalībnieks, redz pogu “Join Group” publiskai grupai vai “Request to Join” privātai grupai. Dalībnieki ar pogu “Invite Members” var uzaicināt grupā savus draugus, bet ar pogu “Leave Group” pamest grupu.



6.15. att. Grupas lapa

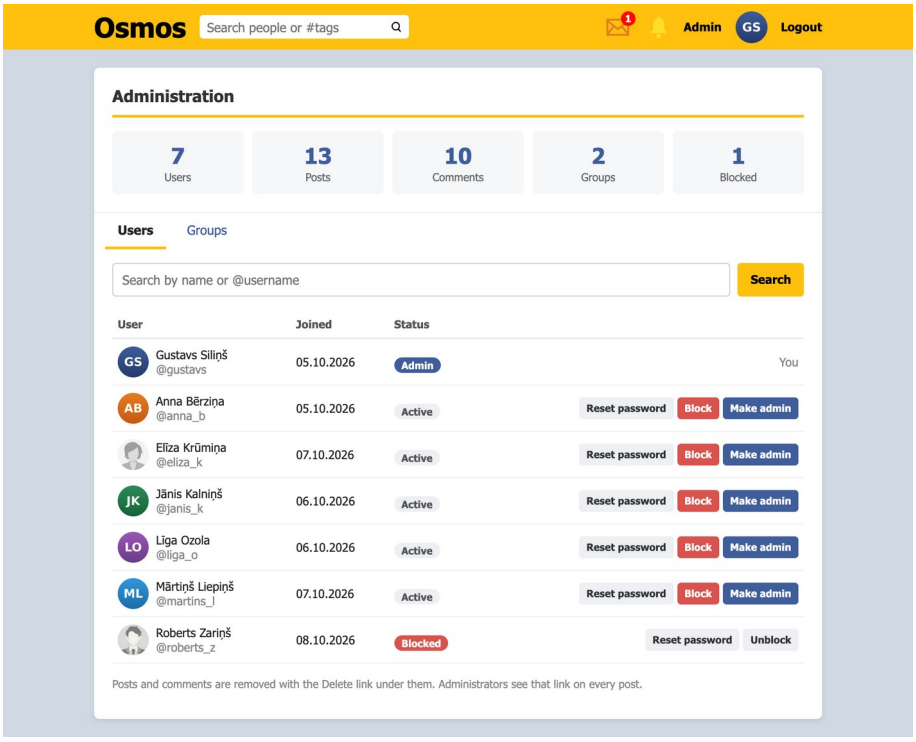
Cilnē “Members” redzami grupas dalībnieki un viņu lomas (skat. 6.16. att.). Grupas administrators var mainīt dalībnieka lomu uz “Member”, “Moderator” vai “Admin” un noņemt dalībnieku no grupas. Īpašnieka lomu mainīt nevar.



6.16. att. Grupas dalībnieki

Administratora panelis.

Administratora panelis tiek atvērts ar saiti “Admin” lapas galvenē (skat. 6.17. att.). Paneļa augšdaļā ir sistēmas statistika, bet zem tās cilnes “Users” un “Groups”. Cilnē “Users” lietotājus var meklēt pēc vārda vai lietotājvārda. Pie katra lietotāja ir pogas “Reset password” (izveidot pagaidu paroli), “Block” vai “Unblock” (bloķēt vai atbloķēt kontu) un “Make admin” vai “Remove admin” (piešķirt vai noņemt administratora tiesības). Cilnē “Groups” ir visu grupu saraksts ar pogu “Delete”. Ierakstus un komentārus administrators dzēš ar saiti “Delete”, kas viņam redzama pie katra ieraksta.



6.17. att. Administratora panelis

7. PROGRAMMATŪRAS TESTĒŠANA

Sistēmas darbība tika pārbaudīta ar funkcionālo testēšanu pēc “melnās kastes” principa: katram testa gadījumam tika sagatavoti ievaddati, sistēmā izpildīta darbība un faktiskais rezultāts salīdzināts ar sagaidāmo. Testa gadījumi sagrupēti pa funkcionālajām prasībām (skat. 2.2. apakšnodaļu), un katrā grupā pārbaudīta gan veiksmīga darbība, gan nederīgu datu un neatļautu darbību apstrāde. Testēšanā tika izmantota datubāze ar demonstrācijas datiem: lietotāji “gustavs” (administrators), “anna_b”, “janis_k”, “liga_o”, “martins_l”, “eliza_k” (profils redzams tikai draugiem) un bloķētais lietotājs “roberts_z”, kā arī privātā grupa “Programmēšana 9. grupa” un publiskā grupa “Skolas sports”.

Testa gadījumi, kuros sistēmai jāatsakās izpildīt darbību, tika izpildīti arī, apejot lietotāja saskarni, piemēram, mainot adresi vai nosūtot formu bez drošības žetona. Tā tika pārbaudīts, ka piekļuves tiesības tiek kontrolētas servera pusē, nevis tikai paslēpjot pogas. Testos, kas maina datus, datubāzē papildus tika pārbaudīts, vai dati ir saglabāti vai, gluži otrādi, nav mainīti. Visu testa gadījumu faktiskais rezultāts sakrita ar sagaidāmo, tāpēc to statuss tabulās ir “pareizi”.

7.1. Funkcionālā testēšana

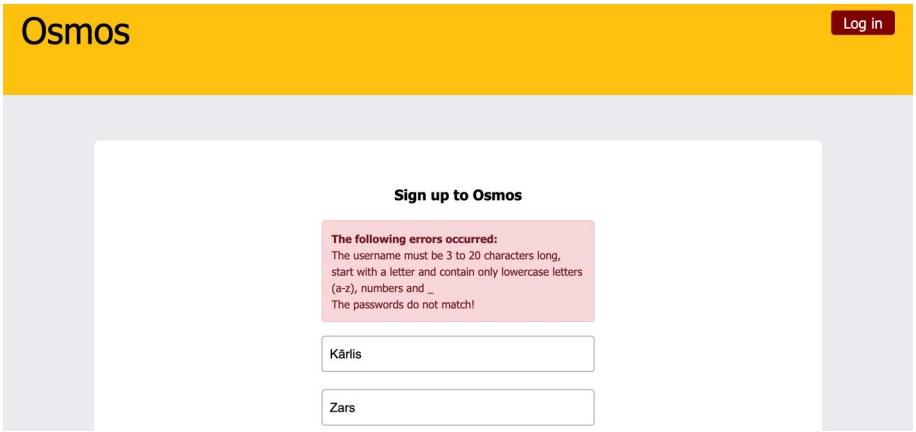
7.1. tabula

Reģistrācijas funkcionalitātes testēšana

Lietotāja loma:	Apmeklētājs	
Prasības identifikators:	FP-01	
Prasības apraksts		
Jānodrošina, ka apmeklētājs var izveidot kontu tikai ar derīgiem datiem un unikālu lietotājvārdu, bet parole tiek saglabāta droši.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Apmeklētājs aizpilda visus laukus korekti: vārds “Kārlis”, uzvārds “Zars”, lietotājvārds “karlis_z”, dzimums “Male”, parole “parole123” divas reizes.	Konts tiek izveidots, parole datubāzē saglabāta kā jaucējvērtība. Tiek atvērta pieslēgšanās lapa ar paziņojumu “Your account was created. You can log in now.”	pareizi
2. Apmeklētājs ievada lietotājvārdu “1karlis”, kas sākas ar ciparu.	Sistēma attēlo kļūdas paziņojumu: “The username must be 3 to 20 characters long, start with a letter and contain only lowercase letters (a-z), numbers and _”.	pareizi
3. Apmeklētājs ievada lietotājvārdu “anna_b”, kas jau pieder citam lietotājam.	Sistēma attēlo kļūdas paziņojumu: “That username is already taken!”.	pareizi

4. Apmeklētājs ievada paroli “abc12”, kas ir īsāka par 6 rakstzīmēm.	Sistēma attēlo kļūdas paziņojumu: “The password must be at least 6 characters long!”.	pareizi
5. Apmeklētājs paroles atkārtojumā ievada “parole124”, kas nesakrīt ar paroli.	Sistēma attēlo kļūdas paziņojumu: “The passwords do not match!”.	pareizi
6. Apmeklētājs atstāj tukšu vārda lauku.	Sistēma attēlo kļūdas paziņojumu: “First name is empty!”. Konts netiek izveidots.	pareizi
7. Apmeklētājs ievada lietotājvārdu “Karlis_Z2” ar lielajiem burtiem.	Lietotājvārds tiek pārveidots mazajos burtos, un konts tiek izveidots ar lietotājvārdu “karlis_z2”.	pareizi

Ja apmeklētājs vienlaikus ievada vairākus nederīgus datus, sistēma parāda visus kļūdu paziņojumus kopā, bet jau ievadītais vārds, uzvārds un lietotājvārds formā saglabājas (skat. 7.1. att.).



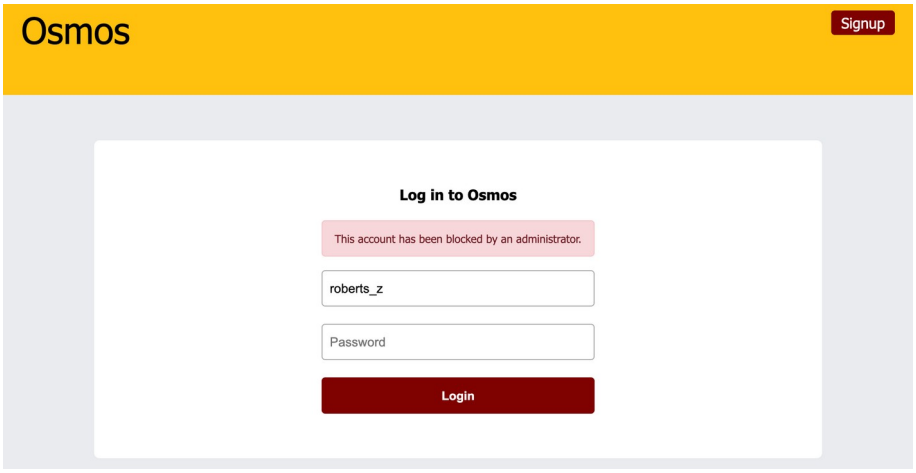
7.1. att. Reģistrācijas kļūdu paziņojumi

7.2. tabula

Pieslēgšanās funkcionalitātes testēšana

Lietotāja loma:	Apmeklētājs, reģistrēts lietotājs	
Prasības identifikators:	FP-02	
Prasības apraksts		
Jānodrošina, ka sistēmai var pieslēgties tikai ar pareizu lietotājvārdu un paroli, bloķēts lietotājs nevar pieslēgties, bet nepieslēdzies apmeklētājs nevar atvērt sistēmas lapas.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Apmeklētājs ievada lietotājvārdu “gustavs” un pareizu paroli.	Tiek atvērta lietotāja profila lapa.	pareizi
2. Apmeklētājs ievada lietotājvārdu “gustavs” un nepareizu paroli.	Sistēma attēlo kļūdas paziņojumu: “Wrong username or password!”.	pareizi
3. Apmeklētājs atstāj abus laukus tukšus.	Sistēma attēlo kļūdas paziņojumu: “Please enter your username and password!”.	pareizi
4. Apmeklētājs ievada bloķētā lietotāja “roberts_z” lietotājvārdu un pareizu paroli.	Sistēma attēlo kļūdas paziņojumu: “This account has been blocked by an administrator.” (skat. 7.2. att.).	pareizi
5. Apmeklētājs lietotājvārda laukā ievada `' OR '1'='1`.	Sistēma attēlo kļūdas paziņojumu: “Wrong username or password!”. Pieslēgšanās nenotiek.	pareizi

6. Nepieslēdzies apmeklētājs atver adresi `home`.	Tiek atvērta pieslēgšanās lapa.	pareizi
7. Lietotājs nospiež pogu “Logout” un pēc tam atver adresi `home`.	Tiek atvērta pieslēgšanās lapa.	pareizi



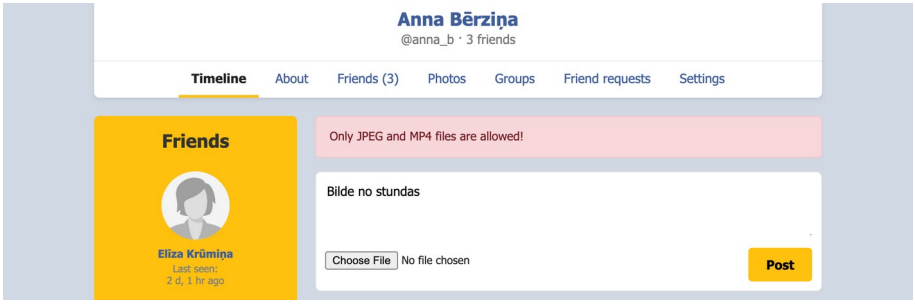
7.2. att. Bloķēta lietotāja pieslēgšanās mēģinājums

7.3. tabula

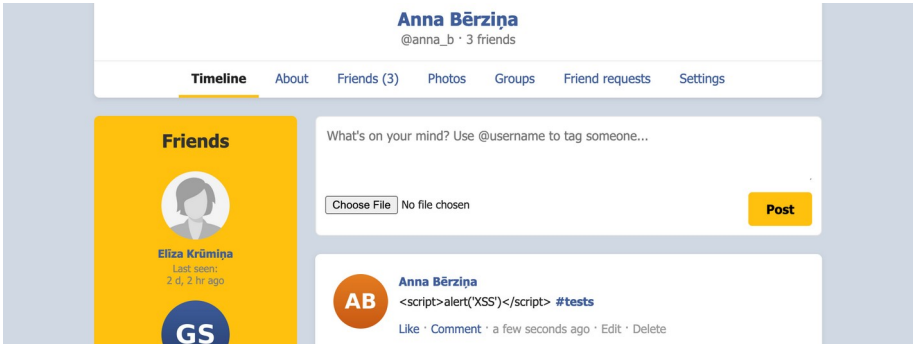
Ierakstu, komentāru un “patīk” atzīmju funkcionalitātes testēšana

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	FP-05, FP-07, FP-08	
Prasības apraksts		
Jānodrošina, ka lietotājs var publicēt ierakstus ar pieļaujamiem failiem, komentēt un atzīmēt tos ar “patīk”, bet ievadītais teksts tiek attēlots droši un atzīmes un tēmturi tiek atpazīti pareizi.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Lietotāja “anna_b” publicē ierakstu “Atkārtojam #mysql kopā ar @janis_k”.	Ieraksts parādās ziņu lentā. “#mysql” ir saite uz tēmtura lapu, bet “@janis_k” saite uz lietotāja profilu. Lietotājs “janis_k” saņem paziņojumu par atzīmi.	pareizi
2. Lietotāja publicē ierakstu, kas satur tikai atstarpes, bez faila.	Sistēma attēlo kļūdas paziņojumu: “Please type something to post!”.	pareizi
3. Lietotāja ierakstam pievieno PNG attēlu.	Sistēma attēlo kļūdas paziņojumu: “Only JPEG and MP4 files are allowed!” (skat. 7.3. att.).	pareizi
4. Lietotāja ierakstam pievieno teksta failu ar PHP kodu, kura nosaukums ir “bilde.jpg”.	Sistēma pēc faila satura nosaka, ka tas nav attēls, un attēlo kļūdas paziņojumu: “Only JPEG and MP4 files are allowed!”. Ieraksts netiek saglabāts.	pareizi
5. Lietotāja publicē ierakstu ar tekstu `<script>alert('XSS')</script>`.	Teksts tiek attēlots kā parasts teksts, un skripts netiek izpildīts (skat. 7.4. att.).	pareizi
6. Lietotājs “janis_k” komentē lietotājas “anna_b” ierakstu.	Komentārs parādās zem ieraksta, ieraksta komentāru skaits palielinās no 0 uz 1, un ieraksta autore saņem paziņojumu.	pareizi
7. Lietotājs “janis_k” atver lietotājas “anna_b” ieraksta labošanas adresi `edit/<ieraksta identifikators>`.	Sistēma attēlo kļūdas paziņojumu: “Access denied! You cannot edit this post.”.	pareizi

8. Lietotājs “janis_k” divas reizes nospiež saiti “Like” pie lietotājas “anna_b” ieraksta.	Pēc pirmā klikšķa bez lapas pārlādes atzīmju skaits ir 1 un parādās teksts “You liked this post”, pēc otrā klikšķa atzīme tiek noņemta. Ieraksta autore saņem tikai vienu paziņojumu.	pareizi
9. Lietotājs publicē ierakstu “Rakstiet uz janis@gmail.com, runāsim par C# un it's #php”.	Saite tiek izveidota tikai tēmturim “#php”. E-pasta adrese, “C#” un apostrofs vārdā “it's” netiek pārveidoti par saitēm.	pareizi



7.3. att. PNG faila pievienošanas mēģinājums ierakstam



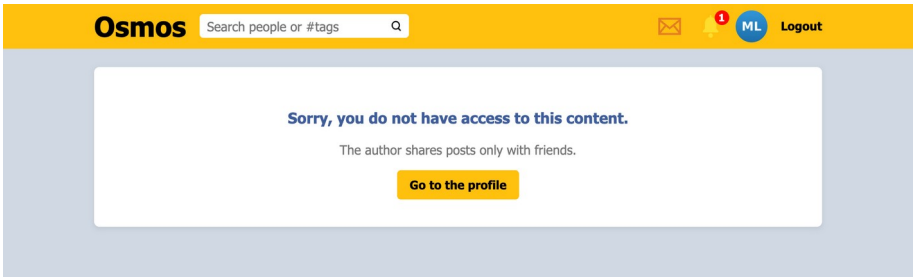
7.4. att. Ieraksts ar HTML skriptu tiek attēlots kā teksts

7.4. tabula

Draudzības un privātuma funkcionalitātes testēšana

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	FP-09, FP-10	
Prasības apraksts		
Jānodrošina, ka lietotāji var kļūt par draugiem, bet profila saturu, kas pieejams tikai draugiem, citi lietotāji nevar redzēt nevienā sistēmas daļā.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Lietotājs “martins_l” atver lietotājas “eliza_k” profilu. Elīzas profils redzams tikai draugiem, un viņi nav draugi.	Tiek attēlots paziņojums “This profile is visible only to friends”, ieraksti netiek rādīti.	pareizi
2. Lietotājs “martins_l” atver Elīzas ieraksta tiešo saiti.	Tiek attēlots paziņojums “Sorry, you do not have access to this content.” (skat. 7.5. att.).	pareizi
3. Lietotājs “martins_l” atver tēmtura “#regex” lapu.	Tiek parādīts lietotājas “anna_b” ieraksts ar šo tēmturi, bet Elīzas ieraksts ar to pašu tēmturi netiek parādīts.	pareizi

4. Lietotājs “martins_l” Elīzas profilā nospiež pogu “Add friend”.	Tiek attēlots paziņojums “Friend request sent.”, pieprasījuma statuss ir “pending”, un Elīza saņem paziņojumu.	pareizi
5. Lietotāja “eliza_k” apstiprina pieprasījumu.	Tiek attēlots paziņojums “You are now friends!”, un Mārtiņš Elīzas profilā redz viņas ierakstus.	pareizi
6. Lietotājs mēģina nosūtīt draudzības pieprasījumu pats sev, mainot adresi.	Pieprasījums netiek izveidots.	pareizi
7. Lietotājs atver draudzības pieprasījuma saiti bez drošības žetona.	Sistēma attēlo kļūdas paziņojumu: “That link has expired. Please try again.”. Pieprasījums netiek izveidots.	pareizi

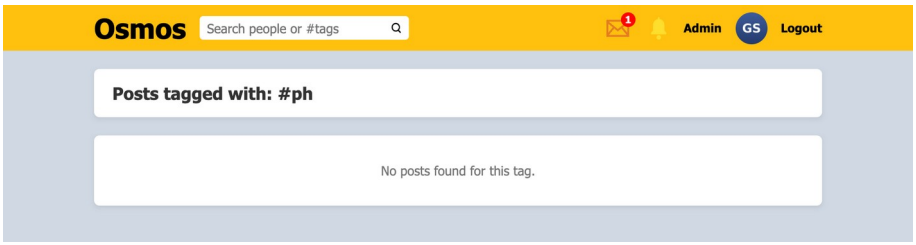


7.5. att. Tikai draugiem redzama ieraksta atvēršanas mēģinājums

7.5. tabula

Meklēšanas un tēmturu funkcionalitātes testēšana

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	FP-11	
Prasības apraksts		
Jānodrošina, ka lietotājs var atrast lietotājus, grupas un ierakstus, bet tēmturi tiek meklēti kā atsevišķi vārdi.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Lietotājs meklēšanas laukā ievada “#php”.	Tiek atvērta lapa “Posts tagged with: #php” ar ierakstiem, kuros ir šis tēmturis.	pareizi
2. Lietotājs meklēšanas laukā ievada “@anna_b”.	Rezultātos ir lietotāja “Anna Bērziņa”.	pareizi
3. Lietotājs atver tēmtura “#ph” lapu.	Tiek attēlots paziņojums “No posts found for this tag.”, jo ieraksti ar tēmturi “#php” neatbilst (skat. 7.6. att.).	pareizi
4. Lietotājs meklēšanas laukā ievada bloķētā lietotāja vārdu “Roberts”.	Tiek attēlots paziņojums “No people were found”.	pareizi
5. Lietotājs meklēšanas laukā ievada “an”.	Rezultātos ir lietotāji “Anna Bērziņa” un “Jānis Kalniņš” un grupa “Programmēšana 9. grupa”.	pareizi



7.6. att. Tēmtura “#ph” meklēšanas rezultāts

7.6. tabula

Grupu funkcionalitātes testēšana

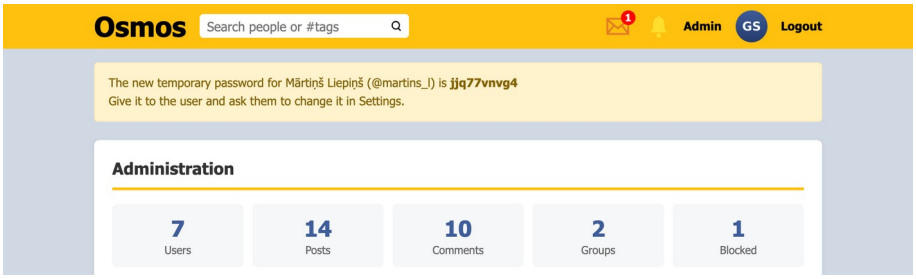
Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	FP-14	
Prasības apraksts		
Jānodrošina, ka privātas grupas saturu redz tikai dalībnieki, bet dalībnieku pārvaldības darbības var veikt tikai lietotāji ar atbilstošu lomu.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Lietotāja “eliza_k”, kas nav dalībniece, atver privāto grupu “Programmēšana 9. grupa”.	Tiek attēlots virsraksts “Private Group” un poga “Request to Join”. Grupas ieraksti netiek rādīti.	pareizi
2. Lietotāja “eliza_k” nosūta ierakstu privātajai grupai, apejot lietotāja saskarni.	Sistēma attēlo kļūdas paziņojumu: “Only members can post in this group.”. Ieraksts netiek saglabāts.	pareizi
3. Grupas īpašnieks “gustavs” apstiprina lietotājas “liga_o” pieprasījumu pievienoties grupai.	Tiek attēlots paziņojums “The request was accepted.”, un Līga redz grupas ierakstus.	pareizi
4. Grupas īpašnieks mēģina noņemt no grupas pats sevi.	Sistēma attēlo kļūdas paziņojumu: “The group owner can't be changed or removed.”.	pareizi
5. Grupas dalībnieks “janis_k”, kuram nav administratora lomas, atver grupas iestatījumu adresi.	Tiek attēlots paziņojums “You don't have access to see this content!”.	pareizi
6. Lietotājs “janis_k” pievienojas publiskajai grupai “Skolas sports”.	Tiek attēlots paziņojums “Welcome! You are now a member of this group.”.	pareizi

7.7. tabula

Administrēšanas funkcionalitātes testēšana

Lietotāja loma:	Administrators, reģistrēts lietotājs	
Prasības identifikators:	FP-15	
Prasības apraksts		
Jānodrošina, ka administratora paneli var lietot tikai administratori un ka administratora darbības tiek izpildītas, ievērojot noteiktos ierobežojumus.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Lietotāja “anna_b”, kurai nav administratora tiesību, atver adresi `admin`.	Sistēma attēlo kļūdas paziņojumu: “You don't have access to that page.”.	pareizi
2. Administrators bloķē lietotāju “janis_k”, kurš tobrīd ir pieslēdzies sistēmai.	Tiek attēlots paziņojums “Jānis Kalniņš was blocked and can no longer log in.”. Jāņa nākamajā lapas ielādē tiek atvērta pieslēgšanās lapa.	pareizi
3. Administrators piešķir lietotājai “anna_b” administratora tiesības un pēc tam mēģina viņu bloķēt.	Sistēma attēlo kļūdas paziņojumu: “Remove the admin rights first, then block the user.”.	pareizi
4. Administrators atiestata lietotāja “martins_l” paroli.	Tiek parādīta 10 rakstzīmju pagaidu parole (skat. 7.7. att.). Ar to var pieslēgties, bet ar veco paroli pieslēgties vairs nevar.	pareizi

5. Administrators mēģina bloķēt pats sevi, nosūtot formu ar savu identifikatoru.	Sistēma attēlo kļūdas paziņojumu: “You can't change your own account here.”.	pareizi
6. Administrators dzēš lietotājas “liga_o” ierakstu.	Tiek attēlots paziņojums “The post was deleted.”. Kopā ar ierakstu tiek dzēsti arī tā komentāri.	pareizi



7.7. att. Lietotāja paroles atiestatīšana

7.8. tabula

Drošības prasību testēšana

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	Drošības prasības (2.3.5. apakšnodaļa)	
Prasības apraksts		
Jānodrošina, ka sistēma atsakās izpildīt pieprasījumus bez drošības žetona, neatver veidņu failus un aizsargā sesiju.		
Ievaddati / situācijas apraksts	**Sagaidāmais rezultāts**	**Statuss**
1. Lietotājs nosūta ieraksta publicēšanas formu bez CSRF žetona.	Sistēma attēlo kļūdas paziņojumu: “Your session expired. Please try again.”. Ieraksts netiek saglabāts.	pareizi
2. Lietotājs atver “patīk” atzīmes saiti bez drošības žetona.	Sistēma attēlo kļūdas paziņojumu: “That link has expired. Please try again.”.	pareizi
3. Lietotājs atver adresi `post`, kas ir veidnes fails un nav atļauto lapu sarakstā.	Tiek atvērta lapa “Page not found”, servera atbildes statuss ir 404.	pareizi
4. Tiek pārbaudīta servera nosūtītā sesijas sīkdatne.	Sīkdatnei ir atribūti HttpOnly un SameSite=Lax.	pareizi
5. Lietotājs pieslēdzas sistēmai.	Pēc pieslēgšanās sesijas identifikators tiek nomainīts.	pareizi

7.2. Veiktspējas testēšana

Nefunkcionālā prasība par ziņu lentas ielādes laiku (skat. 2.3.4. apakšnodaļu) tika pārbaudīta, datubāzei pievienojot 1000 testa ierakstus no sešiem lietotājiem ar dažādiem tēmturiem, tāpēc kopējais ierakstu skaits bija 1023. Lietotāja “gustavs” ziņu lenta pārlūkprogrammā tika ielādēta 10 reizes pēc kārtas, un katru reizi ar pārlūkprogrammas Navigation Timing datiem tika noteikts laiks no pieprasījuma sākuma līdz lapas pilnai ielādei, ieskaitot attēlus. Vidējais ielādes laiks bija 40 milisekundes, bet ilgākais 79 milisekundes. Servera atbildes laiks bez attēlu ielādes 20 pieprasījumos bija vidēji 8 milisekundes. Tēmtura

“#php” lapai, kurā ierakstu atlase notiek ar operatoru REGEXP visā tabulā, servera atbildes laiks 20 pieprasījumos bija vidēji 6 milisekundes. Testēšana tika veikta lokālajā izstrādes vidē XAMPP 8.2.4 (Apache 2.4.56, PHP 8.2.4, MariaDB 10.4.28) macOS datorā. Iegūtie rezultāti ir daudzkārt mazāki par prasībā noteiktajām 2 sekundēm, tāpēc prasība ir izpildīta.

SECINĀJUMI

Kvalifikācijas darba rezultātā tika izstrādāta sociālā tīkla tīmekļa lietotne “Osmos”, kas paredzēta vienas klases, kursa vai skolas audzēkņu savstarpējai saziņai. Sistēmā ir realizētas visas uzdevumā noteiktās funkcijas: lietotāju profili, draudzības pieprasījumi, ziņu lenta ar draugu ierakstiem, komentāri, “patīk” atzīmes un privātuma iestatījums, ar kuru lietotājs izvēlas, vai viņa saturu redz visi tīkla lietotāji vai tikai draugi. Lietotājvārdu pārbaude, lietotāju atzīmes un tēmturu atpazīšana un meklēšana ir realizēta ar regulārajām izteiksmēm. Papildus uzdevumā prasītajam sistēmā tika izveidotas grupas ar dalībnieku lomām, privātās ziņas, paziņojumi par citu lietotāju darbībām un administratora panelis, kas ļauj uzturēt kārtību tīklā bez tiešas iejaukšanās datubāzē.

Darba gaitā tika izpildīti visi ievadā izvirzītie uzdevumi. Izpētot līdzīgu sociālo tīklu funkcionalitāti, tika noteiktas 15 funkcionālās prasības un nefunkcionālās prasības izstrādei, saskarnei, programmatūrai, aparatūrai un drošībai. Izstrādei tika izvēlēta XAMPP vide ar PHP valodu un MariaDB datubāzi, jo šie rīki ir bezmaksas, darbojas dažādās operētājsistēmās un ir plaši dokumentēti. Tika izstrādāta sistēmas arhitektūra ar vienotu ieejas punktu un atļauto lapu sarakstu, kā arī datubāze ar 11 tabulām. Sistēmas darbība tika pārbaudīta ar 52 testa gadījumiem, un visos gadījumos faktiskais rezultāts sakrita ar sagaidāmo. Veiktspējas pārbaudē ziņu lenta ar vairāk nekā 1000 ierakstiem datubāzē ielādējās mazāk nekā 0,1 sekundē.

Izstrādes laikā tika gūtas vairākas atziņas. Pirmkārt, privātuma iestatījums ir jāpārbauda katrā vietā, kur tiek attēlots saturs, nevis tikai profila lapā. Ieraksts, kas paslēpts profilā, citādi joprojām būtu redzams tēmtura meklēšanā, populārāko tēmturu blokā vai “patīk” atzīmju sarakstā. Tāpēc visa redzamības loģika tika apvienota vienā metodē, ko izmanto visas sistēmas daļas. Otrkārt, regulārās izteiksmes jāpārbauda ne tikai ar pareiziem piemēriem, bet arī ar tekstu, kurā līdzīgas rakstzīmju virknes parādās citā nozīmē, piemēram, e-pasta adresēm, programmēšanas valodas nosaukumam “C#” un HTML vienībām. Treškārt, lietotāju un grupu glabāšana vienā tabulā ļāva grupām izmantot tās pašas funkcijas kā profiliem, taču prasīja papildu pārbaudes, lai grupa nevarētu pieslēgties sistēmai, saņemt draudzības pieprasījumus vai privātās ziņas.

Īpaša uzmanība tika pievērsta drošībai, jo sociālajā tīklā gandrīz visu saturu ievada paši lietotāji. Paroles tiek glabātas tikai kā jaucējvērtības, lietotāju ievadītais teksts pirms attēlošanas tiek aizsargāts pret HTML koda izpildi, augšupielādēto failu tips tiek noteikts pēc to satura, un katra datus mainoša darbība tiek aizsargāta ar sesijai piesaistītu žetonu. Piekļuves tiesības tiek pārbaudītas servera pusē, tāpēc tās nevar apiet, mainot adresi vai nosūtot formu ārpus lietotāja

saskarnes. Ievērojot datu minimizēšanas principu, sistēmā netiek glabāta lietotāju e-pasta adrese, jo tā sistēmas darbībai nav nepieciešama.

Pašreizējai sistēmas versijai ir arī ierobežojumi, kurus būtu lietderīgi novērst turpmākajā attīstībā.

- SQL vaicājumos lietotāju ievadītās vērtības tiek aizsargātas ar funkciju `mysqli_real_escape_string`. Drošāks un pārskatāmāks risinājums būtu sagatavotie vaicājumi (prepared statements), kuros dati tiek nodoti atsevišķi no vaicājuma teksta.
- Saites starp tabulām tiek uzturētas programmas kodā. Ārējo atslēgu ierobežojumi ar automātisku saistīto ierakstu dzēšanu datu integritāti nodrošinātu arī datubāzes līmenī.
- "Patīk" atzīmes tiek glabātas JSON masīvā vienā rindā katram ierakstam. Šāds risinājums ir vienkāršs, taču neļauj efektīvi atlasīt, piemēram, visus ierakstus, kurus konkrēts lietotājs ir atzīmējis. Labāks risinājums būtu atsevišķa tabula, kurā katra atzīme ir viena rinda.
- Lietotāja saskarne paredzēta datora ekrānam. Lai sistēmu ērti varētu lietot viedtālrunī, tā būtu jāpielāgo mazākiem ekrāniem.
- Jaunas privātās ziņas un paziņojumi kļūst redzami tikai pēc lapas pārlādes. Sarunas būtu ērtākas, ja jaunās ziņas tiktu ielādētas automātiski.
- Aizmirstu paroli var atiestatīt tikai administrators. Ja sistēmu izmantotu lielāks lietotāju skaits, būtu jāapsver paroles atjaunošana bez administratora līdzdalības.
- Lietotāja saskarne ir angļu valodā. Sistēmu varētu papildināt ar latviešu valodas versiju un iespēju valodu izvēlēties iestatījumos.

Izstrādātā sistēma ir pilnībā darbspējīga, un to var uzstādīt jebkurā datorā ar XAMPP vidi un lietot vienas klases vai skolas ietvaros. Darba izstrāde ļāva praktiski apgūt pilnu tīmekļa lietotnes izstrādes ciklu no prasību noteikšanas un datubāzes projektēšanas līdz programmēšanai, testēšanai un dokumentēšanai.

INFORMĀCIJAS AVOTI

1. Eiropas Parlamenta un Padomes Regula (ES) 2016/679 (2016. gada 27. aprīlis) par fizisku personu aizsardzību attiecībā uz personas datu apstrādi un šādu datu brīvu apriti un ar ko atceļ Direktīvu 95/46/EK (Vispārīgā datu aizsardzības regula) / internets. — https://eur-lex.europa.eu/legal-content/LV/TXT/?uri=CELEX:32016R0679 — 15.10.2026.
2. Kultūras informācijas sistēmu centrs: Latvijas Nacionālais terminoloģijas portāls / internets. — https://termini.gov.lv — 15.10.2026.
3. Apache Friends: XAMPP Apache + MariaDB + PHP + Perl / internets. — https://www.apachefriends.org — 15.10.2026.
4. Apache Software Foundation: Apache Module mod_rewrite / internets. — https://httpd.apache.org/docs/2.4/mod/mod_rewrite.html — 15.10.2026.
5. MariaDB: MariaDB Server Documentation / internets. — https://mariadb.com/docs/server — 15.10.2026.
6. MariaDB: REGEXP / internets. — https://mariadb.com/docs/server/reference/sql-functions/string-functions/regular-expressions-functions/regexp — 15.10.2026.
7. Mozilla: MDN Web Docs. XMLHttpRequest / internets. — https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest — 15.10.2026.
8. OWASP Foundation: Cross-Site Request Forgery Prevention Cheat Sheet / internets. — https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html — 15.10.2026.
9. OWASP Foundation: OWASP Top 10 / internets. — https://owasp.org/www-project-top-ten/ — 15.10.2026.
10. OWASP Foundation: Password Storage Cheat Sheet / internets. — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html — 15.10.2026.
11. The PHP Documentation Group: PHP Manual / internets. — https://www.php.net/manual/en/ — 15.10.2026.
12. The PHP Documentation Group: PHP Manual. Image Processing and GD / internets. — https://www.php.net/manual/en/book.image.php — 15.10.2026.
13. The PHP Documentation Group: PHP Manual. password_hash / internets. — https://www.php.net/manual/en/function.password-hash.php — 15.10.2026.
14. The PHP Documentation Group: PHP Manual. preg_match / internets. — https://www.php.net/manual/en/function.preg-match.php — 15.10.2026.

PIELIKUMI

1. pielikums. Programmas pirmkods

Pielikumā ir sistēmas galveno failu pirmkods: vienotais ieejas punkts, sesijas un drošības iestatījumi, datubāzes klase, palīgfunkcijas ar regulārajām izteiksmēm, pieslēgšanās, reģistrācijas, lietotāju un ierakstu klases, kā arī AJAX pieprasījumu apstrāde. Kopā 1414 rindas.

Fails index.php

```
<?php
// .htaccess sends every request here

include_once("classes/autoload.php");

// allowed pages, anything else is a 404
$routes = [
    "home",
    "profile",
    "login",
    "logout",
    "signup",
    "search",
    "single_post",
    "image_view",
    "like",
    "friend",
    "admin",
    "likes",
    "edit",
    "delete",
    "messages",
    "notifications",
    "group",
    "create_group",
    "join",
    "invite",
    "group_request_accept",
    "change_profile_image",
    "tags",
    "ajax",
];

$page = $URL[0] ?? "home";

if (in_array($page, $routes, true) && file_exists($page . ".php")) {
    require($page . ".php");
} else {
    require("404.php");
}
```

Fails classes/autoload.php

```
<?php


date_default_timezone_set('Europe/Riga');

if (session_status() === PHP_SESSION_NONE) {

    session_set_cookie_params([
        'lifetime' => 0,
        'path' => '/',
        'httponly' => true,
        'samesite' => 'Lax'
    ]);
    session_start();
}

include_once("classes/connect.php");
include_once("classes/login.php");
include_once("classes/user.php");
include_once("classes/post.php");
include_once("classes/image.php");
include_once("classes/profile.php");
include_once("classes/settings.php");
```

```
include_once("classes/time.php");
include_once("classes/functions.php");
include_once("classes/signup.php");
include_once("classes/messages.php");
include_once("classes/group.php");

if (!defined("ROOT")) {
    $scheme = $_SERVER['REQUEST_SCHEME'] ?? ((!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !
    == 'off') ? 'https' : 'http');
    $root = $scheme . "://" . $_SERVER['SERVER_NAME'] . $_SERVER['PHP_SELF'];


    $root = trim(str_replace(basename($_SERVER['PHP_SELF']), "", $root), "/");
    define("ROOT", $root . "/");
}

// $URL isn't set when a page is opened directly
if (!isset($URL)) {
    $URL = split_url();
}

// csrf check for every POST
if ($_SERVER['REQUEST_METHOD'] == "POST") {

    $is_ajax = (($_SERVER['HTTP_X_REQUESTED_WITH'] ?? "") == "XMLHttpRequest");

    // over post_max_size php just gives us an empty $_POST and $_FILES
    if (empty($_POST) && empty($_FILES) && !empty($_SERVER['CONTENT_LENGTH'])) {
        if ($is_ajax) {
            echo json_encode(["action" => "error", "message" => "That upload is too large."]);
            die;
        }
        set_flash("That upload is too large. Please choose a smaller file.", "error");
        redirect_back();
    }

    if (!csrf_check($_POST['csrf_token'] ?? "")) {
        if ($is_ajax) {
            echo json_encode(["action" => "error", "message" => "Your session expired. Please
    reload the page."]);
            die;
        }
        set_flash("Your session expired. Please try again.", "error");
        redirect_back();
    }
}
```

Fails classes/connect.php

```
<?php

class Database
{
    private $host = "localhost";
    private $username = "root";
    private $password = "";
    private $db = "osmos_db";

    // one connection per page load
    private static $connection = null;

    function connect()
    {
        if (self::$connection === null) {
            mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

            try {
                $connection = mysqli_connect($this->host, $this->username, $this->password,
    $this->db);
            } catch (mysqli_sql_exception $e) {
                error_log("Osmos DB connection failed: " . $e->getMessage());
                die("Could not connect to the database.");
            }

            mysqli_set_charset($connection, "utf8mb4");

            // keep mysql on the same timezone as php
            mysqli_query($connection, "SET time_zone = '" . date('P') . "'");

            self::$connection = $connection;
        }
```

```
        return self::$connection;
    }

    // false if the query fails or finds nothing
    function read($query)
    {
        $conn = $this->connect();

        try {
            $result = mysqli_query($conn, $query);
        } catch (mysqli_sql_exception $e) {
            error_log("Osmos DB read error: " . $e->getMessage() . " | Query: " . $query);
            return false;
        }

        if (!$result) {
            return false;
        }

        $data = [];
        while ($row = mysqli_fetch_assoc($result)) {
            $data[] = $row;
        }

        return count($data) > 0 ? $data : false;
    }

    function save($query)
    {
        $conn = $this->connect();

        try {
            $result = mysqli_query($conn, $query);
        } catch (mysqli_sql_exception $e) {
            error_log("Osmos DB save error: " . $e->getMessage() . " | Query: " . $query);
            return false;
        }

        if (!$result) {
            return false;
        }

        return true;
    }

    // only safe inside quotes in a query
    function escape($value)
    {
        if (is_array($value) || is_object($value)) {
            return "";
        }
        return mysqli_real_escape_string($this->connect(), (string)$value);
    }
}
```

Fails classes/functions.php (1. līdz 280. rinda)

```
<?php

// routing, redirects, flash messages

function split_url()
{
    $url = (isset($_GET['url']) && is_string($_GET['url']) && $_GET['url'] != "") ?
    $_GET['url'] : "home";
    $url = explode("/", filter_var(trim($url, "/"), FILTER_SANITIZE_URL));

    return $url;
}

function split_url_from_string($str)
{
    $url = explode("/", filter_var(trim($str, "/"), FILTER_SANITIZE_URL));
    return $url;
}

// only redirect to our own pages
function safe_return_url($url, $fallback = "")
{
```

```
    if ($fallback == "") {
        $fallback = ROOT . "home";
    }

    if (is_string($url) && $url !== "" && strpos($url, ROOT) === 0) {
        return $url;
    }

    return $fallback;
}

function return_url($fallback = "")
{
    return safe_return_url($_SERVER['HTTP_REFERER'] ?? "", $fallback);
}

function redirect($url)
{
    header("Location: " . $url);
    die;
}

function redirect_back($fallback = "")
{
    redirect(return_url($fallback));
}

// one-time message shown on the next page
function set_flash($message, $type = "success")
{
    $_SESSION['flash'] = ["message" => $message, "type" => $type];
}

function render_flash()
{
    if (empty($_SESSION['flash']) || !is_array($_SESSION['flash'])) {
        return;
    }

    $flash = $_SESSION['flash'];
    unset($_SESSION['flash']);

    $type = in_array($flash['type'], ["success", "error", "warning", "info"]) ? $flash['type']
    : "info";
    // not escaped on purpose, flash messages are ours and some have <br>
    echo "<div class='alert alert-" . $type . "'>" . $flash['message'] . "</div>";
}



function csrf_token()
{
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function csrf_field()
{
    return '<input type="hidden" name="csrf_token" value="' . csrf_token() . '">';
}

function csrf_url($url)
{
    return $url . (strpos($url, "?") === false ? "?" : "&") . "token=" . csrf_token();
}

function csrf_check($token)
{
    return is_string($token)
        && $token !== ""
        && !empty($_SESSION['csrf_token'])
        && hash_equals($_SESSION['csrf_token'], $token);
}

// helpers

function pagination_link()
{
```

```
    $page_number = isset($_GET['page']) ? (int)$_GET['page'] : 1;
    if ($page_number < 1) $page_number = 1;

    $next_page = $page_number + 1;
    $prev_page = $page_number - 1;
    if ($prev_page < 1) $prev_page = 1;

    $path = (isset($_GET['url']) && is_string($_GET['url']) && $_GET['url'] != "") ?
    $_GET['url'] : "home";
    $url = ROOT . htmlspecialchars($path, ENT_QUOTES);

    // keep other params like ?find=
    $params = $_GET;
    unset($params['url'], $params['page']);

    $query = http_build_query($params);
    $url .= "?" . ($query != "" ? htmlspecialchars($query, ENT_QUOTES) . "&amp;" : "") .
    "page=";

    return [
        'next' => $url . $next_page,
        'prev' => $url . $prev_page
    ];
}

function i_own_content($row)
{
    $myid = $_SESSION['osmos_userid'] ?? 0;
    if (!$myid || !is_array($row)) {
        return false;
    }
    if (isset($row['gender']) && ($row['type'] ?? '') == 'profile' && $myid == $row['userid'])
    {
        return true;
    }
    if (isset($row['gender']) && ($row['type'] ?? '') == 'group' && $myid == $row['owner']) {
        return true;
    }
    if (isset($row['post_id']) && $myid == $row['user_id']) {
        return true;
    }
    return false;
}

function esc($value)
{
    $DB = new Database();
    return $DB->escape($value);
}

// random 12-17 digit id that isn't taken yet
// 19 digits can overflow BIGINT
function generate_unique_id($table, $column)
{
    $DB = new Database();
    $id = "";

    for ($attempt = 0; $attempt < 20; $attempt++) {
        $length = random_int(12, 17);
        $id = (string)random_int(1, 9);
        for ($i = 1; $i < $length; $i++) {
            $id .= random_int(0, 9);
        }

        $check = $DB->read("SELECT 1 FROM `$table` WHERE `$column` = '$id' LIMIT 1");
        if (empty($check)) {
            return $id;
        }
    }

    return $id;
}

// ucfirst() that also works with ā, ģ, š
function osmos_ucfirst($text)
{
    $text = (string)$text;
    return mb_strtoupper(mb_substr($text, 0, 1)) . mb_substr($text, 1);
}
```

```
// "Jānis Bērziņš" -> "janisberzins"
function ascii_slug($text)
{
    $ascii = function_exists('iconv') ? @iconv("UTF-8", "ASCII//TRANSLIT//IGNORE", (string)
    $text) : (string)$text;
    if ($ascii === false) {
        $ascii = (string)$text;
    }
    return preg_replace("/[^a-z0-9_]/", "", strtolower($ascii));
}

// 3-20 chars, a-z 0-9 and _, starts with a letter
define("USERNAME_PATTERN", "/^[a-z][a-z0-9_]{2,19}$/");

// @username, but not the @ in an email
define("MENTION_PATTERN", "/(^|[^\w&@])@([a-zA-Z0-9_]+)/");

// #php, #school_2026 but not C# or &#039;
define("HASHTAG_PATTERN", "/(^|[^\w&#])#([a-zA-Z_][a-zA-Z0-9_]*)/");

function extract_hashtags($text)
{
    $matches = [];
    preg_match_all(HASHTAG_PATTERN, (string)$text, $matches);
    return array_values(array_unique(array_map('strtolower', $matches[2])));
}

// ["php" => 5, "mysql" => 3], only counts posts the viewer can see
function popular_hashtags($viewer_id, $limit = 8)
{
    $DB = new Database();
    $posts = $DB->read("SELECT * FROM posts WHERE post LIKE '%#%' ORDER BY id DESC LIMIT
    300");
    if (!is_array($posts)) {
        return [];
    }

    $post_class = new Post();
    $counts = [];
    foreach ($posts as $row) {
        if (!$post_class->can_view_post($row, $viewer_id)) {
            continue;
        }
        foreach (extract_hashtags($row['post']) as $tag) {
            $counts[$tag] = ($counts[$tag] ?? 0) + 1;
        }
    }

    arsort($counts);
    return array_slice($counts, 0, $limit, true);
}

define("PASSWORD_MIN_LENGTH", 6);

// "" if it's fine, otherwise the error
function username_error($username, $exclude_userid = 0)
{
    if ($username === "") {
        return "Username is empty!<br>";
    }

    if (!preg_match(USERNAME_PATTERN, $username)) {
        return "The username must be 3 to 20 characters long, start with a letter and contain
    only lowercase letters (a-z), numbers and _<br>";
    }

    $DB = new Database();
    $safe_name = esc($username);
    $safe_id = esc($exclude_userid);
    $taken = $DB->read("SELECT id FROM users WHERE tag_name = '$safe_name' AND userid !=
    '$safe_id' LIMIT 1");

    if (!empty($taken)) {
        return "That username is already taken!<br>";
    }

    return "";
}

// for "updated his/her/their profile image"
```

```
function pronoun($user)
{
    $gender = is_array($user) ? strtolower((string)($user['gender'] ?? "")) : "";
    if ($gender == "female") return "her";
    if ($gender == "male") return "his";
    return "their";
}

// "You and 2 others liked this post" etc.
function likes_info_text($likes_count, $i_liked)
{
    $likes_count = (int)$likes_count;
    if ($likes_count < 1) {
        return "";
    }

    if ($likes_count == 1) {
        return $i_liked ? "You liked this post" : "1 person liked this post";
    }

    if ($i_liked) {
        $others = $likes_count - 1;
        return "You and " . $others . " other" . ($others == 1 ? "" : "s") . " liked this
    post";
    }

    return $likes_count . " people liked this post";
}
```

Fails classes/login.php

```
<?php

class Login
{
    private $error = "";

    public function evaluate($data)
    {
        $username = isset($data['username']) && is_string($data['username']) ?
    strtolower(trim(ltrim(trim($data['username']), "@"))) : "";
        $password = isset($data['password']) && is_string($data['password']) ?
    $data['password'] : "";

        // groups have an empty password, so never let an empty one through
        if ($username === "" || $password === "") {
            $this->error .= "Please enter your username and password!<br>";
            return $this->error;
        }

        $username = esc($username);

        // profiles only
        $query = "select * from users where tag_name = '$username' and (type is null or type =
    '' or type = 'profile') limit 1";

        $DB = new Database();
        $result = $DB->read($query);

        if ($result)
        {
            $row = $result[0];
            $stored = (string)$row['password'];

            $authenticated = false;
            $needs_upgrade = false;

            if ($stored !== "") {
                $info = password_get_info($stored);

                if (!empty($info['algo'])) {
                    $authenticated = password_verify($password, $stored);
                    $needs_upgrade = $authenticated && password_needs_rehash($stored,
    PASSWORD_DEFAULT);
                } else {
                    // old accounts: sha256, sha3-256 or plain text, rehashed below
                    $authenticated =
                        hash_equals($stored, hash("sha256", $password)) ||
                        hash_equals($stored, hash("sha256", addslashes($password))) ||
                        hash_equals($stored, hash("sha3-256", $password)) ||
```

```
                    hash_equals($stored, $password);
                $needs_upgrade = $authenticated;
            }
        }

        if ($authenticated && !empty($row['blocked']))
        {
            $this->error .= "This account has been blocked by an administrator.<br>";
            return $this->error;
        }

        if ($authenticated)
        {
            // upgrade old hashes
            if ($needs_upgrade) {
                $userid = esc($row['userid']);
                $new_hash = esc(password_hash($password, PASSWORD_DEFAULT));
                $DB->save("UPDATE users SET password = '$new_hash' WHERE userid =
'$userid' LIMIT 1");
            }

            // against session fixation
            session_regenerate_id(true);
            $_SESSION['osmos_userid'] = $row['userid'];
        }
        else
        {
            $this->error .= "Wrong username or password!<br>";
        }
    }
    else
    {
        $this->error .= "Wrong username or password!<br>";
    }

    return $this->error;
}

public function check_login($id, $redirect = true)
{
    if(isset($_SESSION['osmos_userid']))
    {
        $userid = $_SESSION['osmos_userid'];

        if(is_numeric($id) && $id > 0)
        {
            $userid = $id;
        }

        $userid = esc($userid);
        $query = "select * from users where userid = '$userid' limit 1";

        $DB = new Database();
        $result = $DB->read($query);

        // kick out blocked users, groups should never have a session anyway
        if ($result && $result[0]['userid'] == $_SESSION['osmos_userid'] && (($result[0]
['type'] ?? "") == "group" || !empty($result[0]['blocked'])))
        {
            unset($_SESSION['osmos_userid']);
            $result = false;
        }

        if ($result)
        {
            return $result[0];
        }
    }

    if($redirect)
    {
        header("Location: " . ROOT . "login");
        die;
    }
    else
    {
        return false;
    }
}
}
```

Fails classes/signup.php

```
<?php
class Signup
{
    private $error = "";

    function evaluate($data)
    {
        $fields = [
            "first_name" => "First name",
            "last_name" => "Last name",
            "username" => "Username",
            "gender" => "Gender",
            "password" => "Password",
            "confirm_password" => "Confirm password",
        ];

        $clean = [];
        foreach ($fields as $key => $label)
        {
            $value = (isset($data[$key]) && is_string($data[$key])) ? trim($data[$key]) : "";
            // don't trim passwords
            if ($key == "password" || $key == "confirm_password") {
                $value = (isset($data[$key]) && is_string($data[$key])) ? $data[$key] : "";
            }
            $clean[$key] = $value;

            if ($value === "")
            {
                $this->error .= $label . " is empty!<br>";
            }
        }

        $clean['username'] = strtolower($clean['username']);
        if ($clean['username'] !== "") {
            $this->error .= username_error($clean['username']);
        }

        foreach (["first_name" => "First name", "last_name" => "Last name"] as $key => $label)
        {
            if ($clean[$key] === "") continue;

            if (is_numeric($clean[$key])) {
                $this->error .= $label . " can't be a number!<br>";
            }
            if (strstr($clean[$key], " ")) {
                $this->error .= $label . " can't have spaces!<br>";
            }
            if (mb_strlen($clean[$key]) > 50) {
                $this->error .= $label . " is too long!<br>";
            }
        }

        if ($clean['gender'] !== "" && !in_array($clean['gender'], ["Male", "Female"])) {
            $this->error .= "Please select a gender!<br>";
        }

        if ($clean['password'] !== "" && mb_strlen($clean['password']) < PASSWORD_MIN_LENGTH)
    {
            $this->error .= "The password must be at least " . PASSWORD_MIN_LENGTH . "
    characters long!<br>";
        }

        if ($clean['password'] !== "" && $clean['password'] !== $clean['confirm_password']) {
            $this->error .= "The passwords do not match!<br>";
        }

        if($this->error == "")
        {
            $this->create_user($clean);
            return "";
        }
        else
        {
            return $this->error;
        }
    }

    public function create_user($data)
```

```
    {
        $first_name = osmos_ucfirst($data['first_name']);
        $last_name = osmos_ucfirst($data['last_name']);
        $gender = $data['gender'];

        $password = password_hash($data['password'], PASSWORD_DEFAULT);

        // username doubles as the @tag
        $tag_name = $data['username'];

        $userid = $this->create_userid();
        $date = date("Y-m-d H:i:s");

        // no pictures yet
        $profile_image = "";
        $cover_image = "";
        $type = "profile";

        $first_name = esc($first_name);
        $last_name = esc($last_name);
        $gender = esc($gender);
        $password = esc($password);
        $tag_name = esc($tag_name);

        $query = "insert into users
            (userid, first_name, last_name, gender, password, profile_image, cover_image,
    date, tag_name, type)
            values
            ('$userid', '$first_name', '$last_name', '$gender', '$password', '$profile_image',
    '$cover_image', '$date', '$tag_name', '$type')";

        $DB = new Database();
        $DB->save($query);
    }

    private function create_userid()
    {
        return generate_unique_id("users", "userid");
    }
}
```

Fails classes/user.php

```
<?php

class User
{
    public function get_user($id)
    {
        if(!is_numeric($id)) {
            return false;
        }

        $id = esc($id);
        $query = "select * from users where userid = '$id' limit 1";
        $DB = new Database();
        $result = $DB->read($query);

        if(is_array($result) && isset($result[0])) {
            return $result[0];
        }

        return false;
    }

    // friends: one row per pair, status 'pending' or 'accepted'

    private function get_friend_row($user_a, $user_b)
    {
        $DB = new Database();
        $a = esc($user_a);
        $b = esc($user_b);

        $result = $DB->read("select * from friends where (sender = '$a' and receiver = '$b')
    or (sender = '$b' and receiver = '$a') limit 1");
        return is_array($result) ? $result[0] : false;
    }

    // "friends", "sent", "received" or "none"
    public function friendship_status($me, $other)
```

```
{
    if (empty($me) || empty($other) || $me == $other) {
        return "none";
    }

    $row = $this->get_friend_row($me, $other);
    if (!$row) {
        return "none";
    }

    if ($row['status'] == "accepted") {
        return "friends";
    }

    return ($row['sender'] == $me) ? "sent" : "received";
}

public function are_friends($user_a, $user_b)
{
    return $this->friendship_status($user_a, $user_b) == "friends";
}

// no groups, no blocked users, not yourself
private function can_be_friend($me, $other)
{
    if (empty($me) || !is_numeric($other) || $me == $other) {
        return false;
    }

    $row = $this->get_user($other);
    return is_array($row) && ($row['type'] ?? "") != "group" && empty($row['blocked']);
}

public function send_friend_request($me, $other)
{
    if (!$this->can_be_friend($me, $other)) {
        return false;
    }

    $status = $this->friendship_status($me, $other);

    // they already asked me, so just accept
    if ($status == "received") {
        return $this->accept_friend_request($me, $other);
    }

    if ($status != "none") {
        return false;
    }

    $DB = new Database();
    $sender = esc($me);
    $receiver = esc($other);
    $date = date("Y-m-d H:i:s");
    $DB->save("insert into friends (sender, receiver, status, date) values ('$sender',
'$receiver', 'pending', '$date')");

    add_notification($me, "friend_request", ["userid" => $other, "sender" => $me]);
    return true;
}

public function accept_friend_request($me, $other)
{
    if ($this->friendship_status($me, $other) != "received") {
        return false;
    }

    $DB = new Database();
    $sender = esc($other);
    $receiver = esc($me);
    $date = date("Y-m-d H:i:s");
    $DB->save("update friends set status = 'accepted', date = '$date' where sender =
'$sender' and receiver = '$receiver' limit 1");

    add_notification($me, "friend_accept", ["userid" => $other, "sender" => $me]);
    return true;
}

// used for decline, cancel and unfriend
public function remove_friend_row($me, $other)
```

```
{
    if ($this->friendship_status($me, $other) == "none") {
        return false;
    }

    $DB = new Database();
    $a = esc($me);
    $b = esc($other);
    $DB->save("delete from friends where (sender = '$a' and receiver = '$b') or (sender =
'$b' and receiver = '$a')");

    $DB->save("delete from notifications where activity = 'friend_request' and ((user_id =
'$a' and content_owner = '$b') or (user_id = '$b' and content_owner = '$a'))");
    return true;
}

public function get_friend_ids($id)
{
    if (!is_numeric($id)) {
        return [];
    }

    $DB = new Database();
    $id = esc($id);
    $result = $DB->read("select sender, receiver from friends where status = 'accepted'
and (sender = '$id' or receiver = '$id')");

    $ids = [];
    if (is_array($result)) {
        foreach ($result as $row) {
            $ids[] = ($row['sender'] == $id) ? $row['receiver'] : $row['sender'];
        }
    }

    return $ids;
}

public function get_friends($id)
{
    $ids = $this->get_friend_ids($id);
    if (count($ids) == 0) {
        return false;
    }

    $DB = new Database();
    $list = "'" . implode("','", array_map('esc', $ids)) . "'";
    $result = $DB->read("select * from users where userid in ($list) order by first_name,
last_name");

    return is_array($result) ? $result : false;
}

public function count_friends($id)
{
    return count($this->get_friend_ids($id));
}

public function get_received_requests($id)
{
    if (!is_numeric($id)) {
        return false;
    }

    $DB = new Database();
    $id = esc($id);
    $result = $DB->read("select * from friends where receiver = '$id' and status =
'pending' order by id desc");

    return is_array($result) ? $result : false;
}

public function get_sent_requests($id)
{
    if (!is_numeric($id)) {
        return false;
    }

    $DB = new Database();
    $id = esc($id);
```

```
        $result = $DB->read("select * from friends where sender = '$id' and status = 'pending'
    order by id desc");

        return is_array($result) ? $result : false;
    }

    public function count_received_requests($id)
    {
        $requests = $this->get_received_requests($id);
        return is_array($requests) ? count($requests) : 0;
    }

    // privacy 'friends' = only friends, the owner and admins

    public function can_view_profile($profile_row, $viewer_id)
    {
        if (!is_array($profile_row)) {
            return false;
        }

        if (($profile_row['privacy'] ?? "public") != "friends") {
            return true;
        }

        if (empty($viewer_id)) {
            return false;
        }

        if ($profile_row['userid'] == $viewer_id || is_admin($viewer_id)) {
            return true;
        }

        return $this->are_friends($profile_row['userid'], $viewer_id);
    }
}
```

Fails classes/post.php

```
<?php
class Post {
    private $error = "";

    private $allowed_uploads = [
        "image/jpeg" => "jpg",
        "video/mp4" => "mp4",
    ];

    public function create_post($user_id, $data, $files, $owner = 0) {
        $this->error = "";

        if (!is_numeric($user_id) || $user_id <= 0) {
            return "You must be logged in to post!<br>";
        }

        $text = (isset($data['post']) && is_string($data['post'])) ? $data['post'] : "";

        // change_profile_image.php passes the file path as a string in $files
        // a real upload is always an array, so this can't be faked
        $is_image_post = is_string($files) && $files !== "" &&
    (isset($data['is_profile_image']) || isset($data['is_cover_image']));
        $has_upload = is_array($files) && isset($files['file']) && !empty($files['file']
    ['name']);

        if (trim($text) === "" && !$has_upload && !$is_image_post) {
            return "Please type something to post! <br>";
        }

        $myimage = "";
        $has_image = 0;
        $is_cover_image = 0;
        $is_profile_image = 0;

        if ($is_image_post) {
            $myimage = $files;
            $has_image = 1;
            if (isset($data['is_cover_image'])) $is_cover_image = 1;
            if (isset($data['is_profile_image'])) $is_profile_image = 1;
        } elseif ($has_upload) {
            $upload = store_upload($files['file'], $user_id, $this->allowed_uploads);
            if ($upload['error'] != "") {
```

```
            $this->error .= $upload['error'];
            return $this->error;
        }
        $myimage = $upload['path'];
        $has_image = 1;
    }

    $parent = 0;
    $parent_post = false;
    if (isset($data['parent']) && is_numeric($data['parent']) && $data['parent'] > 0) {
        $parent_post = $this->get_one_post($data['parent']);
        if (!$parent_post) {
            return "The post you are replying to no longer exists!<br>";
        }
        $parent = $parent_post['post_id'];
    }

    $post_id = $this->create_postid();

    $DB = new Database();
    $safe_user = esc($user_id);
    $safe_post = esc($text);
    $safe_image = esc($myimage);
    $safe_parent = esc($parent);
    $safe_owner = esc(is_numeric($owner) ? $owner : 0);

    $query = "insert into posts (post_id, user_id, post, image, has_image, is_cover_image,
is_profile_image, parent, owner) values ('$post_id', '$safe_user', '$safe_post',
'$safe_image', '$has_image', '$is_cover_image', '$is_profile_image', '$safe_parent',
'$safe_owner')";
    if (!$DB->save($query)) {
        return "Your post could not be saved, please try again.<br>";
    }

    // notify @mentions
    $tags = array();
    preg_match_all(MENTION_PATTERN, $text, $tags);

    if (isset($tags[2]) && is_array($tags[2])) {
        foreach (array_unique($tags[2]) as $tag) {
            $tag = esc($tag);
            $sql = "select userid from users where tag_name = '$tag' and (type is null or
type = '' or type = 'profile') limit 1";
            $tagged_user = $DB->read($sql);

            if (is_array($tagged_user)) {
                $tagged_userid = $tagged_user[0]['userid'];
                $tag_post = array('userid' => $tagged_userid, 'postid' => $post_id);
                if ($tagged_userid != $user_id) {
                    add_notification($user_id, "tag", $tag_post);
                }
            }
        }
    }

    if ($parent > 0) {
        $sql = "update posts set comments = comments + 1 where post_id = '$safe_parent'
limit 1";
        $DB->save($sql);

        // followers get it through content_i_follow
        if ($parent_post['user_id'] != $user_id) {
            add_notification($user_id, "comment", $parent_post);
        }

        // commenting on a post follows it
        content_i_follow($user_id, $parent_post);
    } else {
        content_i_follow($user_id, ['postid' => $post_id, 'user_id' => $user_id]);
    }

    return $this->error;
}

// $post has to be htmlspecialchars'd already
public function get_tags($post) {
    $DB = new Database();
    $cache = [];

    // @mentions
```

```
    $result = preg_replace_callback(MENTION_PATTERN, function ($m) use ($DB, &$cache) {
        $tag = $m[2];
        if (!array_key_exists($tag, $cache)) {
            $safe = esc($tag);
            $found = $DB->read("select userid, type from users where tag_name = '$safe'
limit 1");
            $cache[$tag] = is_array($found) ? $found[0] : false;
        }

        if ($cache[$tag]) {
            $path = (($cache[$tag]['type'] ?? "") == "group" ? "group/" : "profile/") .
$cache[$tag]['userid'];
            return $m[1] . "<a href='" . ROOT . $path . "' class='tag-link'>@" . $tag .
"</a>";
        }
        return $m[0];
    }, $post);

    if ($result !== null) {
        $post = $result;
    }

    // #hashtags
    $result = preg_replace_callback(HASHTAG_PATTERN, function ($m) {
        return $m[1] . "<a href='" . ROOT . "tags?tag=" . urlencode($m[2]) . "'
class='tag-link'>#" . $m[2] . "</a>";
    }, $post);

    if ($result !== null) {
        $post = $result;
    }

    return $post;
}

public function get_post($id, $post_type = "profile") {
    $DB = new Database();
    $id = esc($id);

    $query = "SELECT * FROM posts WHERE parent = 0 AND user_id = '$id' AND owner = 0 ORDER
BY id DESC LIMIT 10";

    if($post_type == "group") {
        $query = "SELECT * FROM posts WHERE parent = 0 AND owner = '$id' ORDER BY id DESC
LIMIT 10";
    }

    $data = $DB->read($query);
    return is_array($data) ? $data : false;
}

public function get_one_post($postid) {
    if(!is_numeric($postid)) return false;
    $DB = new Database();
    $postid = esc($postid);
    $result = $DB->read("select * from posts where post_id = '$postid' limit 1");
    if(is_array($result)) return $result[0];
    return false;
}

public function get_comments($parent_id) {
    if (!is_numeric($parent_id)) return false;
    $DB = new Database();
    $parent_id = esc($parent_id);
    $result = $DB->read("select * from posts where parent = '$parent_id' order by id
asc");
    return $result ? $result : false;
}

// also deletes replies, likes, notifications, follows and the file
public function delete_post($postid, $depth = 0) {
    if(!is_numeric($postid) || $depth > 50) return;

    $row = $this->get_one_post($postid);
    if (!$row) return;

    $DB = new Database();
    $id = esc($row['post_id']);

    // replies first, recursively
```

```
    $children = $DB->read("select post_id from posts where parent = '$id'");
    if (is_array($children)) {
        foreach ($children as $child) {
            $this->delete_post($child['post_id'], $depth + 1);
        }
    }

    $DB->save("delete from posts where post_id = '$id' limit 1");

    delete_upload_if_unused($row['image'] ?? "", $row['post_id']);

    $DB->save("delete from likes where contentid = '$id' and (type = 'post' or type =
'comment')");
    $DB->save("delete from notifications where content_id = '$id' and (content_type =
'post' or content_type = 'comment')");
    $DB->save("delete from content_i_follow where content_id = '$id' and (content_type =
'post' or content_type = 'comment')");

    if ($row['parent'] > 0) {
        $parent = esc($row['parent']);
        $DB->save("update posts set comments = GREATEST(comments - 1, 0) where post_id =
'$parent' limit 1");
    }
}

public function i_own_post($post_id, $osmos_userid) {
    if(!is_numeric($post_id)) return false;
    $DB = new Database();
    $post_id = esc($post_id);
    $result = $DB->read("select * from posts where post_id = '$post_id' limit 1");
    if(is_array($result) && $result[0]['user_id'] == $osmos_userid) return true;
    return false;
}

// author, parent post's author, group mods/admins/owner and site admins
public function can_delete_post($row, $user_id) {
    if (empty($user_id) || !is_array($row)) return false;

    if ($row['user_id'] == $user_id) return true;

    if (is_admin($user_id)) return true;

    if ($row['parent'] > 0) {
        $parent = $this->get_one_post($row['parent']);
        if ($parent && $parent['user_id'] == $user_id) return true;
    }

    if ($row['owner'] > 0) {
        $group_class = new Group();
        $group = $group_class->get_group($row['owner']);
        if ($group && group_access($user_id, $group, 'moderator')) return true;
    }

    return false;
}

// group posts follow the group, profile posts the author's privacy setting
public function can_view_post($row, $user_id) {
    static $profile_cache = [];

    if (!is_array($row)) return false;

    if (is_admin($user_id)) return true;

    if ($row['owner'] > 0) {
        $group_class = new Group();
        $group = $group_class->get_group($row['owner']);
        if ($group) {
            return group_can_view($user_id, $group);
        }
    }

    // comments follow their post
    if ($row['parent'] > 0) {
        $parent = $this->get_one_post($row['parent']);
        return $parent ? $this->can_view_post($parent, $user_id) : false;
    }

    $key = $row['user_id'] . "_" . $user_id;
    if (!array_key_exists($key, $profile_cache)) {
```

```
        $user_class = new User();
        $author = $user_class->get_user($row['user_id']);
        $profile_cache[$key] = $author ? $user_class->can_view_profile($author,
$user_id) : false;
    }

    return $profile_cache[$key];
}

public function like_post($id, $type, $osmos_userid) {
    $targets = ['post' => ['table' => 'posts', 'column' => 'post_id'], 'comment' =>
['table' => 'posts', 'column' => 'post_id'], 'user' => ['table' => 'users', 'column' =>
'userid']];
    if(!isset($targets[$type]) || !is_numeric($id) || empty($osmos_userid)) return;

    $table = $targets[$type]['table'];
    $type_id = $targets[$type]['column'];
    $id = esc($id);
    $DB = new Database();
    $result = $DB->read("select likes from likes where type='$type' && contentid = '$id'
limit 1");

    if(!empty($result)) {
        $likes = json_decode($result[0]['likes'], true);
        if(!is_array($likes)) $likes = [];
        $user_ids = array_column($likes, "user_id");

        if(!in_array($osmos_userid, $user_ids)) {
            $likes[] = ["user_id" => (string)$osmos_userid, "date" => date("Y-m-d
H:i:s")];
            $likes_string = esc(json_encode(array_values($likes)));
            $DB->save("update likes set likes = '$likes_string' where type='$type' &&
contentid = '$id' limit 1");
            $DB->save("update $table set likes = likes + 1 where $type_id = '$id' limit
1");

            if ($type == "post" || $type == "comment") {
                $single_post = $this->get_one_post($id);
                add_notification($osmos_userid, "like", $single_post);
            }
        } else {
            $key = array_search($osmos_userid, $user_ids);
            unset($likes[$key]);
            $likes_string = esc(json_encode(array_values($likes)));
            $DB->save("update likes set likes = '$likes_string' where type='$type' &&
contentid = '$id' limit 1");
            $DB->save("update $table set likes = GREATEST(likes - 1, 0) where $type_id =
'$id' limit 1");
        }
    } else {
        $arr2 = [];
        $arr2[] = ["user_id" => (string)$osmos_userid, "date" => date("Y-m-d H:i:s")];
        $likes = esc(json_encode($arr2));
        $DB->save("insert into likes (type, contentid, likes) values ('$type', '$id',
'$likes')");
        $DB->save("update $table set likes = likes + 1 where $type_id = '$id' limit 1");

        if ($type == "post" || $type == "comment") {
            $single_post = $this->get_one_post($id);
            add_notification($osmos_userid, "like", $single_post);
        }
    }
}

public function get_likes($id, $type) {
    $DB = new Database();
    $type = esc($type);
    if(is_numeric($id)) {
        $id = esc($id);
        $result = $DB->read("select likes from likes where type='$type' && contentid =
'$id' limit 1");
        if(is_array($result)) {
            $likes = json_decode($result[0]['likes'], true);
            return (is_array($likes) && count($likes) > 0) ? $likes : false;
        }
    }
    return false;
}

public function user_liked($id, $type, $user_id) {
```

```
        if (empty($user_id)) return false;
        $likes = $this->get_likes($id, $type);
        if (!is_array($likes)) return false;
        return in_array((string)$user_id, array_map('strval', array_column($likes,
    "user_id")), true);
    }

    private function create_postid() {
        return generate_unique_id("posts", "post_id");
    }

    // Photos tab, no group posts or videos
    public function get_images($id) {
        $id = esc($id);
        $DB = new Database();
        $result = $DB->read("select * from posts where has_image = 1 && user_id = '$id' &&
    owner = 0 && image NOT LIKE '%.mp4' order by id desc limit 30");
        return $result ? $result : false;
    }

    // $postid has to come from the url, never from the form
    public function edit_post($postid, $data, $files) {
        $this->error = "";

        $row = $this->get_one_post($postid);
        if (!$row) {
            return "No such post was found!<br>";
        }

        $text = (isset($data['post']) && is_string($data['post'])) ? $data['post'] : "";

        $upload = store_upload($files['file'] ?? null, $row['user_id'], $this-
    >allowed_uploads);
        if ($upload['error'] != "") {
            $this->error .= $upload['error'];
            return $this->error;
        }

        if (trim($text) === "" && $upload['path'] === "" && empty($row['image'])) {
            return "Please type something to post! <br>";
        }

        $DB = new Database();
        $post = esc($text);
        $safe_id = esc($row['post_id']);

        if ($upload['path'] !== "") {
            $myimage = esc($upload['path']);
            $query = "update posts set post = '$post', image = '$myimage', has_image = '1'
    where post_id = '$safe_id' limit 1";
            $DB->save($query);

            delete_upload_if_unused($row['image'] ?? "", $row['post_id']);
        } else {
            $query = "update posts set post = '$post' where post_id = '$safe_id' limit 1";
            $DB->save($query);
        }

        return $this->error;
    }
}
```

Fails ajax.php

```
<?php
// likes and @tag suggestions
// csrf is checked in autoload
include_once("classes/autoload.php");

$me = $_SESSION['osmos_userid'] ?? 0;

if (isset($_POST['link'])) {

    header("Content-Type: application/json");

    if (empty($me)) {
        echo json_encode(["action" => "login"]);
        die;
    }
```

```
    // the link looks like ROOT/like/post/123?token=...
    $data_link = is_string($_POST['link']) ? $_POST['link'] : "";
    $path = parse_url($data_link, PHP_URL_PATH);
    $parts = split_url_from_string((string)$path);

    $type = null;
    $id = null;
    $key = array_search('like', $parts);
    if ($key !== false) {
        $type = $parts[$key + 1] ?? null;
        $id = $parts[$key + 2] ?? null;
    }

    if (in_array($type, ["post", "comment"], true) && is_numeric($id)) {
        $post = new Post();
        $row = $post->get_one_post($id);

        if (!$row || !$post->can_view_post($row, $me)) {
            echo json_encode(["action" => "error", "message" => "You can't like this post."]);
            die;
        }

        // like_post() sends the notification
        $post->like_post($id, $type, $me);

        $likes = $post->get_likes($id, $type);
        $likes_count = is_array($likes) ? count($likes) : 0;
        $liked = $post->user_liked($id, $type, $me);

        $obj = (object)[];
        $obj->likes = $likes_count;
        $obj->liked = $liked;
        $obj->info = likes_info_text($likes_count, $liked);
        $obj->action = "like_post";
        $obj->id = $id;

        echo json_encode($obj);
    }
    die;
}

if (isset($_POST['tag_search'])) {

    if (empty($me) || !is_string($_POST['tag_search'])) {
        die;
    }

    $query_tag = trim($_POST['tag_search']);
    if (!preg_match("/^[a-zA-Z0-9_]*$/", $query_tag)) {
        die;
    }

    $DB = new Database();
    // escape % and _ for LIKE
    $like = esc(addcslashes($query_tag, "%_"));

    // people only, best match first
    $sql = "select tag_name, first_name, last_name from users
            where (type is null or type = '' or type = 'profile')
            and blocked = 0
            and tag_name != ''
            and (tag_name like '%$like%' or first_name like '$like%' or last_name like '$like
    %')
            order by (tag_name like '$like%') desc, tag_name asc
            limit 6";
    $result = $DB->read($sql);

    if (is_array($result)) {
        foreach ($result as $row) {
            $tag = htmlspecialchars($row['tag_name'], ENT_QUOTES);
            $name = htmlspecialchars($row['first_name'] . " " . $row['last_name']);
            echo "<div class='tag-suggestion' data-tag='" . $tag . "'>";
            echo "<strong>@" . $tag . "</strong> <span class='tag-suggestion-name'>(" .
    $name . ")</span>";
            echo "</div>";
        }
    }
    die;
}
```